

Université de Sfax

Institut Supérieur d'Informatique
et de Multimédia de Sfax

MÉMOIRE

Présenté à

*L'Institut Supérieur d'Informatique
et de Multimédia de Sfax*

En vue de la validation de

DIPLÔME NATIONAL D'INGÉNIEUR EN INFORMATIQUE

Intitulé

*Systeme intelligent
de suivi de livraison*

élaboré par

Mohamed Aziz Hadjkacem

Soutenu le XX/06/2026 devant le jury composé de

M. XXX

M. XXX

M. Bassem Bouaziz

M. XXX

Président

Rapporteur

Encadrant académique

Encadrant professionnel

Dédicace

Je dédie ce travail, fruit de plusieurs années d'efforts et d'apprentissage, à toutes les personnes qui m'ont accompagné jusqu'ici.

Je dédie ce travail à mes parents pour leur soutien constant, leur patience et les sacrifices qu'ils ont faits pour me permettre d'arriver là où je suis aujourd'hui. Sans eux, rien de tout cela n'aurait été possible.

À ma famille, pour ses encouragements et sa présence à chaque étape de ce chemin.

À mes amis, pour les bons moments partagés, pour leur aide et leur soutien dans les périodes difficiles comme dans les bonnes.

À mes professeurs, à l'ISIMS et auparavant, pour les connaissances qu'ils m'ont transmises et qui ont posé les bases de ce travail.

Enfin, je dédie ce mémoire à moi-même, pour la persévérance et le travail qu'il a représenté tout au long du projet.

Mohamed Aziz

■ Remerciements

La réalisation de ce projet de fin d'études n'aurait pas été possible sans le soutien, la confiance et l'accompagnement de plusieurs personnes, à qui j'exprime ma profonde gratitude.

À mon encadrant académique, **M. Bassem Bouaziz**, pour la qualité de son encadrement, la pertinence de ses conseils et sa disponibilité tout au long de ce travail. Son exigence et sa rigueur ont été une véritable source de progression.

À mon encadrant professionnel, [Nom à compléter], pour son accompagnement et ses orientations techniques tout au long de ce projet.

À la direction et aux équipes d'**ASM (All Soft Multimedia)**, pour m'avoir accueilli dans un environnement de travail stimulant, pour la confiance accordée et pour les moyens mis à ma disposition tout au long de cette expérience.

Aux membres du jury, pour l'honneur qu'ils me font en évaluant ce travail, ainsi que pour l'intérêt et les remarques qui contribueront à son amélioration.

Mes remerciements vont également à l'ensemble du corps enseignant de l'**ISIMS**, pour la formation solide et les compétences qui ont constitué la base de ce projet et de mon parcours académique.

Enfin, j'exprime ma gratitude à toutes les personnes qui, de près ou de loin, ont contribué à la réussite de ce travail.



Table des matières

Dédicace	i
Remerciements	ii
Liste des abréviations	xii
INTRODUCTION GÉNÉRALE	1
1 Cadre général du projet	3
1.1 Introduction	3
1.2 Présentation de l'organisme d'accueil	3
1.3 Présentation du projet	4
1.3.1 Contexte du projet	4
1.3.2 Problématique	4
1.3.3 Objectifs du projet	4
1.3.4 Périmètre du projet	5
1.4 Étude et critique de l'existant	5
1.4.1 Aperçu des outils existants	5
1.4.2 Synthèse comparative et solution proposée	6
1.5 Méthodologie de développement	7
1.5.1 Présentation de Scrum	8
1.5.2 Organisation du travail et outils	8
1.6 Conclusion	9
2 Sprint 0 — Initialisation, besoins et étude technique	10
2.1 Introduction	10
2.2 Identification des acteurs	10
2.3 Analyse et spécification des besoins	12
2.3.1 Besoins fonctionnels	12
2.3.2 Besoins non fonctionnels	13
2.3.3 Diagramme global des cas d'utilisation	14
2.3.4 Diagramme de classes global	16

2.4	Gestion du projet avec Scrum	18
2.4.1	Product Backlog	18
2.4.2	Planification des sprints	23
2.5	Étude technique	23
2.5.1	Environnement matériel	23
2.5.2	Environnement logiciel de développement	24
2.5.3	Technologies retenues	24
2.6	Conclusion	25
3	Architecture et conception générale	27
3.1	Introduction	27
3.2	Une application unique ou plusieurs services	27
3.3	Vue d'ensemble	29
3.3.1	La passerelle	30
3.3.2	Les services	31
3.3.3	Les composants d'infrastructure	32
3.4	Du modèle de classes aux bases de données	32
3.4.1	Ce que devient une relation entre deux services	33
3.4.2	L'évolution du schéma	33
3.5	L'isolation des entreprises clientes	33
3.5.1	Trois façons de séparer, une seule retenue	34
3.5.2	Le trajet d'une requête	34
3.5.3	Le refus plutôt que le repli	35
3.5.4	Le cas des traitements asynchrones	35
3.6	Les échanges entre services	35
3.6.1	Quand un message ne passe pas	36
3.7	La mise en marche	37
3.8	Conclusion	37
4	Sprint 1 — Socle d'identité, d'autorisation et de multi-tenance	39
4.1	Introduction	39
4.2	Objectifs du sprint et planification des tâches	39
4.3	Concepts et mécanismes utilisés	41
4.4	Analyse et spécification des besoins	43
4.4.1	Diagramme de cas d'utilisation du sprint	43
4.4.2	Description détaillée des cas d'utilisation	43
4.5	Modélisation conceptuelle	45
4.5.1	Diagramme de classes	45
4.5.2	Diagrammes de séquence	46
4.6	Réalisation	48

4.7	Tests et validation	49
4.7.1	Scénarios de validation	49
4.7.2	Tests automatisés	49
4.8	Conclusion	51
5	Sprint 2 — Intégration ERP agnostique	52
5.1	Introduction	52
5.2	Objectifs du sprint et planification des tâches	52
5.2.1	Objectifs attendus	52
5.2.2	Sprint backlog	53
5.3	Concepts et mécanismes utilisés	54
5.4	Analyse et spécification des besoins	55
5.4.1	Diagramme de cas d'utilisation du sprint	55
5.4.2	Description détaillée des cas d'utilisation	56
5.5	Modélisation conceptuelle	58
5.5.1	Diagramme de classes	58
5.5.2	Diagrammes de séquence	59
5.6	Réalisation	61
5.7	Tests et validation	63
5.7.1	Scénarios de validation	63
5.7.2	Tests automatisés	63
5.8	Conclusion	65
6	Sprint 3 — Planification, exécution et suivi temps réel	66
6.1	Introduction	66
6.2	Objectifs du sprint et planification des tâches	66
6.3	Concepts et mécanismes utilisés	68
6.4	Analyse et spécification des besoins	69
6.4.1	Diagramme de cas d'utilisation du sprint	69
6.4.2	Description détaillée des cas d'utilisation	71
6.5	Modélisation conceptuelle	73
6.5.1	Diagramme de classes	73
6.5.2	Diagrammes de séquence	75
6.5.3	L'évaluation des échéances	79
6.6	Réalisation	80
6.7	Tests et validation	82
6.7.1	Scénarios de test	82
6.7.2	Tests automatisés	84
6.8	Conclusion	85

7	Sprint 4 — Application mobile chauffeur et mode hors ligne	86
7.1	Introduction	86
7.2	Objectifs du sprint et planification des tâches	86
7.3	Concepts et mécanismes utilisés	88
7.4	Analyse et spécification des besoins	89
7.4.1	Diagramme de cas d'utilisation du sprint	89
7.4.2	Description détaillée des cas d'utilisation	91
7.5	Modélisation conceptuelle	94
7.5.1	Diagramme de classes	94
7.5.2	Diagramme de séquence	95
7.6	Réalisation	97
7.7	Tests et validation	102
7.7.1	Scénarios de test	102
7.7.2	Tests automatisés	103
7.8	Conclusion	105
8	Sprint 5 — Exploitation, résilience et assistant interne	106
8.1	Introduction	106
8.2	Objectifs du sprint et planification des tâches	106
8.2.1	Objectifs attendus	106
8.2.2	Sprint backlog	107
8.3	Concepts et mécanismes utilisés	109
8.4	Analyse et spécification des besoins	110
8.4.1	Diagramme de cas d'utilisation du sprint	110
8.4.2	Description détaillée des cas d'utilisation	111
8.5	Modélisation conceptuelle	112
8.5.1	Diagramme de classes	112
8.5.2	Diagramme de séquence	114
8.5.3	Le chemin d'une question	115
8.6	Réalisation	116
8.7	Tests et validation	117
8.7.1	Scénarios de test	117
8.7.2	Tests automatisés	118
8.7.3	Qualité de la recherche documentaire	119
8.8	Conclusion	120
9	Validation globale, qualité et déploiement	122
9.1	Introduction	122
9.2	La pyramide de tests	122
9.3	La chaîne d'intégration continue	123

9.4	Le connecteur ERP éprouvé sur deux versions	124
9.5	L'analyse des dépendances	124
9.6	Le déploiement	124
9.7	Conclusion	125
Conclusion générale et perspectives		126
Bibliographie		128
Webographie		129



Table des figures

1.1	Logo de l'entreprise ASM (All Soft Multimedia) [ASM 26]	3
1.2	Logo d'Onfleet [ONF 26a]	5
1.3	Logo de Bringg [BRI 26a]	6
2.1	Diagramme global des cas d'utilisation	15
2.2	Diagramme de classes global	17
2.3	Outils de développement et de collaboration	24
2.4	Technologies retenues pour la plateforme	25
3.1	Vue d'ensemble de l'architecture	30
3.2	Répartition des données entre les services	32
3.3	Le choix du schéma, de la requête jusqu'à la base	34
3.4	Topologie de la messagerie	36
4.1	Matrice des permissions par rôle	42
4.2	Diagramme de cas d'utilisation du sprint 1	43
4.3	Diagramme de classes du module <code>asm-tenant-core</code>	46
4.4	Séquence d'authentification OpenID Connect et d'accès à une ressource	47
4.5	Création d'un compte et attribution d'un rôle	48
4.6	Référentiel des chauffeurs	48
4.7	Accès refusé à une action non permise	49
4.8	Contrôle en CI de l'unicité de la politique d'autorisation	51
5.1	Diagramme de cas d'utilisation du sprint 2	56
5.2	Ports et adaptateurs de l'intégration ERP	58
5.3	Séquence de synchronisation ERP et reprise sur panne	59
5.4	Automate d'états d'un événement de synchronisation	60
5.5	Configuration de la connexion au progiciel	61
5.6	Détection des capacités du progiciel	61
5.7	Aperçu d'une commande avant l'import	62
5.8	Import des commandes depuis le progiciel	62
5.9	File morte des synchronisations en échec	62
5.10	Exécution en CI de la suite ERP contre deux versions d'Odoo	64

6.1	Diffusion temps réel entre plusieurs instances du service	69
6.2	Diagramme de cas d'utilisation du sprint 3	70
6.3	Classes de conception du sprint 3	74
6.4	Séquence de construction, d'optimisation et de validation d'une tournée	76
6.5	Machine à états d'une livraison	78
6.6	Les quatre phases d'échéance d'une livraison	79
6.7	Planificateur de tournées	80
6.8	Aperçu d'optimisation, avec les gains et les fenêtres proposées	80
6.9	Fiche d'une tournée en cours	81
6.10	Centre de dispatch	81
6.11	Réaffectation d'une livraison, chauffeurs classés par temps d'accès	82
6.12	Traitement d'une demande de retour	82
7.1	Diagramme de cas d'utilisation du sprint 4	90
7.2	Classes de conception du sprint 4	94
7.3	Séquence du dépôt d'une preuve, avec mise en file et rejeu	96
7.4	Tournée du jour sur l'application chauffeur	97
7.5	Preuve de livraison : le formulaire, puis le décompte des manques	98
7.6	Centre de synchronisation : une écriture en attente, puis la même refusée par le serveur	99
7.7	Code de transfert de garde : QR, jeton et compte à rebours	100
7.8	Déclaration de la remise de caisse par le chauffeur	101
7.9	Comptage de la remise et arbitrage de l'écart au bureau	102
8.1	Diagramme de cas d'utilisation du sprint 5	110
8.2	Diagramme de classes du sprint 5	113
8.3	Séquence de synchronisation avec le progiciel et reprise après échec	114
8.4	Le chemin d'une question, de l'indexation à la réponse citée	115
8.5	Tableau de bord : indicateurs de la période et comparaison à la précédente	116
8.6	Console Santé Système : points d'attention, commandes à reprendre et journal des synchronisations	117
8.7	Assistant : une réponse tirée des données en direct	117
9.1	Rapport de tests agrégé par la chaîne d'intégration	123
9.2	Les cinq étapes de la chaîne d'intégration	124
9.3	État des conteneurs sur le serveur de déploiement	125

Liste des tableaux

1.1	Fiche d'identité d'ASM	4
1.2	Comparatif des solutions existantes, d'après leur documentation publique . . .	7
1.3	Ce que chaque méthode suppose, au regard des contraintes de ce projet	8
2.1	Acteurs du système et périmètre de chacun	11
2.2	Besoins non fonctionnels	13
2.3	Product Backlog du projet	18
2.4	Planification des sprints	23
2.5	Environnement matériel de développement	24
2.6	Outils de développement et de collaboration	24
2.7	Technologies retenues et motif du choix	25
3.1	Application unique ou services séparés	27
3.2	Les services et leurs responsabilités	31
3.3	Les conteneurs de l'installation	37
4.1	Sprint backlog — sprint 1	39
4.2	Cas d'utilisation «S'authentifier»	44
4.3	Cas d'utilisation «Créer un compte utilisateur»	44
4.4	Cas d'utilisation «Ajouter un chauffeur»	45
4.5	TC-S1-03 — Isolation des données entre deux entreprises	49
4.6	Tests automatisés du sprint 1	49
5.1	Sprint backlog — sprint 2	53
5.2	Cas d'utilisation «Configurer et certifier la connexion ERP»	56
5.3	Cas d'utilisation «Mapper les champs vers le contrat canonique»	57
5.4	Cas d'utilisation «Importer les commandes»	57
5.5	TC-S2-01 — Agnosticité vis-à-vis du progiciel	63
5.6	TC-S2-03 — Reprise d'une synchronisation en panne	63
5.7	Tests automatisés du sprint 2 (extrait)	64
6.1	Sprint backlog — sprint 3	66
6.2	Cas d'utilisation «Construire une tournée»	71
6.3	Cas d'utilisation «Réaffecter une livraison à un autre chauffeur»	71

6.4	Cas d'utilisation «Demander un retour»	72
6.5	TC-S3-01 — Respect de la précédence dans l'optimisation	83
6.6	TC-S3-03 — Mesure des délais et cas du multi-dépôt	83
6.7	TC-S3-04 — Réaffectation d'une livraison déjà chargée	83
6.8	Tests automatisés du sprint 3	84
7.1	Sprint backlog — sprint 4	86
7.2	Cas d'utilisation «Déposer une preuve de livraison»	91
7.3	Cas d'utilisation «Travailler hors réseau»	92
7.4	Cas d'utilisation «Transférer la garde d'un colis»	93
7.5	TC-S4-01 — Dépôt de preuve sans réseau, puis rejeu	102
7.6	TC-S4-02 — Une même preuve envoyée deux fois	103
7.7	TC-S4-05 — Encaissement, remise et arbitrage de l'écart	103
7.8	Tests automatisés du sprint 4	104
8.1	Sprint backlog — sprint 5	107
8.2	Cas d'utilisation «Consulter la console Santé Système»	111
8.3	Cas d'utilisation «Interroger l'assistant»	112
8.4	TC-S5-01 — Une commande en échec repart sans doubler le mouvement de stock	118
8.5	TC-S5-02 — L'assistant ne dépasse pas les droits de celui qui l'interroge	118
8.6	Tests automatisés du sprint 5	118
8.7	Qualité de la recherche sur les 31 questions de référence	120
9.1	Répartition des tests automatisés	122

Liste des abréviations

API	Application Programming Interface
ERP	Enterprise Resource Planning
JWT	JSON Web Token
OIDC	OpenID Connect
RBAC	Role-Based Access Control
SaaS	Software as a Service
SLA	Service Level Agreement
UML	Unified Modeling Language

INTRODUCTION GÉNÉRALE

Dans une entreprise de distribution, la livraison ne s'arrête pas à la sortie de l'entrepôt : elle continue jusqu'à la porte du client, souvent hors de portée du système d'information. Un dispatcheur doit affecter des commandes à des livreurs, suivre leur avancement, réagir à un incident, tout en gardant l'ERP, qu'il s'agisse d'Odoo, d'ERPNext ou d'un autre, synchronisé avec ce qui se passe réellement sur le terrain. Ce dernier maillon, le *dernier kilomètre*, est celui où l'information se perd le plus facilement : un bon de livraison papier égaré, un stock qui ne se met pas à jour, une preuve de remise introuvable en cas de litige.

Les outils du marché répondent rarement à ce problème dans son ensemble. Un logiciel de tournées générique n'a pas connaissance des commandes de l'ERP et impose de les ressaisir ; un module fourni par l'éditeur de l'ERP enferme, à l'inverse, l'entreprise dans cet éditeur précis et ne suit plus si elle en change. Et la plupart de ces outils supposent une connexion permanente, alors qu'un livreur en zone rurale ou en sous-sol de parking n'en a pas toujours une.

Ce projet de fin d'études, mené au sein de l'entreprise **ASM (All Soft Multimedia)**, elle-même éditrice d'un progiciel de gestion, part de ce constat. Le travail réalisé, un **Système intelligent de suivi de livraison**, repose sur deux partis pris structurants : un adaptateur qui dialogue avec l'ERP par un contrat commun plutôt que par un développement propre à chacun, et une isolation des données par entreprise cliente qui permet à une seule plateforme de servir plusieurs d'entre elles. Autour de ce socle s'organisent la planification des tournées, le suivi en temps réel et une application chauffeur qui reste utilisable sans réseau. Le système couvre ainsi tout le trajet d'une commande : de son import depuis l'ERP jusqu'à la preuve de livraison qui y retourne.

Ce rapport retrace la démarche suivie pour construire ce système, chapitre après chapitre :

- **Chapitre 1** : présente le cadre général du projet, l'entreprise d'accueil et l'étude de l'existant.
- **Chapitre 2** : présente le sprint 0, consacré à l'analyse des besoins, à la planification Scrum et à l'étude technique.
- **Chapitre 3** : détaille l'architecture et la conception générale du système.

- **Chapitre 4** : détaille le sprint 1, consacré au socle d'identité, d'autorisation et aux référentiels d'exploitation.
- **Chapitre 5** : décrit le sprint 2, consacré à l'intégration ERP agnostique.
- **Chapitre 6** : expose le sprint 3, consacré à la planification des tournées et à leur suivi en temps réel.
- **Chapitre 7** : présente le sprint 4, consacré à l'application mobile chauffeur et au mode hors ligne.
- **Chapitre 8** : présente le sprint 5, consacré à l'exploitation, à la résilience et à l'assistant interne.
- **Chapitre 9** : présente la validation globale, la qualité et le déploiement du système.

Une conclusion générale reviendra, à la fin de ce rapport, sur ce qui a été construit et sur ce qui reste à faire évoluer.

1.1 Introduction

Ce chapitre pose le cadre dans lequel s'inscrit ce projet de fin d'études. Nous présentons d'abord l'entreprise d'accueil, puis le contexte, la problématique, les objectifs et le périmètre du projet. Nous analysons ensuite les solutions existantes afin de situer notre approche par rapport à elles. Nous terminons par la méthodologie de développement retenue.

1.2 Présentation de l'organisme d'accueil

Ce projet a été réalisé au sein de l'entreprise **ASM (All Soft Multimedia)**, une société de services numériques tunisienne fondée en 2007. Éditrice et intégratrice de solutions logicielles, ASM conçoit des sites web, des applications mobiles, des systèmes de point de vente ainsi qu'un progiciel de gestion intégré propriétaire, **DUX**, distribué auprès de plus de 4500 clients. Son siège se trouve à Sfax, avec une présence à Sousse et Tunis ainsi que des filiales à Alger et à Dakar [ASM 26].



FIG. 1.1 : Logo de l'entreprise ASM (All Soft Multimedia) [ASM 26]

Ce positionnement est directement pertinent pour ce projet : ASM n'est pas seulement le commanditaire, elle est elle-même éditrice d'un ERP. Une plateforme de suivi de livraison qui s'intègre à un ERP sans s'y enfermer profite à ses clients quel que soit le progiciel qu'ils exploitent, et son architecture, bâtie sur un contrat d'adaptateur commun à tout ERP, est conçue pour qu'un autre système puisse s'y brancher sans remettre en cause le reste de la plateforme.

TAB. 1.1 : Fiche d'identité d'ASM

Nom de l'entreprise	ASM (All Soft Multimedia)
Secteur d'activité	Édition et intégration de logiciels : ERP (DUX), sites web, applications mobiles, systèmes de point de vente
Date de création	2007
Siège social	Sfax, Tunisie
Présence	Sfax, Sousse, Tunis (Tunisie); Alger (Algérie); Dakar (Sénégal)
Site web	https://asm-tunisie.com

1.3 Présentation du projet

1.3.1 Contexte du projet

Un ERP, quel qu'il soit, gère efficacement les commandes, les stocks et la facturation, mais s'arrête à la porte de l'entrepôt. Tout ce qui se passe ensuite, l'affectation d'un livreur, le suivi de sa tournée, la collecte de la preuve de remise et un éventuel retour, reste hors de sa portée, ou suppose de développer un module spécifique à chaque ERP concerné. Or les entreprises que sert ASM n'exploitent pas toutes le même système, et ce choix ne devrait pas déterminer si elles peuvent, ou non, suivre leurs livraisons.

1.3.2 Problématique

La problématique de ce projet peut être résumée ainsi :

Comment concevoir une plateforme de suivi de livraison exploitable par plusieurs entreprises clientes, indépendante de l'ERP utilisé par chacune, et capable de fonctionner sur le terrain même en l'absence de connexion réseau ?

1.3.3 Objectifs du projet

L'objectif principal est de couvrir l'ensemble du cycle de vie d'une livraison, depuis l'import de la commande jusqu'à la preuve de sa remise, à travers :

- une architecture **multi-tenant**, où chaque entreprise cliente dispose de ses propres données isolées au sein d'une même plateforme.
- une intégration ERP **agnostique**, opérationnelle sur Odoo et ERPNext par un adaptateur commun plutôt qu'un développement spécifique à chacun, et conçue pour qu'un autre ERP puisse s'y brancher sans toucher au reste du système.
- une application mobile chauffeur fonctionnant en **mode hors ligne**, pour que la perte de

réseau sur le terrain n’interrompt pas la livraison.

- un suivi en **temps réel** des tournées, de la planification jusqu’à la remise au client final.
- une **chaîne d’encaissement** traitée comme un circuit de responsabilité, de la collecte des espèces à la porte du client jusqu’à leur remise au dépôt et l’arbitrage des écarts.
- un **assistant interne** auquel un opérateur pose ses questions en langage naturel, et qui répond sur l’activité de son entreprise sans jamais lui donner accès à ce que son rôle ne l’autorise pas à consulter.

1.3.4 Périmètre du projet

Le projet couvre le cycle complet d’une livraison, de l’import à la planification de tournée, puis l’exécution terrain, la preuve de livraison, l’encaissement et les retours, pour un nombre quelconque d’entreprises clientes. Il se limite au dernier kilomètre : la gestion des stocks, la facturation et la production restent du ressort de l’ERP, que la plateforme consulte et met à jour sans s’y substituer.

1.4 Étude et critique de l’existant

1.4.1 Aperçu des outils existants

Avant de concevoir la solution, nous avons examiné les outils déjà utilisés pour le suivi de livraison du dernier kilomètre.



FIG. 1.2 : Logo d’Onfleet [ONF 26a]

Onfleet [ONF 26a] est une plateforme SaaS de gestion de tournées et de suivi de chauffeurs, reconnue pour son optimisation d’itinéraires et son suivi en temps réel. Elle ne s’intègre toutefois à un ERP que par des connecteurs génériques ou un développement sur mesure. Elle propose un mode hors ligne, mais désactivé par défaut : sans lui, un chauffeur ne dispose que de deux minutes pour démarrer ou terminer une tâche avant d’être bloqué faute de réseau [ONF 26b]. Sa documentation publique ne décrit pas de mécanisme d’encaissement en espèces ni de remise de caisse.



FIG. 1.3 : Logo de Bringg [BRI 26a]

Bringg [BRI 26a] est une plateforme de logistique de livraison orientée grands comptes, proposant orchestration de flotte et visibilité client. Son application chauffeur reste utilisable hors ligne pour les tâches déjà assignées [BRI 26b], et son API prévoit la prise d'un paiement par le livreur ainsi qu'un montant restant dû sur la commande [BRI 26c]. Sa richesse fonctionnelle a pour contrepartie une mise en œuvre lourde, pensée pour un déploiement par ERP ou canal de vente plutôt que pour une intégration légère et multi-ERP.

Le module de livraison natif d'un ERP (par exemple Odoo Inventory) constitue la troisième approche possible. Il a l'avantage d'être déjà connecté aux données de commande et de stock, mais il enferme l'entreprise dans cet ERP précis : le changer, ou en exploiter plusieurs à la fois, ce qui est un cas concret chez les clients d'ASM, n'est pas pris en charge.

1.4.2 Synthèse comparative et solution proposée

Le fonctionnement hors connexion, longtemps distinctif, est aujourd'hui proposé par les plateformes matures du marché. Ce qui les sépare de notre approche tient à deux points : la profondeur de l'intégration ERP, générique chez elles alors que le projet vise un contrat commun réellement implémenté sur plusieurs ERP, et le traitement de l'encaissement à la livraison. Sur ce second point, la distinction porte moins sur la collecte du paiement, que Bringg documente à travers une action chauffeur de prise de paiement et un montant restant dû, que sur ce qui vient après : la remise de la caisse du livreur, son comptage contradictoire et l'arbitrage des écarts. Ce maillon est déterminant sur le marché tunisien, où le paiement en espèces à la remise reste le mode dominant.

Un troisième écart tient à la nature de l'assistance intégrée. Onfleet a doté son tableau de bord d'un assistant conversationnel en avril 2026, inclus dans tous ses abonnements ; sa documentation le présente comme une aide à l'usage du produit : il répond aux questions de prise en main et de dépannage [ONF 26c]. Bringg, de son côté, ne documente aucun assistant conversationnel et concentre son intelligence artificielle sur la prédiction et l'optimisation des tournées [BRI 26d]. Aucun des deux ne documente donc un assistant capable d'interroger les données d'exploitation du client en langage naturel.

C'est précisément la fonction retenue ici, avec deux garde-fous qui en font autre chose qu'un agent conversationnel branché sur une base. D'abord, l'assistant **hérite des droits de celui qui l'interroge** : il ne dispose d'aucun compte propre, et deux utilisateurs de rôles différents

n’obtiennent pas la même réponse à la même question. Ensuite, il **ne peut que lire** : aucune de ses opérations ne modifie l’état du système. Il cite enfin ses sources, de sorte que chaque réponse reste vérifiable par celui qui la reçoit.

TAB. 1.2 : Comparatif des solutions existantes, d’après leur documentation publique

Solution	Multi-ERP	Versions ERP	Remise de caisse	Hors ligne	Assistant sur les données	Limite principale
Onfleet	Partiel	Sans objet	Non	Option	Support produit	Intégration ERP superficielle
Bringg	Partiel	Sans objet	Partiel	Oui	Non documenté	Mise en œuvre lourde, pensée par ERP
Module ERP natif	Non	Non	Partiel	Non	Non	Enferme l’entreprise dans un seul ERP
Solution proposée	Oui	Oui	Oui	Oui	Oui	—

La colonne « versions ERP » mérite un mot. Un progiciel évolue, et ses versions majeures modifient la structure des données qu’il expose : une intégration construite sur une version peut cesser de fonctionner chez un client qui met la sienne à jour. Les plateformes étudiées échappent à cette difficulté en restant à la surface, à travers une API générique, d’où la mention « sans objet » ; un module natif la subit au contraire de plein fouet, puisqu’il doit être porté à chaque version. S’intégrer en profondeur tout en résistant aux montées de version est donc une exigence à part entière, dont le chapitre 5 détaille le traitement.

Ce constat oriente la conception du système autour de quatre partis pris : un adaptateur ERP commun plutôt qu’un développement par ERP, une intégration qui s’adapte à la version rencontrée chez le client plutôt que d’en supposer une seule, une isolation des données par entreprise cliente plutôt qu’un déploiement dédié à chacune, et une chaîne d’encaissement traitée comme un circuit de responsabilité, où le livreur déclare la somme remise, un tiers la compte et un responsable arbitre les écarts, plutôt que comme un simple montant saisi à la livraison.

1.5 Méthodologie de développement

Le projet a été mené par un développeur unique, sur un périmètre fonctionnel qui s’est précisé au fil de l’avancement, notamment sur l’étendue exacte de l’intégration multi-ERP. Le choix d’une méthode s’est donc fait sur ce que chacune exige de celui qui l’applique, plutôt que sur

ses mérites en général. Le tableau 1.3 confronte les trois approches envisagées aux quatre contraintes réelles de ce projet.

TAB. 1.3 : Ce que chaque méthode suppose, au regard des contraintes de ce projet

Contrainte du projet	Cycle en V	RUP	Scrum
Le périmètre s'est précisé en cours de route	Exige un cahier des charges arrêté avant de commencer	Révision possible, au prix d'un changement de phase	Le reste à faire est réexaminé à chaque cycle
Le projet est mené par une seule personne	Suppose des intervenants distincts par phase	Prévoit de nombreux artefacts et disciplines	Trois rôles, cumulables par une même personne
Un résultat montrable était attendu tôt	Rien d'utilisable avant la fin	Prototype à la fin de chaque phase	Un incrément fonctionnel par cycle
L'encadrement demandait un point régulier	Jalons espacés	Revue de phase	Revue et rétrospective toutes les trois à cinq semaines

Le choix s'est porté sur la méthode agile **Scrum**, pour ses cycles courts et sa capacité à réévaluer les priorités à chaque sprint plutôt qu'une seule fois en début de projet.

1.5.1 Présentation de Scrum

Scrum [SCH 20] découpe le projet en cycles courts de durée fixe, appelés *sprints*, chacun devant produire quelque chose d'utilisable plutôt qu'une étape intermédiaire. Ce qui reste à faire est tenu dans une liste unique et ordonnée, le *Product Backlog*, dont on détache au début de chaque cycle ce que ce cycle peut absorber. Ce qui a rendu la méthode utile ici tient surtout à la fin de cycle : la revue oblige à montrer ce qui tourne réellement, et la rétrospective à corriger la façon de travailler avant d'engager la suite. Le périmètre de ce projet s'étant précisé en cours de route, cette réévaluation régulière valait mieux qu'un plan arrêté au départ.

1.5.2 Organisation du travail et outils

Le projet ayant été mené par un seul développeur, les trois rôles Scrum, Product Owner, Scrum Master et équipe de développement, ont été assumés par la même personne. Le suivi du Product Backlog et des Sprint Backlogs a été réalisé avec **GitLab Issues et Boards**, qui a servi de tableau kanban pour chaque sprint.

Le développement a été découpé en un sprint d'initialisation (sprint 0) suivi de cinq sprints fonctionnels, chacun clôturé par une revue et une rétrospective avant l'engagement du suivant.

Le chapitre suivant détaille le sprint 0.

1.6 Conclusion

Ce chapitre a présenté le cadre général du projet : l'entreprise d'accueil ASM, la problématique de l'intégration ERP multiple sans dépendance ni interruption réseau, les objectifs et le périmètre retenus, ainsi que les limites des solutions existantes qui justifient cette approche. La méthodologie Scrum a été retenue pour sa capacité à absorber un périmètre encore mouvant. Le chapitre suivant présente le sprint 0 : l'analyse des besoins, la planification des sprints et l'étude technique préalable.

2.1 Introduction

Le chapitre précédent a posé le cadre : une entreprise de services numériques, un besoin de suivi de livraison adossé à des progiciels de gestion hétérogènes, et une méthode de travail itérative. Reste à transformer ce cadre en un plan de travail exécutable.

C'est l'objet du sprint 0. Contrairement aux cinq sprints qui le suivent, il ne livre aucune fonctionnalité : il produit les décisions sans lesquelles les autres ne pourraient pas commencer. Qui utilise le système et avec quels droits, ce qu'il doit savoir faire, dans quel ordre le construire, et avec quels outils.

Ce chapitre restitue ces quatre travaux dans cet ordre. Il se termine sur trois exigences qui, sans être des fonctionnalités, ont pesé plus lourd que toutes les autres sur les choix qui ont suivi.

2.2 Identification des acteurs

Un acteur est un rôle joué par une personne ou un système extérieur qui interagit avec la solution. Les acteurs retenus ne reprennent pas l'organigramme de l'entreprise cliente : ils suivent les *droits*. Deux postes qui ouvrent les mêmes écrans et font les mêmes opérations forment un seul acteur, même si leurs intitulés diffèrent. C'est pourquoi le système compte trois rôles internes et non quatre, comme le détaille le chapitre 4.

TAB. 2.1 : Acteurs du système et périmètre de chacun

Acteur	Description	Périmètre d'action
Administrateur	Responsable du paramétrage de l'installation pour son entreprise.	Comptes et rôles, correspondance des champs avec le progiciel, référentiels d'exploitation : chauffeurs, véhicules et zones. Hérite de toutes les actions du dispatcheur.
Dispatcheur	Il transforme au quotidien les commandes en tournées et arbitre les aléas.	Import des commandes, planification, affectation, validation, traitement des exceptions et des retours, comptage et arbitrage des remises de caisse, synchronisation des dépôts depuis le progiciel.
Responsable	Supervise l'activité sans intervenir dessus.	Consultation des livraisons, des tournées, des rapports, des espèces en circulation et du journal d'audit. Aucune action de modification.
Livreur	Exécute les tournées sur le terrain depuis l'application mobile.	Consultation de sa tournée du jour, changements de statut, preuve de livraison, déclaration d'échec, encaissement, transfert de garde, collecte des retours.
Destinataire	Client final du chargeur.	Consultation de l'avancement de son colis par un lien public, sans compte ni authentification, et demande de retour d'un article reçu.

Deux systèmes extérieurs complètent ce tableau sans être des acteurs au sens strict, puisqu'ils n'ont ni intention ni objectif propre : le **progiciel de gestion** de l'entreprise cliente (Odoo ou ERPNext), source des commandes et destinataire des statuts, et le **serveur d'identité** Keycloak, qui authentifie les utilisateurs internes. Ils apparaissent dans le diagramme des cas d'utilisation sous forme de composants stéréotypés, à droite de la frontière du système.

2.3 Analyse et spécification des besoins

2.3.1 Besoins fonctionnels

Les besoins fonctionnels décrivent ce que le système doit savoir faire. Ils sont regroupés en huit domaines, qui serviront ensuite de découpage au *product backlog*.

Gestion des identités et des accès. Le système doit authentifier les utilisateurs internes, leur attribuer un rôle, et n'autoriser chaque opération que si le rôle porte la permission correspondante. Il doit également héberger plusieurs entreprises clientes sur une même installation, sans qu'aucune ne puisse atteindre les données d'une autre.

Intégration au progiciel de gestion. Le système doit importer les commandes prêtes à être livrées, puis renvoyer les statuts de livraison, les quantités réellement livrées et les preuves associées. Cette intégration doit fonctionner avec des progiciels différents, et avec des versions différentes d'un même progiciel, sans réécriture du cœur applicatif. L'administrateur doit pouvoir établir lui-même la correspondance entre les champs de son progiciel et ceux du système.

Référentiel logistique. Le système doit gérer les chauffeurs, les véhicules et les zones de couverture. Les dépôts, en revanche, ne sont pas saisis : ils reflètent les entrepôts du progiciel et sont récupérés par synchronisation, afin d'éviter un double référentiel qui divergerait au premier changement.

Planification et affectation. Le dispatcheur doit pouvoir construire une tournée à partir d'un ensemble de livraisons, y affecter un chauffeur et un véhicule, réordonner les arrêts, demander une optimisation automatique du parcours, puis valider la tournée pour la rendre visible au chauffeur.

Exécution terrain. Le chauffeur doit consulter sa tournée, déclarer son départ, marquer son arrivée à chaque arrêt, livrer ou déclarer un échec motivé, et collecter une preuve de livraison comportant une photographie du colis et une photographie du bon signé. Un transfert de garde entre deux chauffeurs doit être possible en cours de journée, avec confirmation du receveur.

Encaissement et remise de caisse. Lorsque la commande est payable à la livraison, le chauffeur doit enregistrer le montant encaissé et son mode de règlement. En fin de tournée, il déclare le total remis ; une autre personne compte la somme reçue et arbitre l'écart éventuel.

Suivi et traitement des aléas. Le dispatcheur doit voir l'avancement des livraisons en temps réel, être alerté des retards et des échecs, réaffecter une livraison bloquée, et enregistrer les retours de marchandise. Le destinataire doit pouvoir suivre son colis par un lien public.

Pilotage et exploitation. Le responsable doit disposer d'indicateurs de taux de réussite, de motifs d'échec, de respect des délais et de performance par chauffeur et par zone, et pouvoir exporter des documents. L'administrateur doit disposer d'une console de santé qui expose l'état de l'intégration et permet de relancer une synchronisation en échec. Un assistant interne doit enfin permettre à chacun d'interroger l'activité de son entreprise en langage naturel, dans la limite de ce que son rôle l'autorise à consulter.

2.3.2 Besoins non fonctionnels

Les besoins non fonctionnels ne décrivent pas des fonctions mais des qualités attendues du système. Trois d'entre eux ont pesé plus lourd que les autres sur l'architecture et sont signalés comme tels.

TAB. 2.2 : Besoins non fonctionnels

Besoin	Exigence
Agnosticité ERP	Le cœur métier ne doit connaître aucun progiciel en particulier. Ajouter un progiciel, ou prendre en charge une nouvelle version, ne doit pas modifier le code métier. <i>Structurant.</i>
Isolation par client	Les données de chaque entreprise cliente doivent être séparées de façon structurelle, et non par un simple filtre applicatif qu'un oubli suffirait à contourner. <i>Structurant.</i>
Continuité hors ligne	L'application chauffeur doit rester utilisable sans réseau. Les actions réalisées hors couverture doivent être rejouées à la reconnexion sans produire de doublon. <i>Structurant.</i>
Sécurité	Communications chiffrées, jetons d'accès à durée limitée, contrôle des droits appliqué à la fois à la passerelle et dans chaque service.
Traçabilité	Toute opération sensible doit laisser une trace horodatée et attribuée à son auteur, consultable a posteriori.
Temps réel	Un changement d'état survenu sur le terrain doit se refléter sur le poste du dispatcheur sans rechargement manuel.
Performance	Les listes doivent être renvoyées par pages plutôt qu'en bloc, et tout critère de recherche exposé à l'utilisateur doit être indexé.
Portabilité	L'ensemble doit se déployer sur une machine unique par conteneurs, sans installation manuelle de dépendances.
Ergonomie	Interface d'administration en français, arabe et anglais ; application chauffeur conçue pour un usage à une main.

2.3.3 Diagramme global des cas d'utilisation

Le diagramme de la figure 2.1 synthétise les besoins fonctionnels sous forme de cas d'utilisation et les rattache à leurs acteurs. Il reste volontairement au premier niveau : le système en compte près de quatre-vingts une fois détaillés, les représenter tous rendrait la figure illisible. Le détail est porté par les diagrammes propres à chaque sprint, aux chapitres 4 à 8.

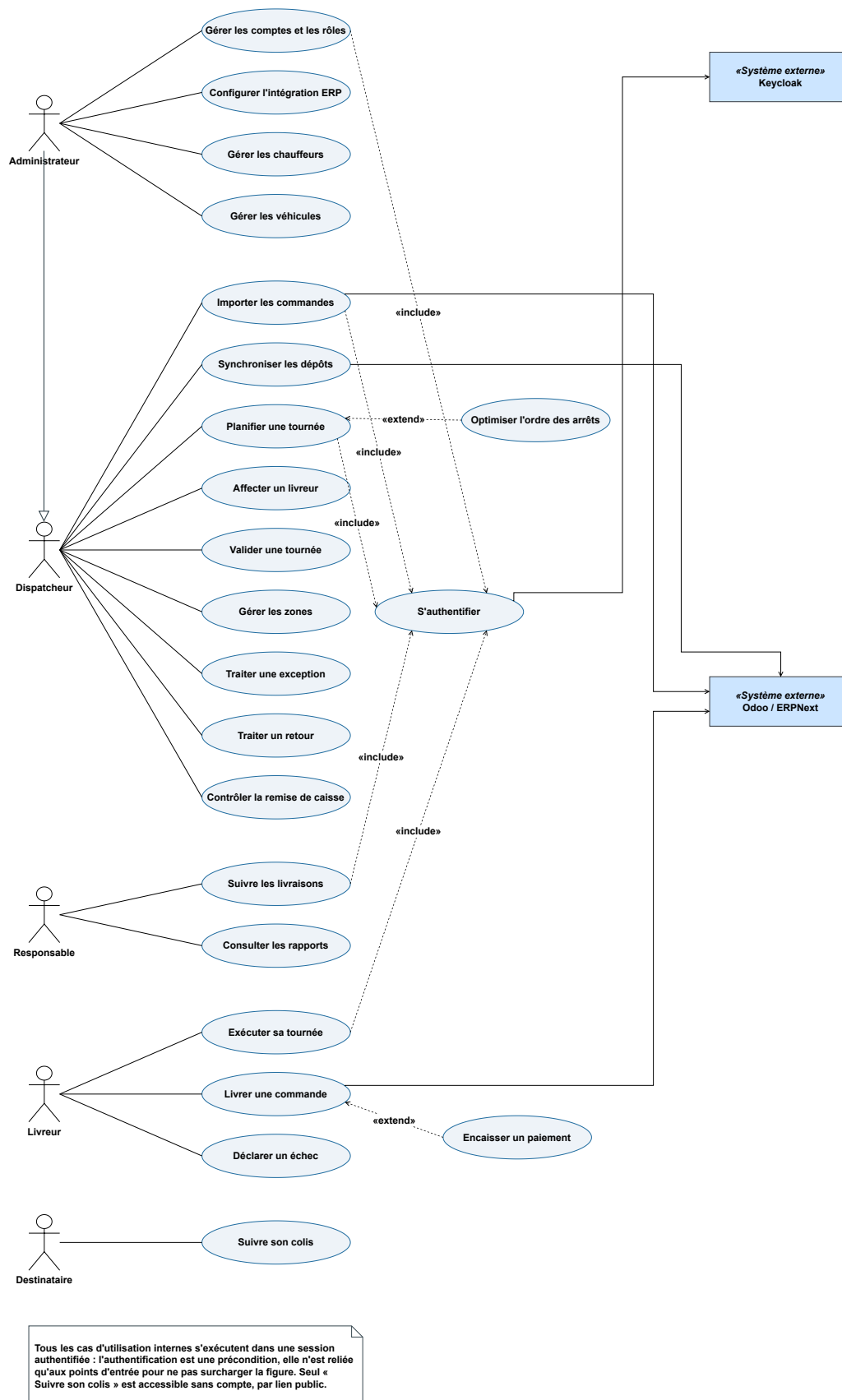


FIG. 2.1 : Diagramme global des cas d'utilisation

Trois éléments de lecture méritent d'être soulignés.

Une seule généralisation relie les acteurs : l'administrateur est un dispatcheur qui dispose en plus du paramétrage, et ses permissions incluent effectivement toutes celles du dispatcheur. La flèche évite de redessiner une douzaine de traits.

Le responsable, lui, reste rattaché à ses propres cas. Il croise le dispatcheur sur les consultations sans jamais le contenir ni en dériver : il est seul à accéder au journal d'audit, et n'a aucune des permissions d'action. Superviser suppose de pouvoir relire ce que font ceux qui opèrent, sans pouvoir opérer soi-même ; c'est pourquoi les rôles sont déclarés à plat dans l'annuaire, chacun portant sa liste explicite, plutôt qu'en chaîne d'héritage. Le chapitre 4 détaille ce modèle.

2.3.4 Diagramme de classes global

Le diagramme de la figure 2.2 présente les concepts manipulés par le système et leurs relations. Il fixe le vocabulaire employé dans tout le reste du rapport.

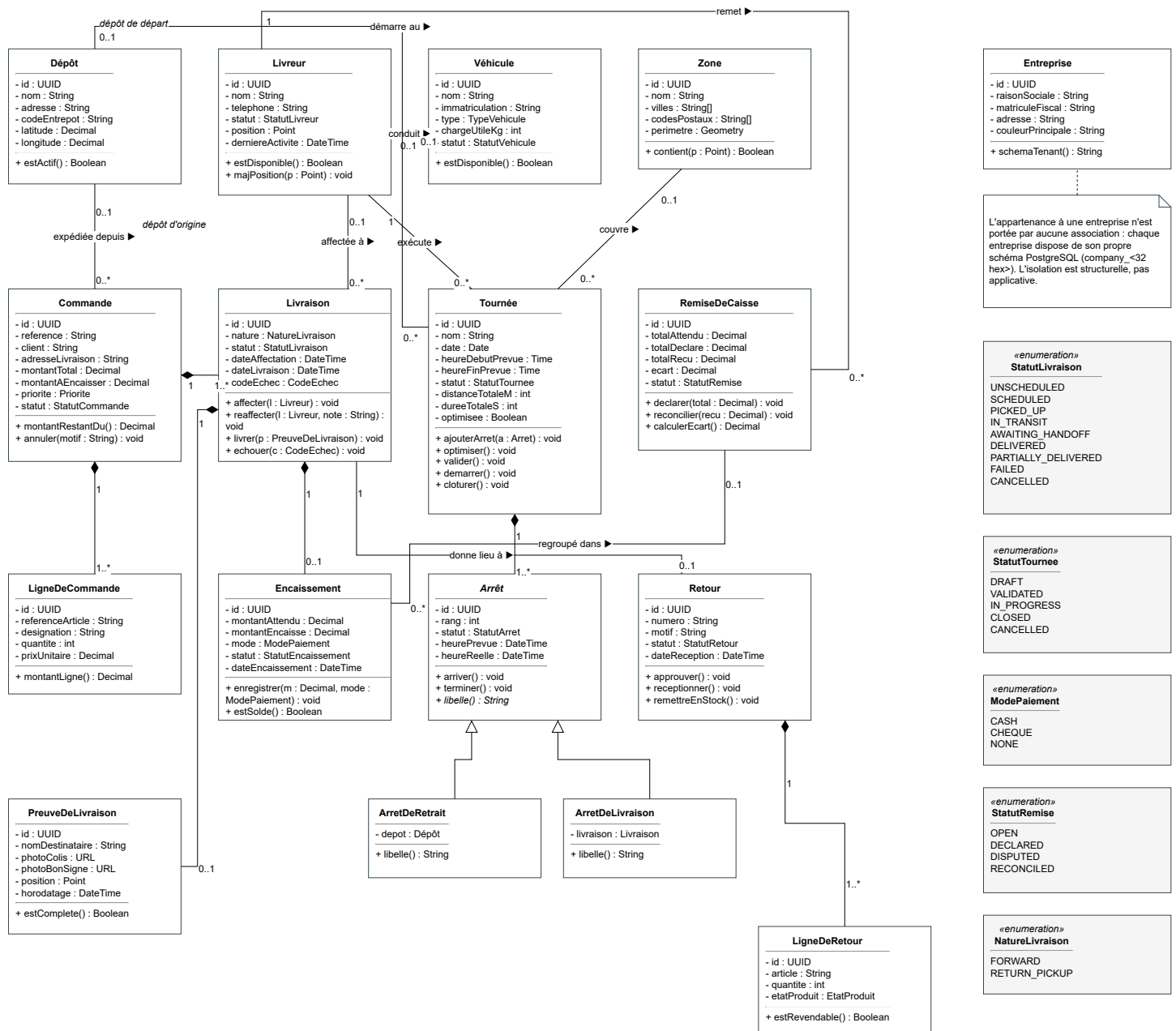


FIG. 2.2 : Diagramme de classes global

Il se lit de haut en bas, du référentiel vers l'exécution. La partie haute regroupe ce qui existe avant toute commande : dépôts, chauffeurs, véhicules et zones. Au centre, une **Commande** issue du progiciel se décompose en une ou plusieurs **Livraisons** ; le losange plein indique une composition, c'est-à-dire que les livraisons n'ont pas d'existence propre en dehors de leur commande. En parallèle, un chauffeur exécute des **Tournées**, elles-mêmes composées d'**Arrêts**.

La classe *Arrêt* est abstraite, et c'est le point le moins évident du modèle. Un arrêt de retrait désigne un dépôt où le chauffeur charge ; un arrêt de livraison désigne une livraison à remettre. Ce ne sont pas les mêmes objets, d'où la généralisation : les représenter par un simple attribut «type» aurait masqué une différence de structure réelle.

La partie basse porte les traces laissées par l'exécution : une **PreuveDeLivraison**, un **Encaissement** lorsque le client règle à la porte, et un **Retour** si la marchandise revient. Les encaissements d'un chauffeur sont ensuite regroupés dans une **RemiseDeCaisse**, ce qui matérialise la séparation des responsabilités : celui qui déclare la somme n'est pas celui qui la valide.

Enfin, la classe **Entreprise** n'est reliée à rien, et c'est délibéré. L'appartenance d'une donnée à une entreprise cliente n'est pas portée par une clé étrangère mais par un schéma de base de données distinct. Le chapitre 3 détaille ce mécanisme.

2.4 Gestion du projet avec Scrum

2.4.1 Product Backlog

Le *product backlog* traduit les besoins fonctionnels en éléments réalisables et priorisés. Ils ont été formulés sous forme de *user stories*, selon le gabarit « en tant que *rôle*, je veux *action*, afin de *bénéfice* », puis regroupés en huit épopées correspondant aux domaines de la section précédente.

Deux échelles complètent chaque élément. La **priorité** suit la méthode MoSCoW : *Must have* pour l'indispensable, *Should have* pour l'important non bloquant, *Could have* pour le souhaitable. L'**estimation** est exprimée en points de complexité selon la suite de Fibonacci ; il s'agit d'une complexité relative entre éléments, non d'une durée. Trois repères ont servi d'étalon tout au long de l'estimation : 3 points pour une gestion de liste sans règle particulière, 8 points pour une fonction qui traverse plusieurs composants, 21 points pour une fonction qui impose une décision d'architecture.

Les *Must* pèsent 240 points, soit près des deux tiers du backlog. C'est au-dessus de la proportion habituellement conseillée, et la raison est assumée : l'isolation des données, l'indépendance vis-à-vis du progiciel et la continuité hors ligne n'admettent pas de version dégradée. La marge de manœuvre est donc portée par les 47 points de *Could*, qui pouvaient être abandonnés sans dénaturer le produit.

TAB. 2.3 : Product Backlog du projet

Réf.	User story	Priorité	Est.	Sprint
Épopée 1 — Identités, autorisations et multi-tenance				
US-01	En tant qu'utilisateur, je veux m'authentifier par un compte unique afin d'accéder aux écrans correspondant à mon rôle.	Must	8	1

suite page suivante

TAB. 2.3 — *suite*

Réf.	User story	Priorité	Est.	Sprint
US-02	En tant qu'administrateur, je veux créer des comptes et leur attribuer un rôle afin de contrôler qui accède à quoi.	Must	3	1
US-03	En tant qu'administrateur, je veux que chaque opération vérifie la permission portée par le rôle de l'utilisateur afin que personne ne puisse dépasser ses droits.	Must	8	1
US-04	En tant qu'entreprise cliente, je veux que mes données soient inaccessibles aux autres entreprises hébergées afin de garantir ma confidentialité.	Must	13	1
Épopée 2 — Intégration au progiciel de gestion				
US-05	En tant que dispatcheur, je veux importer les commandes prêtes depuis le progiciel afin de ne pas les ressaisir.	Must	13	2
US-06	En tant qu'administrateur, je veux relier les champs de mon progiciel à ceux du système afin d'adapter l'intégration à mon paramétrage.	Must	13	2
US-07	En tant qu'entreprise, je veux pouvoir brancher la solution sur n'importe quel progiciel de gestion afin de ne dépendre d'aucun éditeur.	Must	21	2
US-08	En tant que dispatcheur, je veux que le statut d'une livraison remonte automatiquement dans le progiciel afin que la facturation suive.	Must	8	2
US-09	En tant que dispatcheur, je veux qu'une synchronisation interrompue soit réessayée puis signalée afin qu'aucune information ne se perde silencieusement.	Must	13	2
US-10	En tant que dispatcheur, je veux récupérer les entrepôts du progiciel comme dépôts afin d'éviter un double référentiel.	Should	5	2
US-11	En tant qu'administrateur, je veux que le système fonctionne avec plusieurs versions du même progiciel afin de ne pas imposer de montée de version au client.	Must	13	2
Épopée 3 — Référentiel logistique				

suite page suivante

TAB. 2.3 — *suite*

Réf.	User story	Priorité	Est.	Sprint
US-12	En tant qu’administrateur, je veux gérer la liste des chauffeurs afin de refléter l’équipe réelle.	Must	3	1
US-13	En tant qu’administrateur, je veux gérer le parc de véhicules avec leur capacité afin de pouvoir les affecter.	Should	3	1
US-14	En tant qu’administrateur, je veux définir des zones géographiques afin de regrouper les livraisons par secteur.	Should	8	1
Épopée 4 — Planification et affectation				
US-15	En tant que dispatcheur, je veux construire une tournée à partir de livraisons afin d’organiser la journée.	Must	13	3
US-16	En tant que dispatcheur, je veux affecter un chauffeur et un véhicule à une tournée afin d’en désigner le responsable.	Must	8	3
US-17	En tant que dispatcheur, je veux obtenir un ordre de passage optimisé afin de réduire la distance parcourue.	Should	13	3
US-18	En tant que dispatcheur, je veux valider une tournée afin de la rendre visible au chauffeur.	Must	3	3
US-19	En tant que dispatcheur, je veux connaître l’heure d’arrivée estimée à chaque arrêt afin de détecter les retards.	Could	8	3
Épopée 5 — Exécution terrain				
US-20	En tant que chauffeur, je veux consulter ma tournée du jour sur mon téléphone afin de savoir où aller.	Must	8	4
US-21	En tant que chauffeur, je veux déclarer une livraison réussie ou échouée avec un motif afin que le bureau soit informé.	Must	8	4
US-22	En tant que chauffeur, je veux enregistrer une preuve de livraison avec photographies afin de couvrir l’entreprise en cas de litige.	Must	13	4

suite page suivante

TAB. 2.3 — *suite*

Réf.	User story	Priorité	Est.	Sprint
US-23	En tant que chauffeur, je veux continuer à travailler sans réseau et voir mes actions transmises au retour de la couverture afin de ne jamais être bloqué.	Must	21	4
US-24	En tant que chauffeur, je veux transmettre un colis à un collègue avec confirmation afin que la responsabilité soit tracée.	Could	13	4
Épopée 6 — Encaissement et remise de caisse				
US-25	En tant que chauffeur, je veux enregistrer le montant encaissé et son mode de règlement afin d'en garder la trace.	Must	8	4
US-26	En tant que chauffeur, je veux déclarer le total remis en fin de tournée afin de clore ma journée.	Must	3	4
US-27	En tant que dispatcheur, je veux compter la somme reçue et arbitrer l'écart afin que la caisse soit vérifiable.	Must	8	4
Épopée 7 — Suivi, aléas et retours				
US-28	En tant que dispatcheur, je veux voir l'avancement des livraisons sans recharger la page afin de réagir immédiatement.	Must	13	3
US-29	En tant que dispatcheur, je veux être alerté des retards et des échecs afin de traiter les exceptions en priorité.	Should	5	3
US-30	En tant que dispatcheur, je veux réaffecter une livraison bloquée à un autre chauffeur afin de sauver la journée.	Must	5	3
US-31	En tant que destinataire, je veux suivre mon colis par un lien reçu afin de connaître son avancement sans créer de compte.	Should	8	3
US-32	En tant que destinataire, je veux demander le retour d'un article reçu afin que ma demande soit enregistrée et suivie.	Should	3	3
US-33	En tant que dispatcheur, je veux approuver ou refuser une demande de retour et déclencher la reprise en stock afin de garder la main sur ce qui revient.	Should	8	3

suite page suivante

TAB. 2.3 — *suite*

Réf.	User story	Priorité	Est.	Sprint
US-34	En tant que chauffeur, je veux collecter un retour chez le destinataire afin de le ramener au dépôt.	Should	5	4
Épopée 8 — Pilotage et exploitation				
US-35	En tant que responsable, je veux consulter les indicateurs de performance afin de piloter l'activité.	Should	13	5
US-36	En tant que responsable, je veux exporter un rapport de tournée afin de le transmettre ou de l'archiver.	Could	5	5
US-37	En tant qu'administrateur, je veux une console indiquant l'état de l'intégration afin de distinguer une panne du progiciel d'une panne du système.	Must	13	5
US-38	En tant qu'administrateur, je veux relancer une synchronisation en échec afin de réparer sans intervention technique.	Should	5	5
US-39	En tant qu'administrateur, je veux savoir qui a fait quoi et quand sur les données sensibles afin de pouvoir répondre à un audit.	Should	8	5
US-40	En tant qu'utilisateur, je veux poser une question en langage naturel sur l'activité de mon entreprise afin d'obtenir la réponse sans parcourir plusieurs écrans.	Could	13	5
US-41	En tant qu'administrateur, je veux régler moi-même les seuils de retard afin d'adapter les alertes au rythme de mon activité.	Should	3	5
US-42	En tant qu'administrateur, je veux rejouer les messages bloqués d'une file afin de réparer un incident sans perdre d'événement.	Should	5	5
US-43	En tant que dispatcheur, je veux visualiser la densité des commandes par zone afin de répartir mes tournées sur la journée.	Could	5	5
US-44	En tant que responsable, je veux exporter la performance d'un chauffeur afin de préparer son entretien.	Could	3	5

Le backlog totalise quarante-quatre éléments pour 379 points de complexité. Il a été tenu dans

GitLab Issues, chaque épopée correspondant à une étiquette et chaque sprint à un tableau kanban dédié.

2.4.2 Planification des sprints

Le projet s’est déroulé du 2 mars au 17 août 2026, soit vingt-quatre semaines. Il a été découpé en un sprint d’initialisation et cinq sprints fonctionnels de trois à cinq semaines, chacun clos par une revue et une rétrospective avant l’engagement du suivant.

L’ordre des sprints obéit à une contrainte simple : chaque sprint ne doit dépendre que de ce qui a déjà été livré. L’isolation par entreprise et le contrôle des droits viennent en premier, parce que toute donnée créée ensuite doit déjà être rattachée à un client et protégée. Les rajouter après coup aurait imposé de reprendre l’ensemble du code écrit entre-temps. L’intégration au progiciel vient juste après, parce qu’elle alimente le système en commandes : sans elle, les sprints suivants auraient travaillé sur des données saisies à la main, donc sur un cas qui n’existe pas en production.

La priorité MoSCoW n’intervient donc pas dans cet ordre : c’est la dépendance technique qui le fixe. Elle joue à l’intérieur de chaque sprint, où elle désigne ce qui saute en premier si la fin du cycle approche. C’est ce qui explique qu’un *Could* soit livré dès le sprint 3 alors que des *Must* attendent le sprint 5.

TAB. 2.4 : Planification des sprints

Sprint	Objectif	Période	Durée	Points
Sprint 0	Cadrage, besoins et étude technique	2 — 20 mars	3 sem.	—
Sprint 1	Identité, autorisation et référentiels	23 mars — 17 avril	4 sem.	46
Sprint 2	Intégration ERP agnostique	20 avril — 22 mai	5 sem.	86
Sprint 3	Planification des tournées et suivi temps réel	25 mai — 19 juin	4 sem.	87
Sprint 4	Application chauffeur et hors ligne	22 juin — 17 juillet	4 sem.	87
Sprint 5	Exploitation, résilience et assistance	20 juillet — 7 août	3 sem.	73
Recette, rédaction et soutenance		10 — 17 août	1 sem.	—

2.5 Étude technique

2.5.1 Environnement matériel

Le développement a été mené sur un poste unique, qui a également hébergé l’ensemble des services conteneurisés : bases de données, courtier de messages, serveur d’identité, moteur de calcul d’itinéraires et instances de progiciel de test. Faire tourner tout cela ensemble est ce qui a imposé les 24 Go de mémoire.

TAB. 2.5 : Environnement matériel de développement

Élément	Caractéristique
Processeur	Intel Core i5-12450H, 12 ^e génération
Mémoire vive	24 Go
Système d'exploitation	Windows 11 Professionnel
Exécution des services	Docker Desktop, moteur WSL 2
Essais mobiles	Appareil Android physique et émulateur

2.5.2 Environnement logiciel de développement

TAB. 2.6 : Outils de développement et de collaboration

Outil	Usage dans le projet
Visual Studio Code	Éditeur principal pour le code Java, TypeScript et Dart
Android Studio	Compilation, émulation et débogage de l'application chauffeur
Git et GitLab	Gestion de versions, suivi du backlog par issues et tableaux, intégration continue
Docker Desktop	Exécution locale de l'ensemble des services et de leurs dépendances
Postman	Essais manuels des interfaces de programmation
draw.io	Conception des diagrammes UML du présent rapport
L ^A T _E X (XeLaTeX)	Rédaction du rapport

La figure 2.3 rassemble les outils de cette chaîne, de l'écriture du code à la conception des schémas.



FIG. 2.3 : Outils de développement et de collaboration

2.5.3 Technologies retenues

Le choix des technologies a été arrêté au terme du sprint 0, chaque brique étant retenue pour une raison rattachable à un besoin de la section 2.3. Le chapitre 3 en détaille la mise en œuvre.

TAB. 2.7 : Technologies retenues et motif du choix

Couche	Technologie	Motif du choix
Services	Spring Boot 3.3.5, Java 17	Écosystème mature pour les architectures en services, avec prise en charge native de la sécurité par jetons et du multi-schéma
Passerelle	Spring Cloud Gateway	Point d'entrée unique : validation des jetons et application des droits avant tout accès aux services
Interface web	React 19, Vite, TypeScript	Rendu réactif adapté à un tableau de dispatch qui se met à jour en continu
Application mobile	Flutter	Base de code unique, et stockage local performant sur lequel repose le mode hors ligne
Base de données	PostgreSQL 16	Schéma par entreprise cliente pour l'isolation, type jsonb pour les données de forme variable, absence de coût de licence
Messagerie	RabbitMQ 4	Découplage des échanges avec le progiciel, relances et file de rebut intégrées
Identité	Keycloak	Serveur OpenID Connect éprouvé, gestion native des organisations
Fichiers	MinIO	Stockage compatible S3 des photographies de preuve, déployable localement
Itinéraires	OSRM	Moteur de calcul auto-hébergé, sans coût par requête ni dépendance à un service externe

La figure 2.4 présente ces briques, de la couche serveur au moteur d'itinéraires.

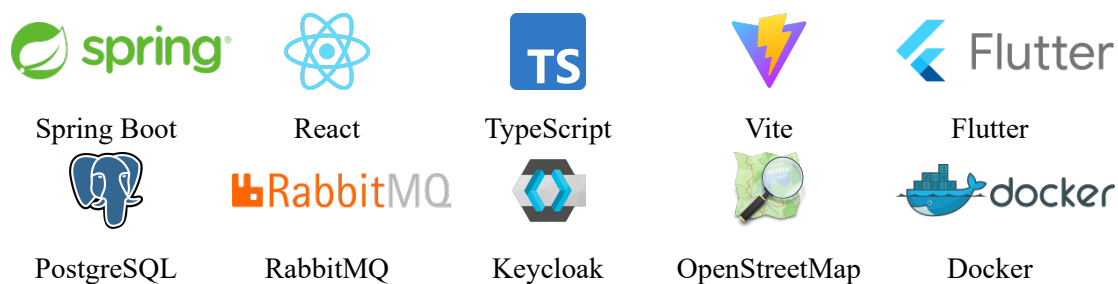


FIG. 2.4 : Technologies retenues pour la plateforme

2.6 Conclusion

Ce sprint d'initialisation a produit quatre livrables : la liste des acteurs et de leurs droits, la spécification des besoins fonctionnels et non fonctionnels, les deux diagrammes qui fixent le vocabulaire du système, et un backlog de quarante-quatre éléments réparti sur cinq sprints.

Son apport principal n'est pourtant pas cette production documentaire. Il tient à trois exigences que l'analyse a fait remonter au rang de contraintes structurantes : l'ouverture à plusieurs progiciels, l'hébergement de plusieurs entreprises clientes sur une même installation, et la continuité de service en l'absence de réseau. Aucune n'est une fonctionnalité visible pour l'utilisateur, et pourtant toutes trois ont déterminé l'architecture, l'ordre des sprints et une partie des choix technologiques.

Le chapitre suivant présente cette architecture. Les cinq chapitres qui lui succèdent déroulent ensuite les sprints dans l'ordre du tableau 2.4.

3.1 Introduction

Le sprint 0 a dit ce que le système doit faire et pour qui. Il reste à décider comment il est construit. C'est l'objet de ce chapitre, placé avant les sprints parce que les décisions qu'il contient ont été prises une fois, au début, et n'ont plus été remises en cause ensuite : chaque sprint s'est inséré dans cette structure au lieu de la redéfinir.

Quatre questions sont traitées dans l'ordre où elles se sont posées. Faut-il une application unique ou plusieurs services ? Où vivent les données une fois le système découpé ? Comment les données d'une entreprise cliente restent-elles hors de portée des autres ? Et comment les services se transmettent-ils l'information ?

3.2 Une application unique ou plusieurs services

Le premier choix engage tous les autres. Une application unique, appelée *monolithe*, regroupe tout le code dans un seul programme, avec une seule base de données. Une architecture en services découpe le programme en plusieurs applications autonomes, chacune avec sa base, qui communiquent par le réseau.

Le tableau 3.1 résume ce que chaque approche apporte et ce qu'elle coûte.

TAB. 3.1 : Application unique ou services séparés

Critère	Application unique	Services séparés
Mise en route	Immédiate	Plus longue : réseau, messagerie, configuration
Déploiement	Tout ou rien	Un service à la fois
Panne	Tout s'arrête	Le reste continue de fonctionner
Données	Une base, jointures libres	Une base par service, pas de jointure entre services
Montée en charge	On duplique l'ensemble	On duplique la partie sollicitée
Complexité	Concentrée dans le code	Déplacée vers les échanges entre services

Les services séparés ont été retenus pour trois raisons propres à ce projet.

La première est l'intégration au progiciel du client. Le code qui parle à Odoo ou à ERPNext est le plus instable du système : il change à chaque nouveau client et à chaque montée de version. L'isoler dans un service à part permet de le modifier sans redéployer le cœur métier, et de le laisser tomber en panne sans arrêter les livraisons du jour.

La deuxième est l'application mobile. Les chauffeurs envoient des positions et des changements de statut à un rythme sans commune mesure avec le trafic du bureau. Séparer le service qui les reçoit évite qu'une journée chargée sur le terrain ne ralentisse l'écran du dispatcheur.

La troisième est la possibilité de remplacer une partie sans toucher au reste. Un service ne montre à ses voisins que son interface : ce qu'il fait à l'intérieur, la base qu'il utilise, la bibliothèque sur laquelle il s'appuie ne regardent que lui. On peut donc le réécrire, changer sa base ou son moteur de calcul sans que les autres s'en aperçoivent. Dans une application unique, la même opération suppose de démêler des dépendances qui se sont installées un peu partout dans le code, faute d'une frontière qui les empêche de le faire.

Cette frontière vaut aussi vers l'extérieur. Ce qui se trouve derrière l'interface d'un service n'a pas à être du code écrit ici. Le service Chauffeur, par exemple, expose des chauffeurs, leur disponibilité et leur position. Le reste du système n'en sait rien de plus, et n'a pas à savoir d'où vient l'information.

Une entreprise qui préférerait confier ses tournées à un transporteur plutôt qu'à ses propres chauffeurs n'aurait donc qu'à mettre, à cette place, un service qui parle à l'interface de ce transporteur. Le service de livraison, la passerelle et le back-office continueraient de fonctionner sans une ligne de changement. Le système est prêt à s'appuyer sur des prestataires extérieurs sans être réécrit pour autant.

Le découpage suit les **domaines métier** identifiés au chapitre précédent, et non les couches techniques. Il n'existe pas de «service base de données» ni de «service interface» : chaque service porte un domaine complet, de son interface HTTP jusqu'à ses tables. Un développeur qui doit corriger le calcul d'une tournée sait donc où aller, sans traverser trois applications.

Ce choix a un prix, et il vaut mieux le nommer que le laisser découvrir au lecteur. Il tient en trois points.

Le premier est l'**incertitude sur le résultat d'un appel**. Un appel de méthode dans une application unique peut évidemment échouer, mais on sait toujours qu'il a échoué, et la transaction en cours annule tout d'un bloc. Un appel entre deux services peut, lui, expirer sans réponse : l'appelant ne sait pas si le travail a été fait ou non. C'est ce doute, et non la panne, qui coûte cher : réessayer risque de faire l'opération deux fois, ne pas réessayer risque de ne

pas la faire du tout. Il faut donc rendre les opérations rejouables sans double effet, ce que le chapitre 5 détaille.

Le deuxième est l'**absence de transaction commune**. Deux écritures dans deux services ne peuvent pas être annulées ensemble. Il faut soit accepter un décalage passager entre les deux, soit prévoir une opération de rattrapage.

Le troisième est le **coût d'exploitation** : plus de conteneurs à démarrer, plus de journaux à consulter, et une panne plus longue à localiser puisqu'elle peut venir de n'importe quel maillon de la chaîne.

Les sections 3.4 et 3.6 montrent comment ces trois points sont traités plutôt qu'ignorés.

3.3 Vue d'ensemble

La figure 3.1 présente l'ensemble du système : les applications qui l'utilisent à gauche, les services au centre, les composants d'infrastructure autour.

Architecture microservices d'ASM Track - Vue d'ensemble

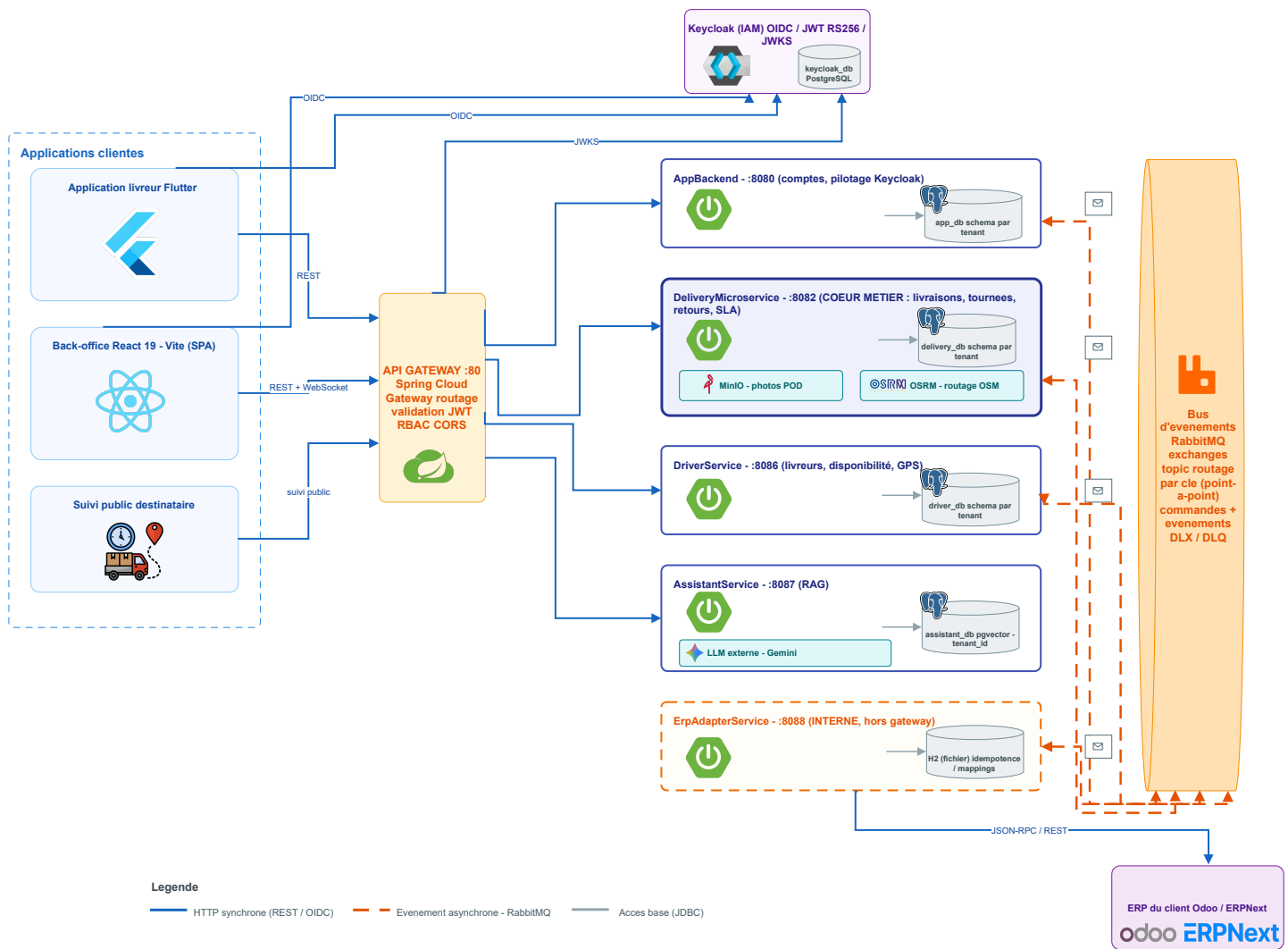


FIG. 3.1 : Vue d'ensemble de l'architecture

Trois trajets se lisent sur cette figure.

Le **trajet ordinaire** part d'une application cliente, traverse la passerelle et atteint un service, qui répond en interrogeant sa base. C'est le chemin de presque toutes les requêtes.

Le **trajet asynchrone**, en pointillés, ne passe par la passerelle ni par personne d'autre : un service dépose un message sur le bus et poursuit son travail sans attendre. Le destinataire le traitera quand il le pourra.

Le **trajet vers le progiciel du client** est le seul à sortir du système. Il est entièrement confié à l'adaptateur, qui est aussi le seul service à ne pas être exposé par la passerelle : aucune application, aucun navigateur ne peut l'appeler directement.

3.3.1 La passerelle

Toutes les requêtes venues de l'extérieur entrent par la passerelle, qui remplit quatre rôles.

Elle **achemine** la requête vers le bon service d'après le début de son adresse : quarante-cinq règles couvrent l'ensemble des routes exposées.

Elle **vérifie le jeton** d'authentification auprès du serveur d'identité, et refuse la requête si la signature ou la date de validité ne conviennent pas.

Elle **contrôle les droits** avant de transmettre. Si le rôle de l'utilisateur ne porte pas la permission attendue, la requête n'atteint jamais le service. Ce contrôle est ensuite refait à l'intérieur de chaque service, pour la raison exposée au chapitre 4.

Elle **réécrit les en-têtes d'identité**. Ceux qui arrivent du client sont effacés, puis reconstruits à partir du contenu du jeton. Un appelant ne peut donc pas se déclarer administrateur en ajoutant un en-tête à sa requête.

3.3.2 Les services

Le tableau 3.2 récapitule les six services, ce dont chacun est responsable et où il range ses données.

TAB. 3.2 : Les services et leurs responsabilités

Service	Responsabilité	Données
Passerelle	Acheminement, vérification du jeton, droits, en-têtes d'identité	Aucune
Comptes	Utilisateurs, rôles, entreprises clientes, pilotage du serveur d'identité	Base des comptes
Livraison	Cœur métier : commandes, livraisons, tournées, preuves, encaissements, retours	Base des livraisons ; photographies dans le stockage d'objets
Chauffeur	Fiches des chauffeurs, disponibilité, positions	Base des chauffeurs
Assistant	Questions posées en langage naturel sur l'activité de l'entreprise	Base avec index vectoriel
Adaptateur ERP	Seul point de contact avec le progiciel du client	Base locale des correspondances

Le service de livraison est nettement plus gros que les autres. Ce n'est pas un défaut de découpage mais le reflet du métier : la commande, la livraison, la tournée et la preuve forment un tout dont les parties se modifient ensemble. Les séparer aurait imposé des transactions réparties sur plusieurs services, c'est-à-dire remplacé une difficulté simple par une difficulté beaucoup plus lourde.

3.3.3 Les composants d'infrastructure

Quatre composants ne portent aucune règle métier mais rendent un service à tous les autres.

Le **serveur d'identité** détient les comptes et délivre les jetons. Aucun service ne conserve de mot de passe.

Le **bus de messages** transporte les échanges asynchrones et conserve les messages tant qu'ils n'ont pas été traités.

Le **stockage d'objets** reçoit les photographies des preuves de livraison. Elles ne sont pas mises en base : un fichier de plusieurs centaines de kilo-octets n'a rien à faire dans une table.

Le **moteur d'itinéraires** calcule distances et durées entre deux points à partir de données cartographiques libres. Il est hébergé avec le reste plutôt qu'appelé chez un fournisseur, ce qui évite de payer à la requête et de dépendre d'un service extérieur.

3.4 Du modèle de classes aux bases de données

Le chapitre 2 a présenté un modèle de classes unique, où toutes les entités apparaissent ensemble. C'est le modèle du métier, et il reste valable. Mais le découpage en services lui impose une contrainte que ce diagramme ne montre pas : ces entités ne vivent pas dans la même base.

La figure 3.2 montre cette répartition.

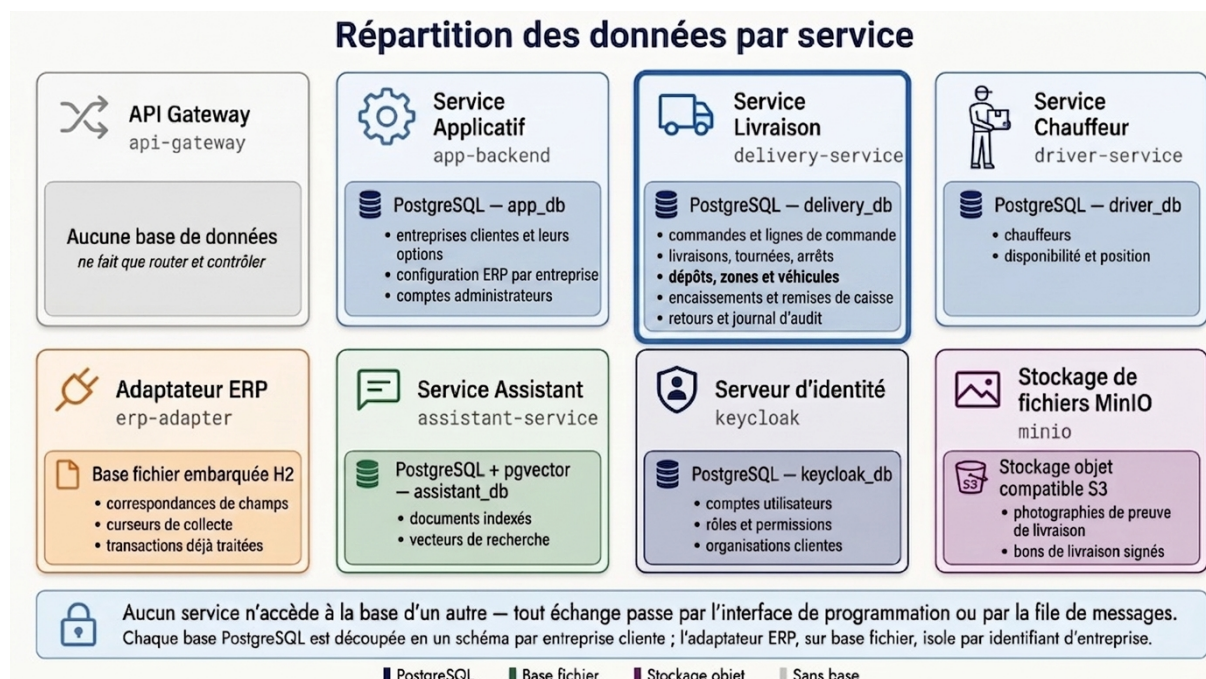


FIG. 3.2 : Répartition des données entre les services

La règle est simple et sans exception : **un service ne lit que sa propre base**. Aucun ne va chercher une donnée directement dans la base d'un autre, même en lecture seule.

Ce raccourci est pourtant tentant, parce qu'il marche tout de suite. Le problème vient après : dès qu'il existe, plus personne ne peut modifier ces tables sans risquer de casser un service qu'il n'avait pas vu.

3.4.1 Ce que devient une relation entre deux services

Cette règle a une conséquence directe sur le modèle. Une livraison est affectée à un chauffeur, mais la livraison est rangée dans la base des livraisons et le chauffeur dans celle des chauffeurs. La relation existe toujours dans le métier, mais elle ne peut plus être une clé étrangère.

Elle devient un **simple identifiant** : la livraison conserve l'identifiant du chauffeur, sans que la base ne garantisse quoi que ce soit à son sujet. Deux différences en découlent, qu'il faut accepter en connaissance de cause.

La base ne vérifie plus rien. Dans une application unique, une clé étrangère interdit d'affecter une livraison à un chauffeur qui n'existe pas : la base refuse l'écriture. Ici, rien ne s'y oppose ; c'est au service de le vérifier lui-même, en interrogeant son voisin ou en réagissant au message qui lui apprend qu'un chauffeur a été supprimé.

La jointure disparaît. Dans une application unique, une seule requête suffit pour afficher une tournée avec le nom de son chauffeur. Ici il en faut deux : le service lit la tournée dans sa base, puis appelle le service Chauffeur pour obtenir le nom.

3.4.2 L'évolution du schéma

Les tables ne sont pas créées à la main. Chaque modification du schéma est écrite dans un fichier numéroté, appliqué automatiquement au démarrage du service et jamais modifié une fois livré : une correction donne lieu à un nouveau fichier. Le projet en compte cent cinquante-neuf, dont la grande majorité pour le service de livraison.

Chaque environnement reconstruit sa base à partir de ces mêmes fichiers. La base de test et celle d'exécution portent donc la même structure et ne risquent pas de diverger. Un problème constaté sur l'une se rejoue à l'identique sur l'autre, là où on peut l'observer et le corriger.

3.5 L'isolation des entreprises clientes

Une même installation héberge plusieurs entreprises clientes. Chacune doit voir ses données, et uniquement les siennes. C'est l'exigence la plus contraignante du projet : une seule fuite, une seule fois, suffirait à disqualifier la solution.

3.5.1 Trois façons de séparer, une seule retenue

La première consiste à ajouter une colonne «entreprise» à chaque table et à filtrer dessus. Elle est simple, et c'est sa faiblesse : la séparation ne tient qu'à la présence d'une condition dans chaque requête. Une seule requête écrite sans elle, et une entreprise voit les données d'une autre. Rien, dans la base, ne s'y oppose.

La deuxième consiste à donner une base complète à chaque entreprise. La séparation devient matérielle, donc incontestable, mais le coût suit le nombre de clients : autant de bases à sauvegarder, à migrer et à surveiller, et autant de connexions ouvertes en permanence.

La troisième, retenue ici, est le **schéma par entreprise**. Une base unique contient autant de schémas que d'entreprises, chacun avec son jeu complet de tables. Une requête envoyée dans un schéma ne peut pas atteindre les tables d'un autre, sans qu'aucune condition n'ait à être écrite. Les sauvegardes et les migrations, elles, restent celles d'une base unique.

Le nom du schéma est déduit de l'identifiant de l'entreprise : le préfixe `company_` suivi des trente-deux caractères de cet identifiant. Cette déduction est faite à un seul endroit du code, à partir d'un identifiant déjà reconnu comme tel, ce qui écarte la possibilité d'y glisser un nom de schéma arbitraire.

3.5.2 Le trajet d'une requête

Encore faut-il que le bon schéma soit choisi à chaque requête, sans que le développeur ait à y penser. La figure 3.3 détaille les huit étapes de ce trajet.

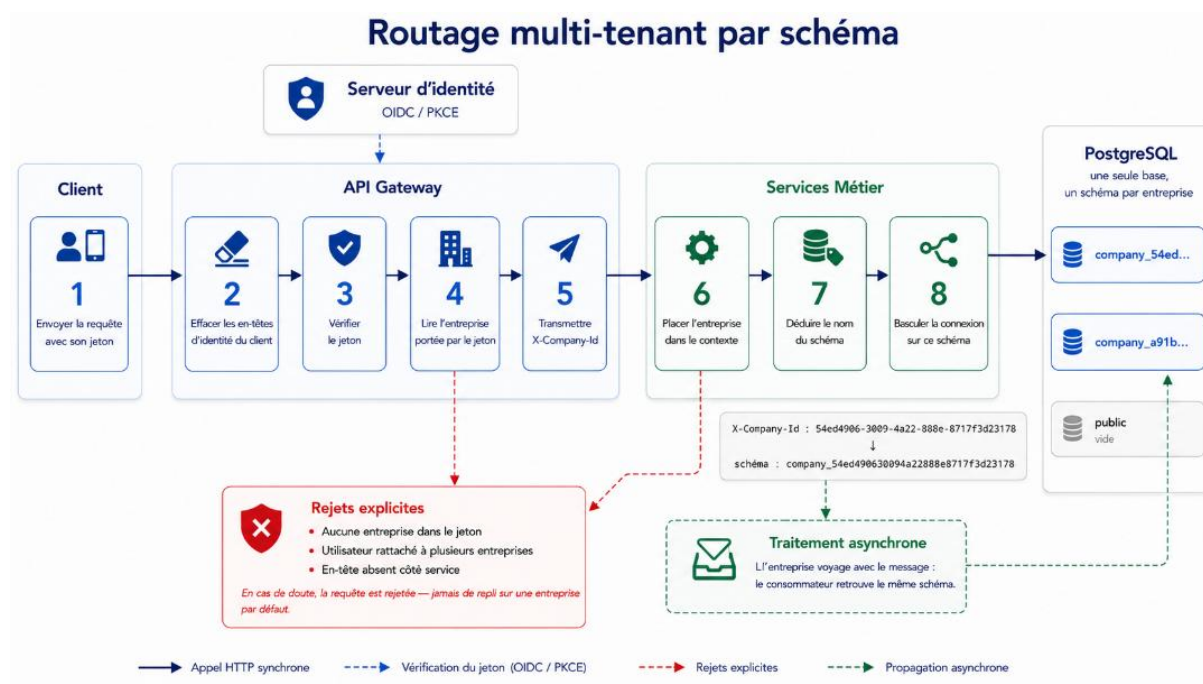


FIG. 3.3 : Le choix du schéma, de la requête jusqu'à la base

Les quatre premières étapes se déroulent dans la passerelle. Elle efface les en-têtes d'identité présents dans la requête, vérifie le jeton, y lit l'entreprise à laquelle l'utilisateur appartient, puis ajoute cette information à la requête qu'elle transmet. L'entreprise vient donc du jeton signé par le serveur d'identité, jamais de ce que l'appelant a bien voulu déclarer.

Les quatre suivantes se déroulent dans le service. Un filtre lit l'entreprise transmise et la place dans le contexte de la requête. Au moment d'ouvrir la connexion à la base, ce contexte est traduit en nom de schéma et la connexion bascule sur ce schéma. Le code métier, lui, écrit ses requêtes sans jamais mentionner l'entreprise ni le schéma : il ne peut donc pas oublier de le faire.

Le basculement porte sur la connexion que la transaction va réellement utiliser, et il est annulé avant que cette connexion ne retourne au réservoir partagé. Sans cette remise à zéro, la requête suivante hériterait du schéma de la précédente.

3.5.3 Le refus plutôt que le repli

Une règle traverse tout ce mécanisme : en cas de doute, la requête est rejetée. Jamais de repli sur une entreprise par défaut, jamais de valeur devinée. Quatre situations conduisent à un refus : le jeton ne porte aucune entreprise, l'utilisateur est rattaché à plusieurs entreprises, l'identifiant n'a pas la forme attendue, ou l'en-tête manque à l'entrée du service.

Le deuxième cas est le moins évident. Une requête émise par un utilisateur rattaché à deux entreprises ne dit pas laquelle des deux elle concerne, et le système devinerait mal. La règle est donc stricte : un utilisateur appartient à une entreprise et à une seule.

Le quatrième cas garantit que la protection tient même si la passerelle est contournée. Un service appelé directement, sans en-tête d'entreprise, refuse la requête au lieu de servir des données.

3.5.4 Le cas des traitements asynchrones

Un message traité plus tard par un autre service n'a plus ni requête HTTP ni jeton derrière lui : le schéma ne peut donc plus s'en déduire. Le service qui publie le message y inscrit lui-même l'identifiant de l'entreprise, dans l'en-tête. Le consommateur le lit avant tout traitement et en tire le schéma, de sorte que l'opération s'exécute dans l'entreprise qui l'a demandée.

3.6 Les échanges entre services

Deux services peuvent communiquer de deux manières, et le choix entre les deux n'est pas affaire de goût.

L'**appel direct** convient lorsque l'appelant a besoin de la réponse pour continuer. Il est simple à écrire et à suivre, mais il lie les deux services : si le second est arrêté, le premier échoue.

Le **message** convient lorsque l'appelant n'attend pas de réponse immédiate. Il dépose son message et continue. Le destinataire peut être arrêté, le message l'attendra.

La règle appliquée dans le projet tient en une phrase : appel direct quand la réponse est nécessaire tout de suite, message dans tous les autres cas. La figure 3.4 montre la topologie qui en résulte.

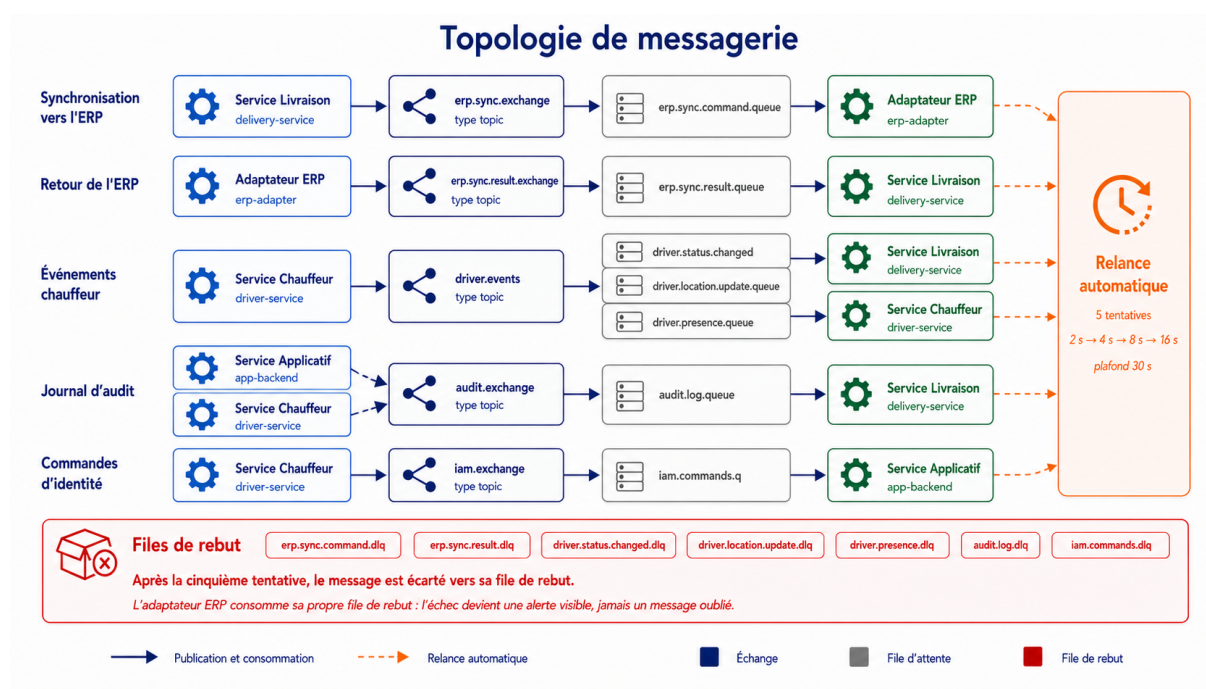


FIG. 3.4 : Topologie de la messagerie

Quatre familles de messages circulent. Les demandes de synchronisation partent du service de livraison vers l'adaptateur, et les résultats reviennent par le chemin inverse. Les événements du chauffeur, changement de statut et position, remontent vers le service de livraison. Les opérations sensibles alimentent le journal d'audit. Les créations de comptes, enfin, sont transmises au service qui pilote le serveur d'identité.

3.6.1 Quand un message ne passe pas

Un message peut échouer parce que le service destinataire est arrêté, parce que la base est indisponible, ou parce que le message lui-même est incorrect. Le traitement est le même dans tous les cas.

Le message est d'abord **réessayé** cinq fois, avec une attente qui double à chaque tentative : deux secondes, puis quatre, huit et seize. Cet espacement croissant laisse à une panne passagère le temps de se résorber, sans saturer un service déjà en difficulté.

Si les cinq tentatives échouent, le message est déplacé dans une **file de rebut**. Il n'est ni perdu ni rejoué indéfiniment : il attend, consultable, que la cause soit corrigée. Sans ce mécanisme, un message définitivement invalide serait repris sans fin et bloquerait tous ceux qui le suivent.

3.7 La mise en marche

L'ensemble est installé par conteneurs. Chaque service, chaque base, chaque composant d'infrastructure est décrit dans un fichier unique qui indique son image, sa configuration et ses dépendances. Une seule commande démarre les dix-sept conteneurs que compte l'installation, répartis comme l'indique le tableau 3.3.

TAB. 3.3 : Les conteneurs de l'installation

Groupe	Nombre	Contenu
Services applicatifs	6	Passerelle, comptes, livraison, chauffeur, assistant, adaptateur ERP
Bases de données	5	Une par service qui en possède une, plus celle du serveur d'identité
Infrastructure	3	Serveur d'identité, bus de messages, stockage d'objets
Moteur d'itinéraires	3	Téléchargement de la carte, préparation, puis exécution

Deux points méritent d'être signalés.

L'**ordre de démarrage** est déclaré, et non espéré. Un service n'est lancé qu'une fois sa base déclarée en bon état, et le serveur d'identité attend la sienne. Sans cette précaution, un service démarrant plus vite que sa base échouerait dès le premier accès.

Les **données survivent aux conteneurs**. Chaque base, le stockage d'objets et le bus écrivent dans un volume nommé, indépendant du conteneur qui l'utilise ; reconstruire une image ne détruit donc aucune donnée.

Ce mode d'installation répond à une exigence posée au chapitre 2 : le système doit pouvoir être installé chez un client sans y installer manuellement une seule dépendance. Le chapitre 9 revient sur son déroulement réel et sur les essais qui l'accompagnent.

3.8 Conclusion

Ce chapitre a fixé la structure dans laquelle les cinq sprints se sont inscrits : six services découpés par domaine métier, chacun maître de sa base ; une passerelle qui vérifie l'identité et les droits avant de transmettre ; une isolation des entreprises clientes obtenue par un schéma

dédié plutôt que par une condition à ne pas oublier ; des échanges par message lorsque la réponse peut attendre, avec réessais et file de rebut ; une installation par conteneurs reproductible.

Ces décisions expliquent une bonne part de ce qui suit. Le chapitre 4 met en place l'identité, les droits et l'isolation qui viennent d'être décrits.

Sprint 1 — Socle d'identité, d'autorisation et de multi-tenance

4.1 Introduction

Le chapitre précédent a fixé la structure du système. Ce sprint la remplit par le bas, car avant d'ouvrir la moindre fonction métier, il faut savoir *qui* parle au système, *ce qu'il a le droit de faire*, et *de quelle entreprise il relève*. Ces trois questions se répondent une fois, au même endroit, et tout le reste du logiciel s'appuie sur ces réponses.

Le sprint livre donc un socle en trois volets. Le premier est l'**identité** : une connexion unique, déléguée à un serveur spécialisé, qui délivre un jeton signé. Le deuxième est l'**autorisation** : une politique déclarative qui dit, règle par règle, quelle permission ouvre quelle route. Le troisième est l'**isolation** : chaque entreprise cliente ne voit que ses propres données, garanties par un schéma de base de données qui lui est réservé.

Sur ce socle, le sprint pose déjà les premiers écrans exploitables : la gestion des comptes et des rôles, puis trois référentiels, chauffeurs, véhicules et zones, qui seront le point de départ des tournées du sprint 3.

4.2 Objectifs du sprint et planification des tâches

Le périmètre couvre sept *user stories*, US-01 à US-04 pour le socle et US-12 à US-14 pour les référentiels, soit **46 points** d'effort. Le tableau 4.1 détaille les tâches du sprint, regroupées par *user story*, avec leur priorité relative. Les tâches de sécurité et d'isolation sont en priorité haute, puisqu'elles conditionnent tout le reste.

TAB. 4.1 : Sprint backlog — sprint 1

ID	User Story	Tâche	Description	Priorité
US-01	Authentification par compte unique (8 pts)	T1-01	Configurer le realm asm et les organisations dans Keycloak	Haute
		T1-02	Intégrer le flux Authorization Code + PKCE au back-office React	Haute

ID	User Story	Tâche	Description	Priorité
US-02	Création de comptes et attribution des rôles (3 pts)	T1-03	Personnaliser le thème de connexion (Keycloakify) : langue et mode sombre de l'application	Moyenne
		T1-04	Valider la signature des jetons par JWKS à la passerelle	Haute
		T1-05	Créer, modifier un compte et lui attribuer un rôle	Haute
		T1-06	Activer ou désactiver un compte, réinitialiser le mot de passe, forcer la déconnexion	Moyenne
US-03	Contrôle des permissions à chaque opération (8 pts)	T1-07	Provisionner une nouvelle entreprise cliente (organisation, schémas et compte administrateur) avec annulation en cas d'échec	Haute
		T1-08	Rédiger le modèle de 18 permissions et la politique déclarative unique qui les applique	Haute
		T1-09	Appliquer la même politique d'autorisation à la passerelle et à chaque service, contrôlée en CI	Haute
		T1-10	Modéliser les trois rôles composites dans Keycloak	Haute
		T1-11	Évaluer la politique à la passerelle et la re-vérifier dans chaque service	Haute
US-04	Isolation des données par entreprise cliente (13 pts)	T1-12	Concevoir le module partagé asm-tenant-core (routage Hibernate par schéma)	Haute
		T1-13	Déduire l'en-tête d'entreprise du contenu du jeton	Haute
		T1-14	Propager le locataire dans les messages asynchrones	Haute
		T1-15	Provisionner et migrer les schémas au démarrage	Haute
US-12	Référentiel des chauffeurs (3 pts)	T1-16	Modéliser et exposer le référentiel des chauffeurs	Moyenne
		T1-17	Réaliser l'écran back-office des chauffeurs	Moyenne

ID	User Story	Tâche	Description	Priorité
US-13	Référentiel des véhicules et de leur capacité (3 pts)	T1-18	Modéliser et exposer le référentiel des véhicules	Moyenne
US-14	Référentiel des zones géographiques (8 pts)	T1-19	Modéliser et exposer le référentiel des zones	Basse

4.3 Concepts et mécanismes utilisés

Ce sprint mobilise cinq notions. Elles sont présentées ici brièvement, dans l'ordre où le jeton d'un utilisateur les rencontre.

OAuth2 et OpenID Connect : L'authentification n'est pas écrite dans le projet. Elle est déléguée à **Keycloak**, un serveur d'identité spécialisé. Le dialogue avec lui suit *OpenID Connect*, de sorte que l'application n'interroge jamais un mot de passe ; elle redirige l'utilisateur vers le serveur d'identité et reçoit en retour une preuve signée de son identité, sous la forme d'un jeton. Écrire soi-même cette mécanique, avec la rotation des clés, l'expiration, la révocation et le verrouillage après échecs, aurait multiplié les occasions de mal faire, sans rien apporter de propre à une plateforme de livraison.

Le jeton JWT : Le jeton délivré est un *JSON Web Token*. Dans ce projet, il transporte trois informations : qui est l'utilisateur, quel rôle il porte, et à quelle entreprise cliente il appartient. Cette troisième donnée est celle qui, à l'étape suivante, désignera son schéma de base. Le tout étant signé, un service peut lire ces trois éléments et trancher seul, sans rappeler Keycloak à chaque requête.

La validation par JWKS : Encore faut-il vérifier cette signature. Keycloak publie ses clés publiques à une adresse connue, le *JWKS*. La passerelle les récupère une fois, les garde en mémoire, et s'en sert pour valider chaque jeton. Elle ne détient donc **aucun secret partagé**, puisqu'elle vérifie une signature avec une clé publique, exactement comme on vérifie un sceau sans posséder le tampon qui l'a apposé.

Le contrôle d'accès par rôles (RBAC) : Un jeton valide dit *qui* parle. Il reste à décider *ce qu'il peut faire*. Le projet suit le modèle *RBAC* [SAN 96] : des permissions élémentaires, comme lire les chauffeurs ou gérer les tournées, sont regroupées dans des rôles. Un rôle n'est qu'un sac de permissions. La figure 4.1 donne la répartition exacte des dix-huit permissions sur les trois rôles de la plateforme.

Domaine	Permission	Administrateur	Dispatcheur	Responsable
Comptes	user:manage	✓	—	—
Entreprise	company:manage	✓	—	—
Chauffeurs	driver:view	✓	✓	✓
	driver:manage	✓	—	—
Tournées	route:view	✓	✓	✓
	route:manage	✓	✓	—
	route:validate	✓	✓	—
Livraisons	delivery:view	✓	✓	✓
	delivery:manage	✓	✓	—
Dispatch	dispatch:operate	✓	✓	—
Caisse	cash:view	✓	✓	✓
	cash:manage	✓	✓	—
ERP	erp:sync	✓	✓	—
	erp:resync	✓	—	—
	erp:config	✓	—	—
Supervision	report:view	✓	✓	✓
	audit:view	✓	—	✓
	settings:manage	✓	—	—
Total permissions		18	11	6

FIG. 4.1 : Matrice des permissions par rôle

Cette matrice se lit ligne à ligne. L'**administrateur** détient les dix-huit permissions et gouverne son entreprise. Le **dispatcheur** en détient onze, celles qui font tourner l'exploitation au quotidien, y compris déclencher un import depuis le progiciel du client. Le **responsable** en détient six, toutes de lecture, plus l'accès au journal d'audit. Il supervise et relit les actions des autres, mais n'opère pas lui-même. La séparation entre le dispatcheur et le responsable est délibérée : superviser n'est pas exécuter.

Les organisations Keycloak : Keycloak sait regrouper ses utilisateurs en *organisations*. Le projet s'en sert pour représenter les entreprises clientes : chaque entreprise est une organisation, chaque utilisateur en est membre, et cette appartenance est inscrite dans son jeton. La notion de client de la plateforme n'a donc pas eu à être réinventée, car elle est portée par une fonctionnalité native du serveur d'identité, et c'est elle qui, à l'étape suivante, désigne le schéma de l'entreprise.

L'isolation par schéma : L'entreprise inscrite dans le jeton désigne un *schéma* de base de données réservé à ce client. Chaque requête s'exécute alors dans ce schéma, sans que le développeur ait à écrire la moindre condition de filtrage. Ce mécanisme, décrit au chapitre 3, est mis en œuvre concrètement dans ce sprint.

4.4 Analyse et spécification des besoins

4.4.1 Diagramme de cas d'utilisation du sprint

La figure 4.2 présente les cas d'utilisation du sprint. Elle distingue ce que fait tout utilisateur authentifié, quel que soit son rôle, de ce que seul l'administrateur peut faire.

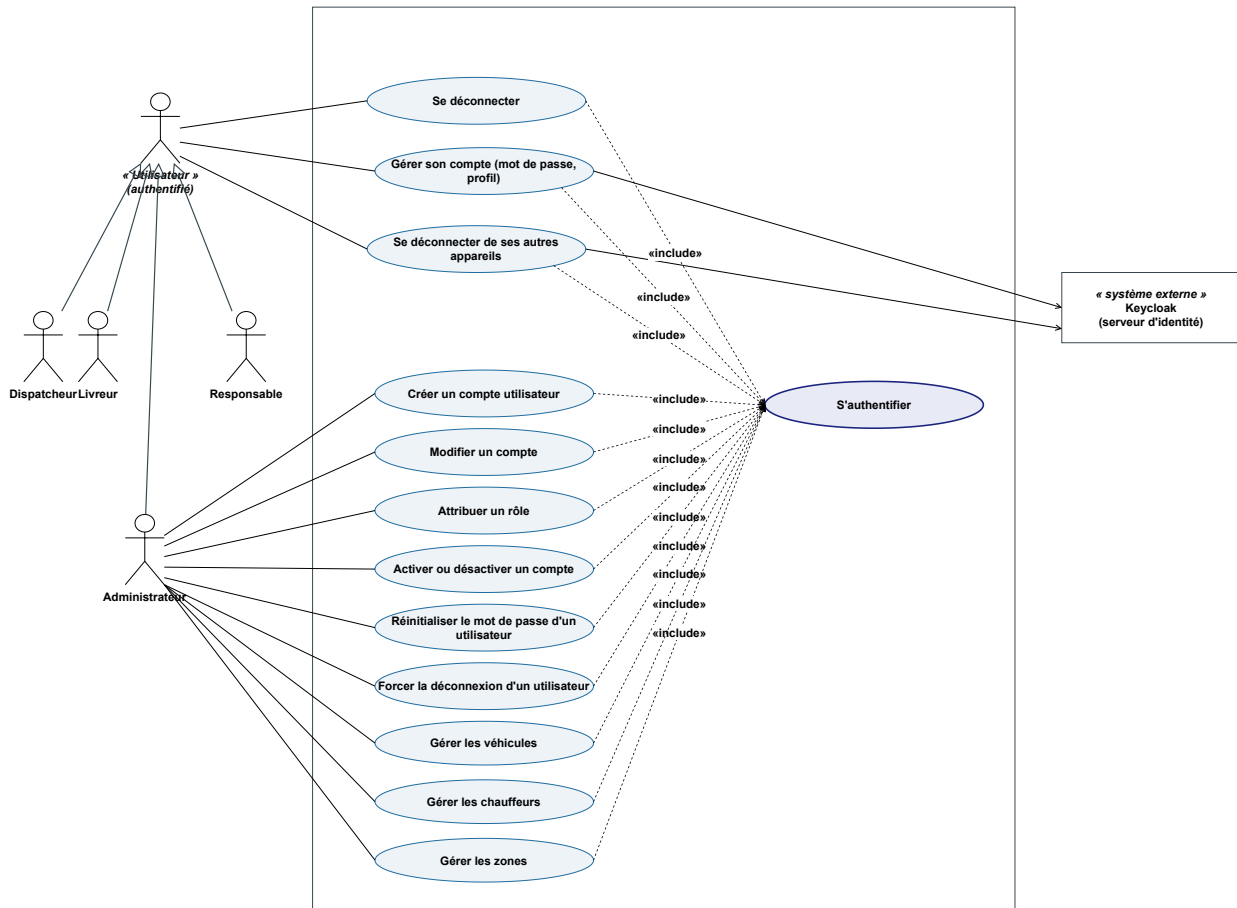


FIG. 4.2 : Diagramme de cas d'utilisation du sprint 1

Trois lectures s'en dégagent. D'abord, tout part de **S'authentifier**, que chaque autre cas inclut, car aucune action n'est possible sans un jeton valide. Ensuite, un **utilisateur authentifié**, qu'il soit administrateur, dispatcheur, responsable ou livreur, gère son propre compte : mot de passe, profil, déconnexion de ses autres appareils. Enfin, l'**administrateur** concentre les actions de gouvernance : créer et modifier des comptes, attribuer des rôles, et administrer les trois référentiels (chauffeurs, véhicules, zones). Keycloak apparaît comme un système externe, car c'est lui qui porte les comptes et délivre les jetons.

4.4.2 Description détaillée des cas d'utilisation

Trois cas d'utilisation sont détaillés : le plus transversal (*S'authentifier*), le plus sensible (*Créer un compte utilisateur*) et un référentiel représentatif (*Ajouter un chauffeur*).

TAB. 4.2 : Cas d'utilisation « S'authentifier »

Titre	S'authentifier
Acteur	Utilisateur (administrateur, dispatcheur, responsable, livreur)
Résumé	Obtenir un jeton d'accès signé auprès de Keycloak afin d'atteindre les ressources protégées de la plateforme.
Préconditions	Le compte existe dans le realm asm et il est actif.
Postconditions	L'utilisateur détient un jeton signé portant son rôle et son entreprise ; une session est établie.
Scénario nominal	1. L'utilisateur ouvre le back-office. 2. L'application le redirige vers Keycloak (Authorization Code + PKCE). 3. Il saisit ses identifiants. 4. Keycloak renvoie un code d'autorisation. 5. L'application échange le code contre les jetons. 6. Les requêtes suivantes portent le jeton, validé par JWKS à la passerelle.
Scénarios alternatifs	3a. Identifiants invalides : Keycloak affiche une erreur, aucun jeton n'est délivré. 6a. Jeton expiré : l'application le renouvelle silencieusement puis rejoue la requête.

TAB. 4.3 : Cas d'utilisation « Créer un compte utilisateur »

Titre	Créer un compte utilisateur
Acteur	Administrateur
Résumé	Créer un compte et lui attribuer le rôle qui fixe ses permissions.
Préconditions	L'administrateur est authentifié et détient la permission <code>user:manage</code> .
Postconditions	Le compte est créé dans Keycloak et rattaché à l'organisation de l'entreprise ; le rôle choisi fixe ses permissions.
Scénario nominal	1. L'administrateur ouvre la section « Identités et accès ». 2. Il choisit « Nouveau compte ». 3. Il renseigne le nom et l'adresse électronique. 4. Il choisit un rôle dans la liste. 5. Il valide ; le compte est créé et ajouté à l'organisation de l'entreprise.
Scénarios alternatifs	5a. L'adresse électronique est déjà utilisée : la création est refusée. 1a. Permission <code>user:manage</code> absente : l'accès est refusé (403).

TAB. 4.4 : Cas d'utilisation « Ajouter un chauffeur »

Titre	Ajouter un chauffeur
Acteur	Administrateur
Résumé	Ajouter un chauffeur au référentiel de l'entreprise.
Préconditions	L'administrateur est authentifié et détient la permission <code>driver:manage</code> .
Postconditions	Le chauffeur est enregistré dans le schéma de l'entreprise courante.
Scénario nominal	1. L'administrateur ouvre l'écran des chauffeurs. 2. Il choisit « Ajouter un chauffeur ». 3. Il renseigne la fiche. 4. Il valide ; la fiche est enregistrée.
Scénarios alternatifs	1a. L'utilisateur ne détient que <code>driver:view</code> (consultation) : l'ajout est refusé (403). 3a. Un champ obligatoire manque ou une saisie est invalide (téléphone, adresse électronique) : l'erreur est signalée et la fiche n'est pas enregistrée. 4a. Un chauffeur au même numéro de téléphone existe déjà dans l'entreprise : la création est refusée.

4.5 Modélisation conceptuelle

4.5.1 Diagramme de classes

Le cœur du sprint tient dans un petit module partagé, `asm-tenant-core`, qui porte la mécanique d'isolation par schéma. La figure 4.3 en donne le diagramme de classes de conception.

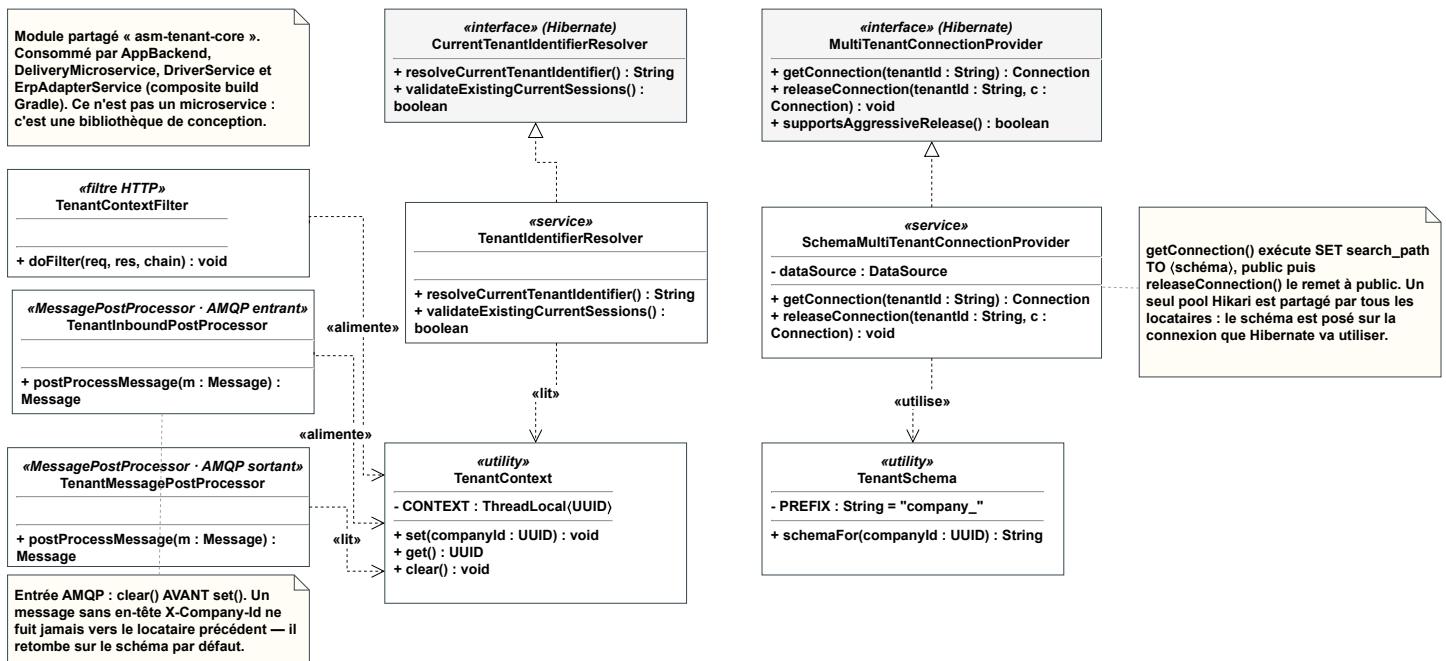


FIG. 4.3 : Diagramme de classes du module asm-tenant-core

Le module s'articule autour de deux interfaces d'Hibernate. `TenantIdentifierResolver` répond à la question « quel est le locataire courant ? » en lisant le contexte de la requête ; `SchemaMultiTenantConnectionProvider` traduit ce locataire en nom de schéma et l'applique à la connexion de base de données. Autour, trois classes remplissent ce contexte, c'est-à-dire y déposent l'entreprise de la requête en cours. Le `TenantContextFilter` le fait au début de chaque requête HTTP, en lisant l'entreprise dans l'en-tête reçu. Les deux post-processeurs font de même pour les messages échangés entre services : l'un ajoute l'entreprise au message au moment de l'envoi, l'autre la relit à la réception, avant que le message ne soit traité. Enfin, `TenantSchema` traduit l'identifiant de l'entreprise en nom de schéma.

Un point mérite d'être souligné : ce module n'est pas un service, mais une **bibliothèque** consommée par quatre services à la fois. La même mécanique d'isolation s'applique ainsi partout, sans être réécrite ni risquer de diverger d'un service à l'autre.

4.5.2 Diagrammes de séquence

La figure 4.4 suit une requête, de la connexion de l'utilisateur jusqu'à l'accès à une ressource protégée.

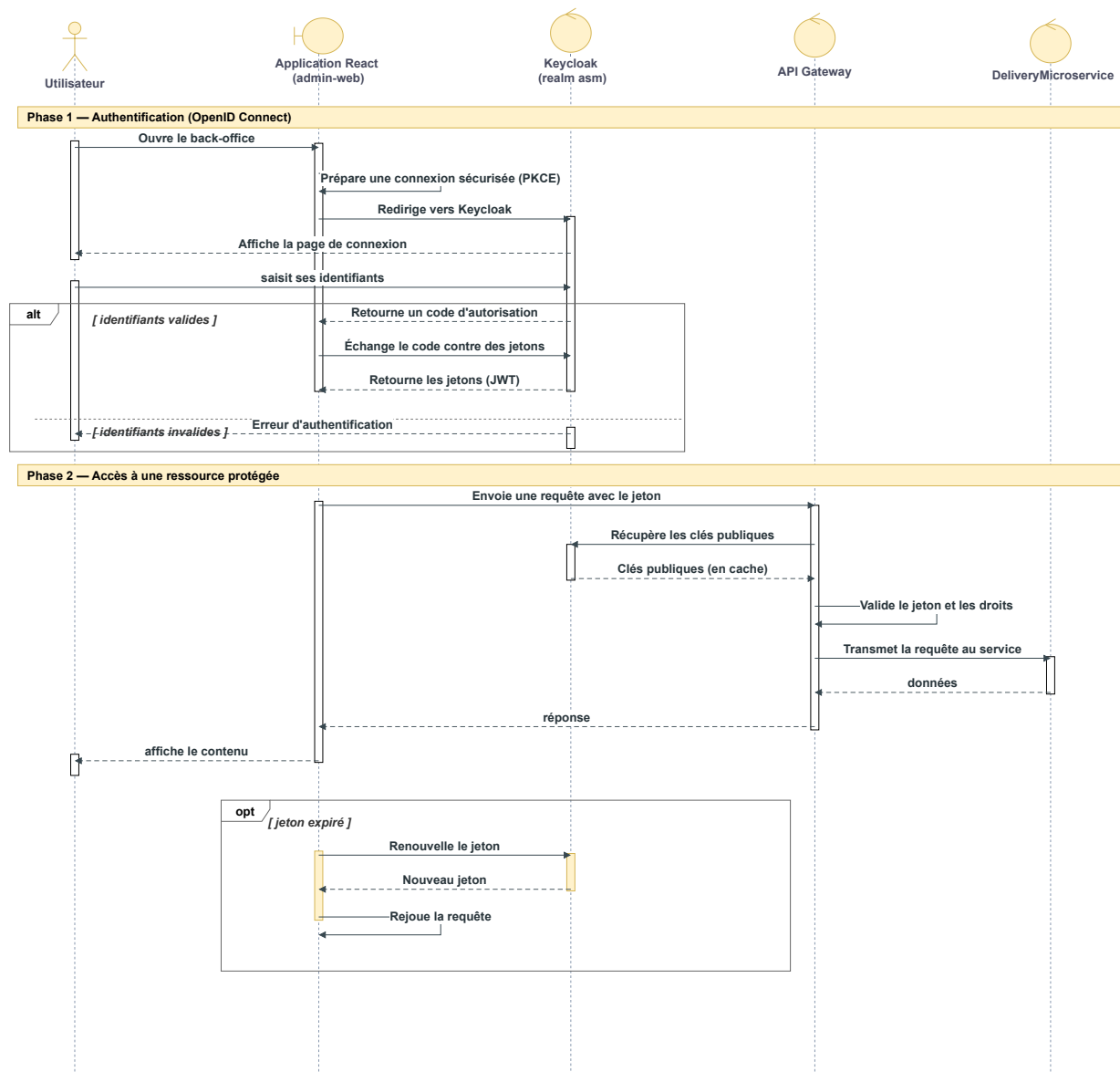


FIG. 4.4 : Séquence d'authentification OpenID Connect et d'accès à une ressource

La séquence se lit en deux phases. La **première** est l'authentification proprement dite. L'application redirige l'utilisateur vers Keycloak, qui vérifie ses identifiants et renvoie un code, lequel est échangé contre un jeton. Le jeton ne transite jamais par l'URL grâce au flux Authorization Code, tandis que PKCE protège le code d'autorisation contre une éventuelle interception.

La **seconde** phase est l'accès à une ressource. La requête, porteuse du jeton, atteint la passerelle. Celle-ci récupère les clés publiques de Keycloak une seule fois, puis les garde en cache. Elle vérifie alors deux choses : que la signature du jeton est valide, et que l'utilisateur porte la permission requise. Elle ne transmet la requête au service que si ces deux conditions sont réunies ; sinon, elle la rejette elle-même et le service n'est jamais sollicité. Un dernier cas ne se produit que si le jeton a expiré. L'application le renouvelle alors sans que l'utilisateur ne s'en aperçoive, puis rejoue la requête d'origine.

4.6 Réalisation

L'écran de connexion est celui de Keycloak, habillé aux couleurs de l'application : le thème reçoit la langue et le mode d'affichage du back-office, pour que le passage par le serveur d'identité ne soit pas visible par l'utilisateur.

La figure 4.5 montre le formulaire de création d'un compte, dans la section « Identités et accès » des paramètres, le menu de choix du rôle déroulé. C'est ici que se lit, concrètement, la matrice de permissions présentée plus haut : le rôle choisi détermine tout ce que le compte pourra faire.

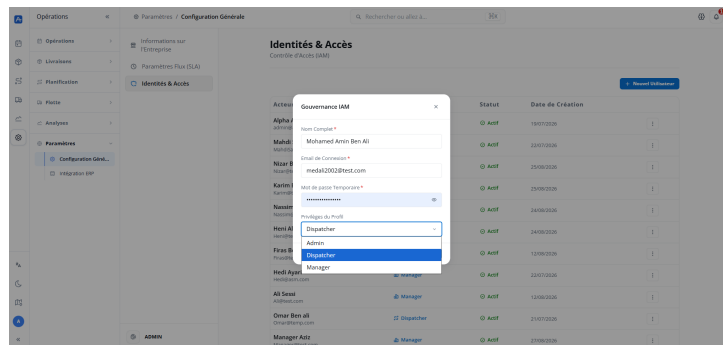


FIG. 4.5 : Création d'un compte et attribution d'un rôle

La figure 4.6 présente l'écran du référentiel des chauffeurs. Ce référentiel, avec ceux des véhicules et des zones, alimentera la planification des tournées au sprint 3.

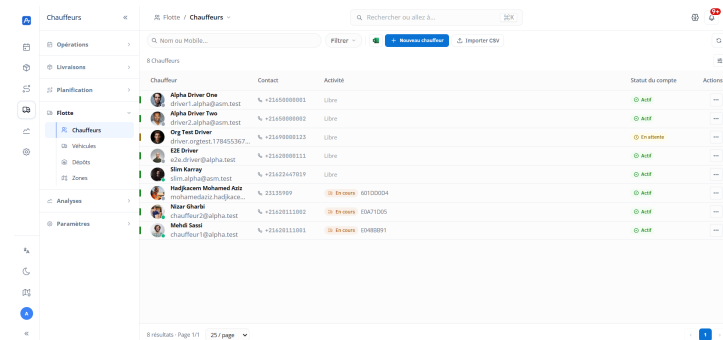


FIG. 4.6 : Référentiel des chauffeurs

Enfin, la figure 4.7 illustre le contrôle d'accès en situation : un compte responsable tente d'atteindre le constructeur de tournées, réservé à qui détient `route:manage`. L'accès est refusé. L'interface, elle, avait déjà masqué l'entrée, mais le refus réel a lieu côté serveur.

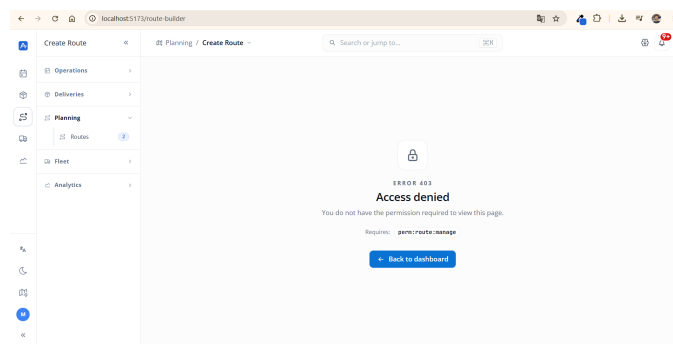


FIG. 4.7 : Accès refusé à une action non permise

4.7 Tests et validation

4.7.1 Scénarios de validation

Le scénario le plus structurant du socle est l'isolation entre deux entreprises, donné dans le tableau 4.5. L'authentification et l'autorisation sont couvertes par les cas d'utilisation détaillés plus haut et par la matrice des permissions.

TAB. 4.5 : TC-S1-03 — Isolation des données entre deux entreprises

Cas de test	TC-S1-03
Description	Une donnée écrite pour l'entreprise A est invisible pour l'entreprise B, et inversement.
Précondition	Deux entreprises provisionnées, chacune avec son schéma.
Scénario	1. Écrire un enregistrement sous A. 2. Sous B, lister les enregistrements. 3. Écrire sous B, puis relire sous A.
Résultat attendu	B ne voit jamais la donnée de A ; chaque entreprise ne voit que la sienne.

4.7.2 Tests automatisés

Ce scénario, comme l'authentification et l'autorisation, est aussi vérifié par des tests automatisés, rejoués à chaque intégration. Le socle du sprint compte **47 méthodes de test**, réparties en deux familles : **40 tests unitaires** sur 6 classes, et **7 tests d'intégration** sur 3 classes, qui s'exécutent sur une vraie base PostgreSQL. Le tableau 4.6 sépare les deux.

TAB. 4.6 : Tests automatisés du sprint 1

Classe de test	Méthodes	Ce qu'elle garantit
Tests unitaires — 6 classes, 40 méthodes		

RbacPolicyTest (passerelle)	8	L'évaluation de la politique d'autorisation
RbacPolicyTest (service Chauffeur)	8	La même politique, re-vérifiée dans le service
JwtAuthConverterTest	8	L'extraction de l'identité et de l'entreprise depuis le jeton
TenantOnboardingServiceTest	6	Le provisionnement d'une entreprise et son annulation
TenantSchemaTest	5	La dérivation du nom de schéma
AnonymousIsNotAuthenticatedTest	5	Le rejet de l'appelant sans jeton
Tests d'intégration (base PostgreSQL réelle) — 3 classes, 7 méthodes		
AuthorizationIT	3	L'accès réservé aux rôles opérateurs
TenantIsolationIT (livraison)	2	L'isolation sur base réelle
TenantIsolationIT (assistant)	2	L'isolation sur base réelle
Total	47	9 classes

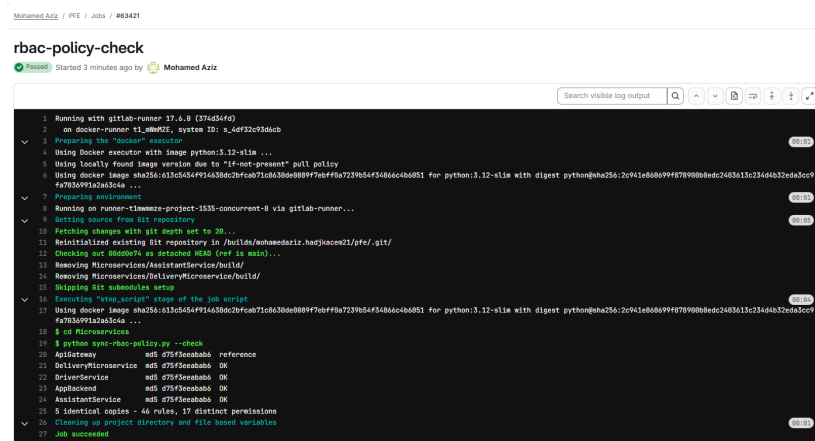
Le nombre de tests compte moins que ce qu'ils protègent. Trois classes méritent qu'on les explique en clair.

RbacPolicyTest vérifie cinq règles de l'autorisation. Un chemin qu'aucune règle ne couvre est refusé. L'API du livreur et le back-office restent séparés. Lire et écrire un même chemin sont deux droits distincts. L'administrateur passe par ses permissions, et non par un statut de super-utilisateur. Enfin, l'ordre des règles est respecté : ce point n'est pas un détail, car l'ordre décide quelle règle s'applique à un chemin, et donc les droits réellement accordés.

JwtAuthConverterTest protège le moment le plus sensible : la lecture de l'entreprise dans le jeton. Une erreur ici ne déclenche aucune alarme : la requête réussit, mais sur le mauvais schéma. Les tests exigent donc que l'entreprise soit lue correctement quand elle est là, et laissée **vide plutôt que devinée** quand elle manque ou est mal formée.

TenantIsolationIT écrit vraiment sous une première entreprise, vérifie que rien n'apparaît sous une seconde, puis fait l'inverse. Ce sont le vrai mécanisme d'Hibernate et le vrai `search_path` de PostgreSQL qui sont mis à l'épreuve : une base simulée testerait l'imitation, pas l'isolation.

Ces tests, unitaires comme d'intégration, s'exécutent tous dans le pipeline d'intégration continue à chaque *commit*. La figure 4.8 montre le job qui s'assure que les cinq services portent la même politique, avec la même empreinte.



The screenshot shows a GitHub Actions workflow run for a job named 'rbac-policy-check'. The job is marked as 'Passed' and was started 3 minutes ago by user 'Mohamed Aziz'. The logs show the following steps:

- Running with gitlab-runner 17.4.0 (174d5f6d)
- on docker-runner ti_mha2f, system ID: 9_d4f32c95dcb
- Preparing the "docker" executor
- Using docker executor with image python:3.12-slim ...
- Using locally found image version due to "if-not-present" pull policy
- Using docker image sha256:d13c5454f91438dc2bfcab71c6a30de089f7ebff8a7239b54f3d66c4b6051 for python:3.12-slim with digest python@sha256:2c941e866699f878908b8edc2483613c234d4b32eda3cc9f478369912a3a5c4a ...
- Preparing environment
- Running on runner-timmmz-project-1535-concurrent-8 via gitlab-runner...
- Getting source from Git repository
- Fetching changes with git depth set to 20...
- Reinitialized existing Git repository in /builds/mohamedaziz.hadjacem21/pre/.git/
- Checking out 08d8b794 as detached HEAD (ref is main)...
- Removing Microservices/AssistantService/build/
- Removing Microservices/DeliveryMicroservice/build/
- Skipping Git submodules setup
- Executing "run_script" stage of the job script
- Using docker image sha256:d13c5454f91438dc2bfcab71c6a30de089f7ebff8a7239b54f3d66c4b6051 for python:3.12-slim with digest python@sha256:2c941e866699f878908b8edc2483613c234d4b32eda3cc9f478369912a3a5c4a ...
- 8 of Microservices
- \$ python app-rbac-policy.py --check
- ApiResponse md5 079f3eeabab4 reference
- DeliveryMicroservice md5 079f3eeabab4 OK
- DriverService md5 079f3eeabab4 OK
- AppHome md5 079f3eeabab4 OK
- AssistantService md5 079f3eeabab4 OK
- 5 identical copies - 44 rules, 17 distinct permissions
- Cleaning up project directory and file based variables
- Job succeeded

FIG. 4.8 : Contrôle en CI de l'unicité de la politique d'autorisation

4.8 Conclusion

Ce sprint a posé le socle sur lequel tout le reste s'appuie. L'identité est déléguée à un serveur spécialisé et prouvée par un jeton signé. L'autorisation tient dans une politique unique, lisible et répliquée à l'identique, qui distingue dix-huit permissions et trois rôles. L'isolation des entreprises clientes est obtenue par un schéma dédié, et vérifiée sur une base réelle plutôt que supposée. Sur ce socle, la gestion des comptes et trois référentiels d'exploitation sont déjà livrés.

Le sprint suivant met à profit cette base pour l'apport le plus original du projet : l'intégration à n'importe quel progiciel de gestion, sans dépendre de son éditeur.

5.1 Introduction

Le socle du sprint précédent sait qui parle au système et ce qu'il a le droit de faire. Il reste à le relier au progiciel de gestion de chaque client. C'est l'objet de ce sprint, et l'apport le plus original du projet.

Le problème n'est pas de parler à *un* progiciel, mais à *n'importe lequel*. Deux variations doivent être absorbées. D'abord, les clients n'ont pas tous le même progiciel, car Odoo domine le marché tunisien des PME sans y être seul. Ensuite, un même progiciel change d'une version à l'autre, puisque entre Odoo 16 et 19 un champ et une méthode changent de nom.

Si ces variations remontent dans le code métier, chaque règle de livraison finit entourée de conditions sur le fournisseur et sa version. Ce sprint met en place une frontière qui les tient à distance, où le métier exprime ce dont il a besoin pendant que des adaptateurs traduisent vers chaque progiciel. S'y ajoutent deux mécanismes : le **mapping des champs**, qui relie le vocabulaire du client à celui du système en vérifiant qu'un champ relié produit bien une donnée exploitable, et une synchronisation qui survit à une panne du progiciel du client.

5.2 Objectifs du sprint et planification des tâches

5.2.1 Objectifs attendus

Ce sprint vise une intégration qui ne dépend d'aucun éditeur. Six résultats sont attendus :

- une **frontière stable** entre le métier et le progiciel, décrite par quelques interfaces qui ne nomment aucun fournisseur.
- deux **connecteurs**, Odoo et ERPNext, derrière cette frontière commune.
- un **mapping guidé** des champs du progiciel vers un contrat canonique, appuyé par un système de types qui refuse un mapping incohérent avant qu'il ne produise une donnée fausse.

- l'**import des commandes** prêtes, avec un aperçu sur de vraies données.
- une **synchronisation** du résultat terrain vers le progiciel, qui survit à une panne.
- la prise en charge de **plusieurs versions** d'un même progiciel, sans code par version.

Le périmètre couvre sept *user stories*, US-05 à US-11, soit **86 points** d'effort. Le tableau 5.1 en donne la décomposition en tâches.

5.2.2 Sprint backlog

Le tableau 5.1 détaille les tâches du sprint, regroupées par *user story*.

TAB. 5.1 : Sprint backlog — sprint 2

ID	User Story	Tâche	Description	Priorité
US-05	Import des commandes prêtes du progiciel (13 pts)	T2-01	Lire la liste des commandes en attente depuis le progiciel	Haute
		T2-02	Prévisualiser le détail d'une commande, champ par champ, avant de l'importer	Haute
		T2-03	Importer une commande, ou un lot de commandes	Haute
US-06	Mapping des champs du progiciel (13 pts)	T2-04	Définir le contrat canonique (27 champs, 5 types, portée en-tête ou ligne)	Haute
		T2-05	Construire le système de types (types source, convertisseurs, verdicts)	Haute
		T2-06	Imposer la compatibilité au choix du champ, à l'enregistrement et à l'aperçu	Haute
		T2-07	Réaliser l'écran de mapping et la confirmation de perte	Moyenne
		T2-08	Signaler qu'un champ a changé de type après la mise en service	Basse
US-07	Intégration agnostique du progiciel (21 pts)	T2-09	Définir les quatre ports d'intégration	Haute
		T2-10	Implémenter l'adaptateur Odoo (JSON-RPC)	Haute
		T2-11	Implémenter l'adaptateur ERPNext (REST)	Haute
		T2-12	Router vers le bon adaptateur et refuser une intégration non certifiée	Haute

ID	User Story	Tâche	Description	Priorité
US-08	Remontée automatique du statut de livraison (8 pts)	T2-13	Écrire le fait métier et son intention de synchronisation dans la même transaction	Haute
		T2-14	Publier les événements vers le progiciel de façon rejouable	Haute
US-09	Reprise d'une synchronisation interrompue (13 pts)	T2-15	Réessayer avec espacement croissant, puis basculer en file de rebut	Haute
		T2-16	Refléter l'état de synchronisation sur la commande et alerter en cas d'échec	Haute
US-10	Récupération des entrepôts comme dépôts (5 pts)	T2-17	Synchroniser les dépôts depuis les entrepôts du progiciel	Moyenne
US-11	Prise en charge de plusieurs versions (13 pts)	T2-18	Déclarer les capacités par liste de candidats	Haute
		T2-19	Résoudre chaque capacité par la présence, jamais par le numéro de version	Haute
		T2-20	Piloter les assistants de confirmation d'Odoo	Haute

5.3 Concepts et mécanismes utilisés

Ce sprint mobilise cinq notions. Elles sont présentées ici brièvement.

Ports et adaptateurs : c'est le patron [COC 05] qui rend l'intégration agnostique. Le métier commence par écrire ce qu'il attend d'un progiciel, sans jamais en nommer un : pousser un résultat de livraison, lire les commandes en attente, résoudre une référence, détecter un changement. Ces quatre besoins forment les *ports*. En face, une classe par progiciel les satisfait à sa manière, et c'est elle que l'on nomme l'*adaptateur*. Le métier ne connaît que le port. Changer de progiciel revient à changer d'adaptateur, sans toucher au reste.

Le contrat canonique : le système nomme les données dont il a besoin, indépendamment de tout progiciel, à l'image de `CUSTOMER_PHONE`, `ITEM_QUANTITY` ou `SCHEDULED_AT`. L'intégrateur relie les champs de son progiciel *vers* ce vocabulaire. Chaque champ canonique déclare une seule fois sa portée (en-tête de commande ou ligne d'article) et son type. Ce type est le même pour tous les clients, un client choisissant *d'où* vient une valeur, jamais *ce qu'elle est*.

Le système de types : un humain relie les champs, mais rien ne garantit qu'un champ texte puisse remplir une quantité. Le système compare le type du champ source au type canonique attendu et rend un verdict, décrit plus loin : SAFE, LOSSY ou UNSUPPORTED. Un mapping incohérent est ainsi refusé avant de produire une donnée fausse.

Le moteur de capacités : pour absorber les versions, les différences ne sont pas écrites dans le code mais *déclarées*. Pour chaque besoin, une liste de noms de champs candidats est donnée. Le connecteur demande au progiciel quels champs il expose réellement, puis choisit le premier présent. Il ne teste jamais un numéro de version : il teste la présence. Une version intermédiaire non prévue fonctionne donc sans modification.

La communication asynchrone par messages : le service de livraison ne parle pas directement au progiciel ; il émet un *événement* sur un *courtier de messages*, que l'adaptateur consomme de son côté. Cette **architecture pilotée par les événements** apporte un *découplage temporel*, l'émetteur et le destinataire n'ayant pas besoin d'être disponibles au même instant. Un progiciel lent ou arrêté ne bloque donc jamais le terrain, puisque le message attend dans la file. Le courtier joue ainsi deux rôles : il **amortit les afflux de messages** quand beaucoup de livraisons sont validées en même temps, et il **ne perd aucun message**, ceux qui échouent définitivement étant mis de côté dans une file de rebut, consultable et rejouable.

L'outbox transactionnel et l'idempotence : [RIC 18] valider une livraison et prévenir le progiciel sont deux écritures dans deux systèmes distincts. Pour ne jamais faire l'une sans l'autre, le fait métier et son intention de synchronisation sont écrits dans **la même transaction** de base de données. Un processus séparé lit ensuite les intentions en attente et parle au progiciel. Comme il peut réessayer, l'adaptateur mémorise l'identifiant de chaque événement qu'il a traité et renvoie le résultat déjà obtenu s'il le revoit passer. Cette propriété, dite *idempotence*, fait qu'un message rejoué après une panne ne crée pas un second mouvement de stock.

5.4 Analyse et spécification des besoins

5.4.1 Diagramme de cas d'utilisation du sprint

La figure 5.1 présente les cas d'utilisation du sprint : la configuration de l'intégration, l'import des commandes, la prévisualisation, la synchronisation des dépôts et le jeu d'une synchronisation en échec.

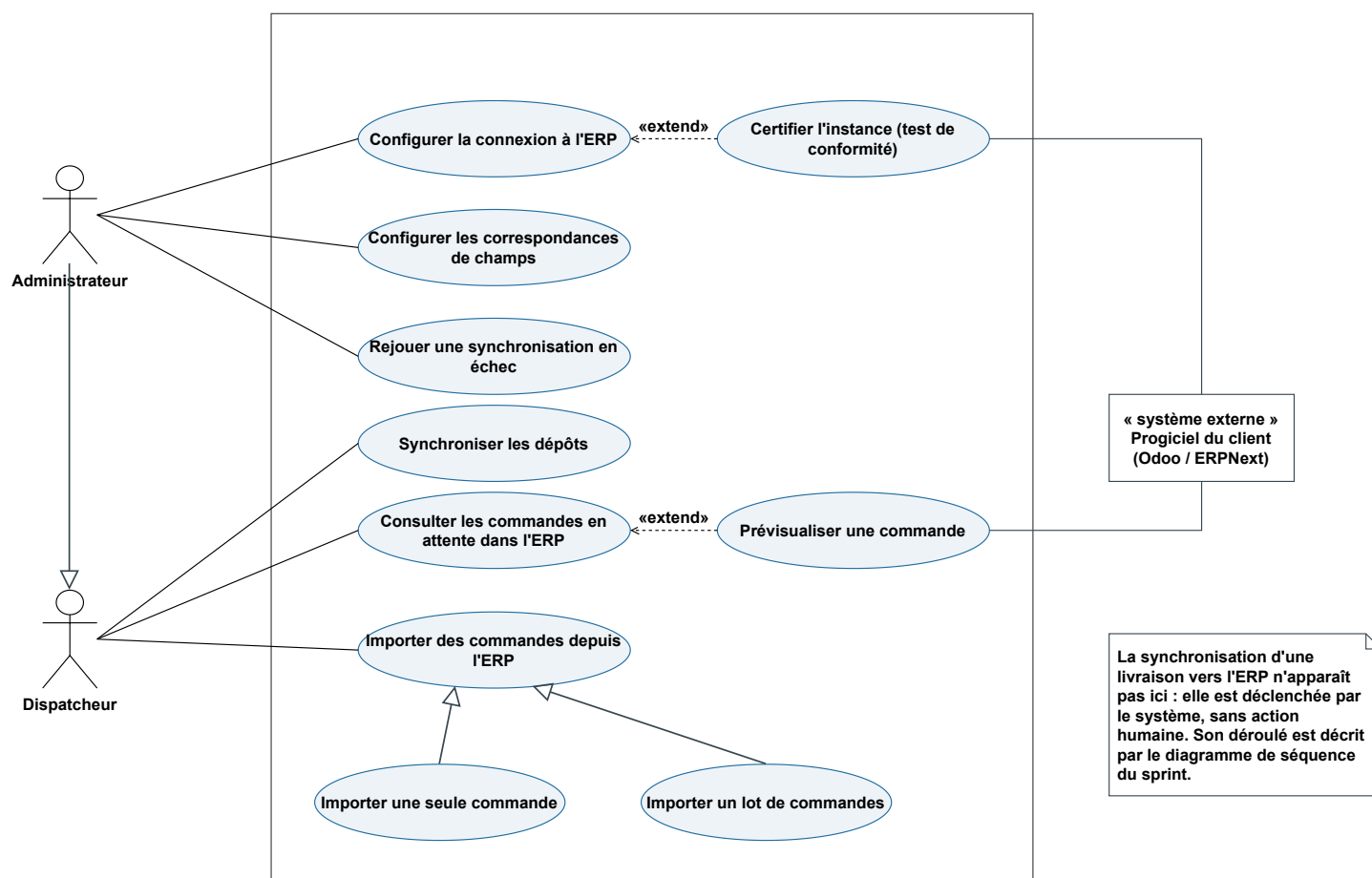


FIG. 5.1 : Diagramme de cas d'utilisation du sprint 2

Deux acteurs se partagent ces cas. L'**administrateur** prépare l'intégration : il connecte le progiciel, le fait certifier, puis relie les champs. Le **dispatcheur** l'exploite au quotidien : il importe les commandes, en prévisualise une, synchronise les dépôts et rejoue au besoin une synchronisation qui a échoué. Le progiciel du client apparaît comme un système externe.

5.4.2 Description détaillée des cas d'utilisation

Trois cas d'utilisation sont détaillés : la préparation (*Configurer et certifier la connexion*), le cœur du sprint (*Mapper les champs*) et l'usage quotidien (*Importer les commandes*).

TAB. 5.2 : Cas d'utilisation « Configurer et certifier la connexion ERP »

Titre	Configurer et certifier la connexion ERP
Acteur	Administrateur
Résumé	Renseigner le fournisseur et les identifiants du progiciel, puis vérifier que la connexion fonctionne avant toute utilisation.
Préconditions	L'administrateur est authentifié et détient la permission <code>erp:config</code> .
Postconditions	L'intégration passe au statut <code>CONNECTED</code> ; les synchronisations deviennent possibles.

Configurer et certifier la connexion ERP (suite)

Scénario nominal	1. L'administrateur choisit le fournisseur (Odoo ou ERPNext). 2. Il saisit l'adresse et les identifiants. 3. Il lance la certification. 4. Le système exécute une vérification en lecture seule et confirme la connexion.
Scénarios alternatifs	3a. Identifiants ou adresse invalides : la certification échoue, le statut reste non connecté. 4a. Tant que le statut n'est pas CONNECTED, toute synchronisation est refusée explicitement.

TAB. 5.3 : Cas d'utilisation « Mapper les champs vers le contrat canonique »

Titre	Mapper les champs vers le contrat canonique
Acteur	Administrateur
Résumé	Relier chaque champ dont le système a besoin à un champ du progiciel, avec un contrôle de compatibilité de type.
Préconditions	L'intégration est certifiée ; l'administrateur détient la permission <code>erp:config</code> .
Postconditions	Chaque champ relié produit une donnée d'un type exploitable ; les correspondances sont enregistrées.
Scénario nominal	1. L'administrateur ouvre l'écran de mapping. 2. Pour un champ canonique, il choisit un champ source. 3. Le système affiche le verdict de compatibilité. 4. Il valide un champ compatible.
Scénarios alternatifs	3a. Le champ source est incompatible (UNSUPPORTED) : il est grisé, le motif est affiché, la sélection est refusée. 3b. La conversion perd de l'information (LOSSY) : une confirmation indique exactement ce qui est perdu avant d'accepter.

TAB. 5.4 : Cas d'utilisation « Importer les commandes »

Titre	Importer les commandes
Acteur	Dispatcheur
Résumé	Récupérer les commandes prêtes du progiciel et les faire entrer dans le système, sans ressaisie.
Préconditions	L'intégration est certifiée et les champs nécessaires sont reliés ; le dispatcheur détient la permission <code>erp:sync</code> .
Postconditions	Les commandes importées existent dans le système, prêtes à être planifiées.

Importer les commandes (suite)

Scénario nominal	1. Le dispatcheur ouvre l'écran d'import. 2. La liste des commandes prêtes s'affiche. 3. Il prévisualise une commande : chaque champ montre sa valeur lue, ou « vide dans le progiciel », ou « illisible », c'est-à-dire présent mais qu'aucune conversion ne sait lire. 4. Il importe une commande ou un lot.
Scénarios alternatifs	3a. Un champ est illisible : l'aperçu affiche la raison, sans bloquer l'import des autres. 2a. Le progiciel est injoignable : l'erreur est signalée, aucune commande n'est importée.

5.5 Modélisation conceptuelle

5.5.1 Diagramme de classes

La figure 5.2 présente la conception « ports et adaptateurs » : les quatre interfaces au centre, et les familles d'adaptateurs qui les réalisent.

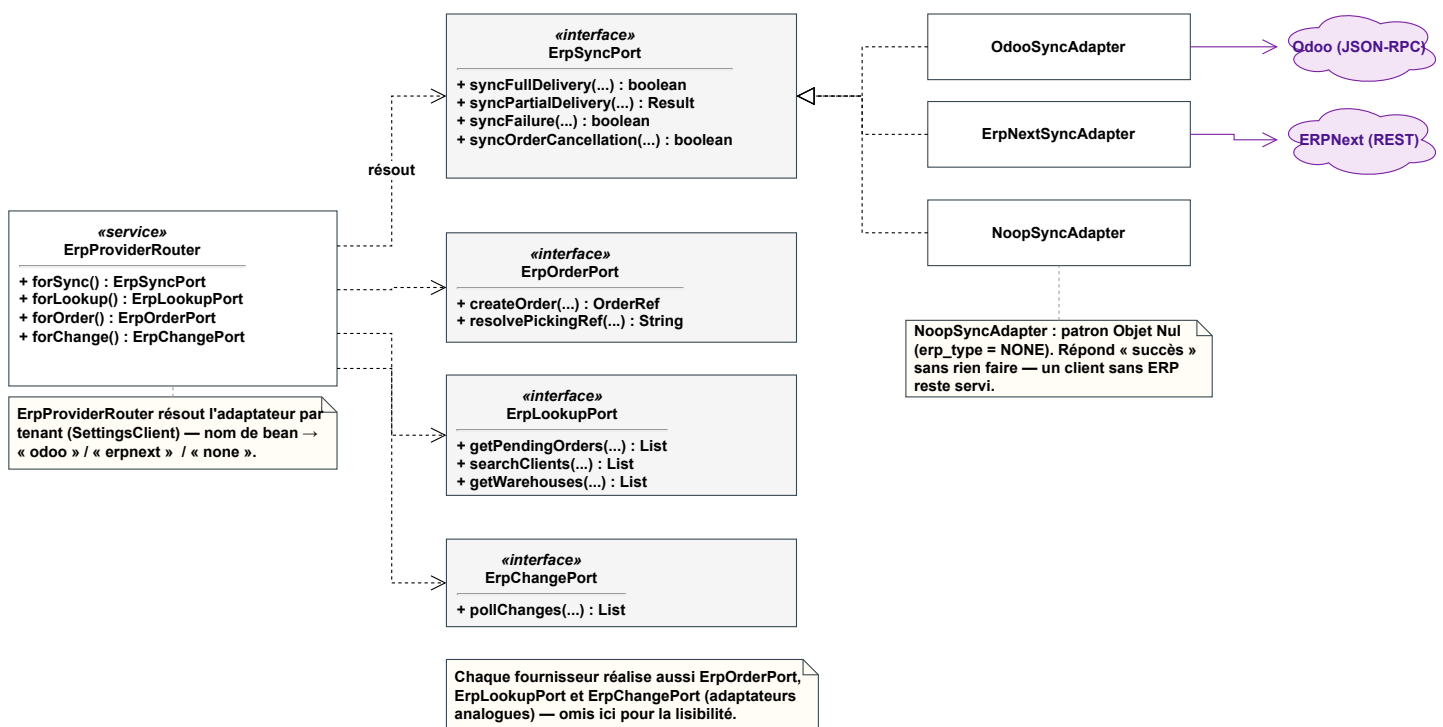


FIG. 5.2 : Ports et adaptateurs de l'intégration ERP

Quatre ports décrivent les besoins du métier. ErpSyncPort pousse un résultat terrain : livré, partiel, échoué ou annulé. ErpLookupPort lit des commandes en attente, des clients, des dépôts. ErpOrderPort crée et résout des références de commande. ErpChangePort détecte qu'une commande a changé côté progiciel. Aucune de ces interfaces ne nomme un progiciel.

Deux familles d'adaptateurs les réalisent. L'adaptateur **Odoo** parle en JSON-RPC. Il est le plus volumineux, car Odoo expose des assistants de confirmation à piloter et plusieurs versions à couvrir. L'adaptateur **ERPNext** parle en REST, une interface plus uniforme.

Le choix de l'adaptateur ne dépend pas de l'appelant, mais du progiciel configuré pour l'entreprise courante. Ajouter un progiciel revient à créer des classes nommées d'après lui, et aucun code existant n'est modifié. C'est cette propriété qui a été vérifiée en ajoutant ERPNext après Odoo, sans toucher aux ports.

5.5.2 Diagrammes de séquence

La figure 5.3 suit une synchronisation, de la validation d'une livraison jusqu'à son écriture dans le progiciel, avec la reprise en cas de panne.

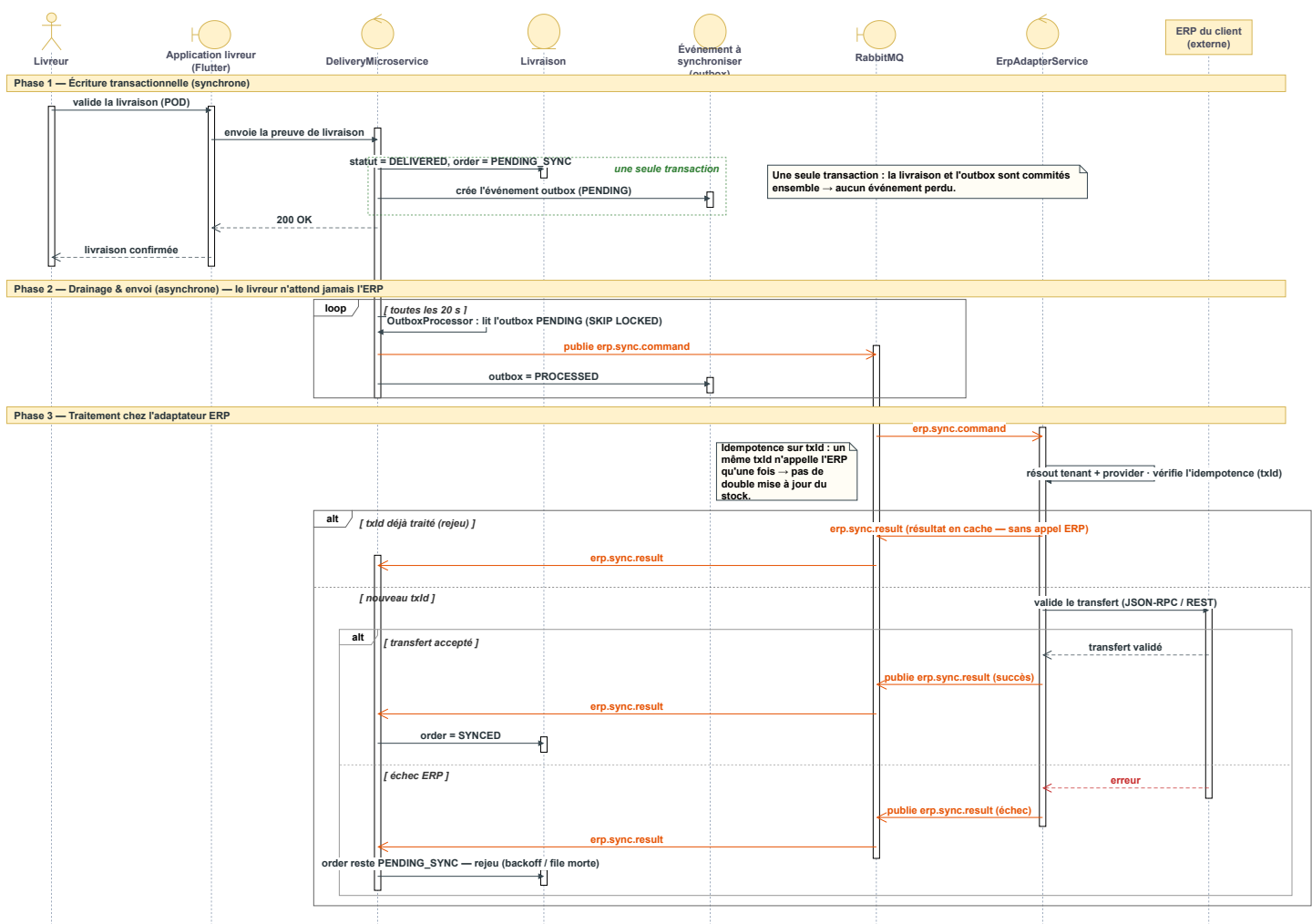


FIG. 5.3 : Séquence de synchronisation ERP et reprise sur panne

La séquence commence dans la même transaction que le fait métier, où la livraison passe à son nouvel état et un événement d'intention de synchronisation est enregistré, ensemble ou pas du tout. Cette garantie ne repose pas sur la vigilance du développeur, car la méthode qui enregistre

l'événement exige une transaction déjà ouverte et échoue sinon. Il devient donc impossible d'avoir une livraison validée sans son événement en attente.

La reprise sur panne se joue à **deux étages qu'il ne faut pas confondre**. Le premier protège la remise du message au courtier, car si celui-ci est indisponible l'outbox réessaie, avec un espacement croissant, jusqu'à réussir à le déposer. Le second étage, **le plus important, protège contre une panne du progiciel**. L'adaptateur, qui consomme la commande, réessaie **cinq fois sur une trentaine de secondes** de joindre Odoo. Passé ce délai, la commande part dans une file de rebut, un résultat d'échec revient, et la commande passe à «échec de synchronisation». L'administrateur est alors prévenu, et la reprise devient un geste manuel, le bouton «resynchroniser». En cas de succès, au contraire, la commande passe à «synchronisée». Dans tous les cas, le livreur n'a jamais attendu : il a validé, l'écran a répondu, il est reparti.

La figure 5.4 détaille l'automate d'états d'un événement **outbox**, celui du premier étage, la remise au courtier. Il précise le cycle de vie que l'on vient de décrire.

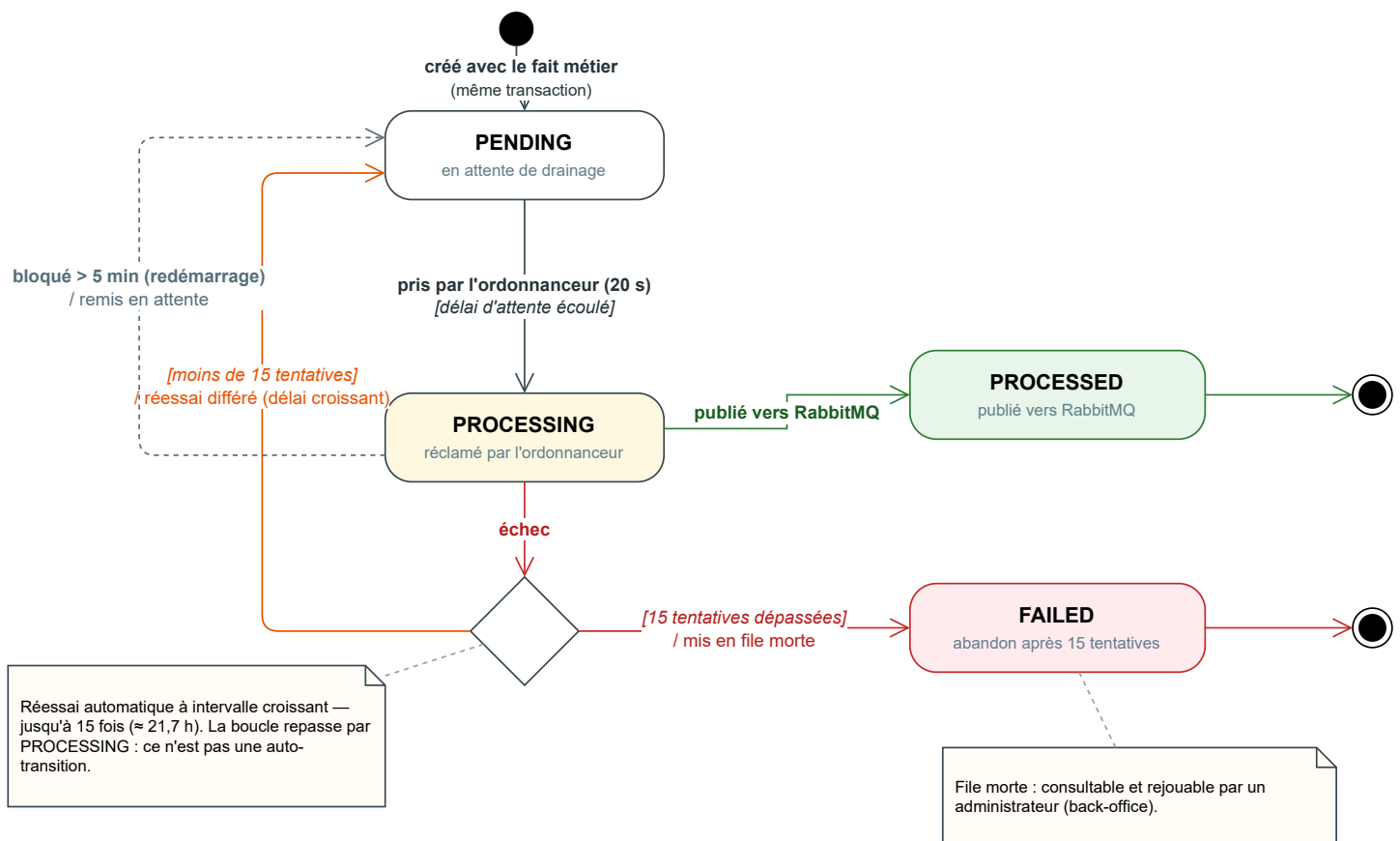


FIG. 5.4 : Automate d'états d'un événement de synchronisation

Un événement naît *en attente*. Il passe *en cours* le temps d'être traité, puis *traité* en cas de succès. En cas d'échec, il retourne *en attente*, mais avec un délai avant la prochaine tentative. Ce délai **double à chaque essai** : les premières tentatives sont rapprochées, pour rattraper une coupure de quelques secondes sans que personne ne le remarque, puis l'espacement grandit

pour ne pas marteler un courtier hors service.

Deux garde-fous complètent cet automate. D’abord, un événement resté *en cours* après un arrêt brutal du service est ramené *en attente* au redémarrage, pour ne pas rester bloqué. Ensuite, après un nombre fixé de tentatives, l’événement devient *en échec* et rejoint la file de rebut. La commande est alors marquée «échec de synchronisation», un dispatcheur est prévenu, et l’événement reste consultable et rejouable depuis le back-office. Un échec définitif est donc visible et réparable, jamais perdu dans un journal.

5.6 Réalisation

La figure 5.5 montre l’écran de connexion au progiciel : le choix du fournisseur, l’adresse, les identifiants, et le statut de certification.

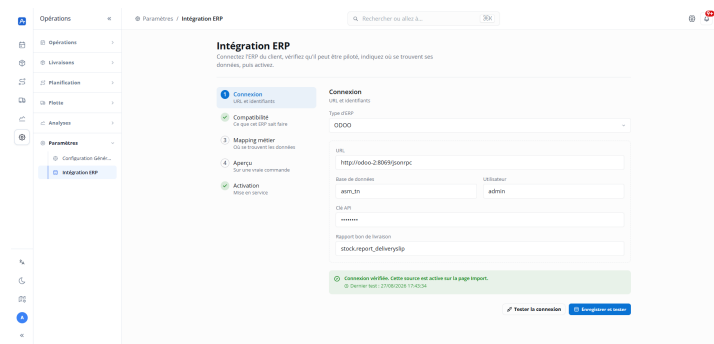


FIG. 5.5 : Configuration de la connexion au progiciel

La figure 5.6 présente l’étape de compatibilité : le système interroge le progiciel et affiche les champs et méthodes qu’il expose réellement. Le connecteur choisit alors le champ présent, au lieu de le deviner d’après un numéro de version, de sorte qu’une version jamais vue fonctionne sans modification tant qu’elle expose les champs attendus.

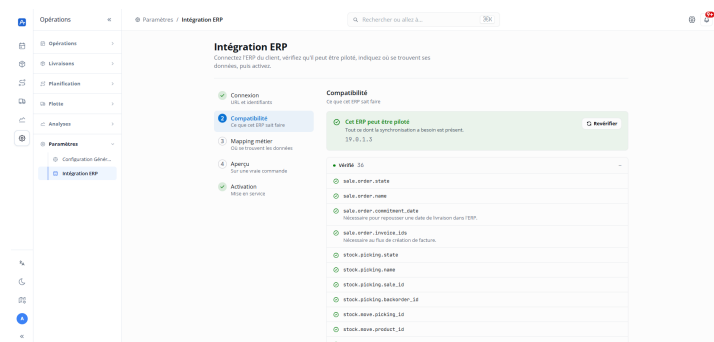


FIG. 5.6 : Détection des capacités du progiciel

La figure 5.7 montre l’aperçu d’une commande sur de vraies données. Chaque champ y affiche l’un des trois états : sa valeur, «vide dans le progiciel», ou «illisible» avec la raison. Une

cellule sans valeur indique ainsi laquelle des deux causes s’applique, vide à la source ou présente mais illisible, au lieu de laisser un blanc que personne ne saurait expliquer.

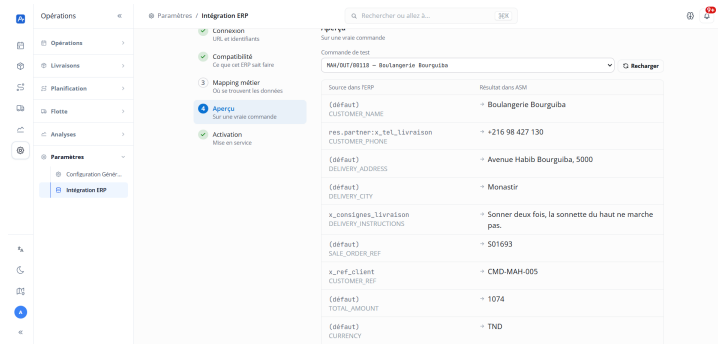


FIG. 5.7 : Aperçu d’une commande avant l’import

La figure 5.8 présente l’écran d’import des commandes prêtes : la liste, et l’import d’une commande ou d’un lot.

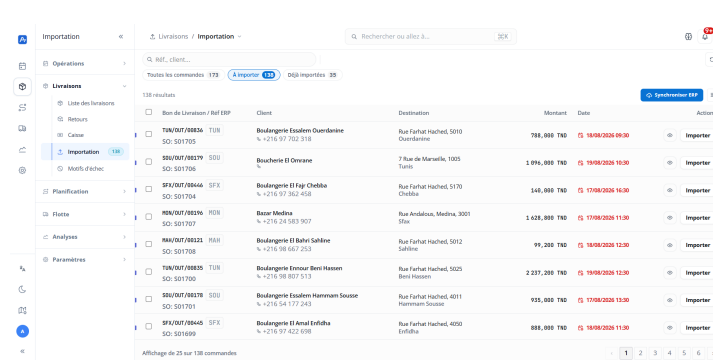


FIG. 5.8 : Import des commandes depuis le progiciel

Enfin, la figure 5.9 montre la file de rebut : les synchronisations définitivement en échec, consultables et rejouables depuis le back-office. Un échec n’est donc jamais perdu dans un journal.

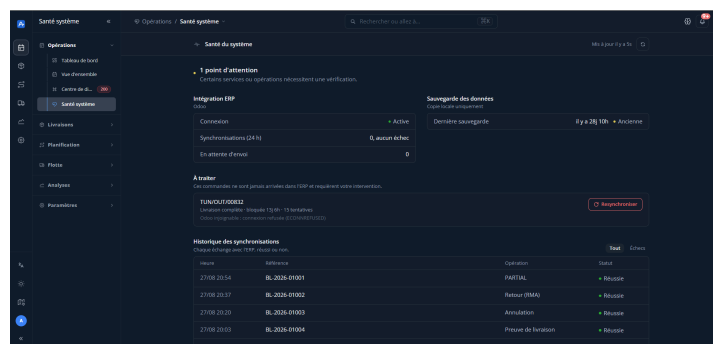


FIG. 5.9 : File morte des synchronisations en échec

5.7 Tests et validation

5.7.1 Scénarios de validation

Deux scénarios couvrent les apports du sprint : l’agnosticité et la reprise sur panne. Ils sont donnés dans les tableaux 5.5 et 5.6. Le contrôle de compatibilité d’un mapping est décrit par le cas d’utilisation *Mapper les champs*, scénarios 3a et 3b.

TAB. 5.5 : TC-S2-01 — Agnosticité vis-à-vis du progiciel

Cas de test	TC-S2-01
Description	Un client configuré sur un progiciel n’est jamais servi par l’adaptateur d’un autre ; le même parcours métier fonctionne sur Odoo et sur ERPNext.
Précondition	Deux entreprises, l’une sur Odoo, l’autre sur ERPNext, toutes deux certifiées.
Scénario	1. Déclencher une synchronisation pour chaque entreprise. 2. Observer l’adaptateur retenu pour chacune.
Résultat attendu	Chaque entreprise est servie par l’adaptateur de son propre progiciel ; le métier appelé est identique.

TAB. 5.6 : TC-S2-03 — Reprise d’une synchronisation en panne

Cas de test	TC-S2-03
Description	Une synchronisation vers un progiciel indisponible est réessayée, puis basculée en file de rebut, sans jamais bloquer le terrain.
Précondition	Une livraison validée, le progiciel du client rendu injoignable.
Scénario	1. Valider la livraison. 2. Laisser les tentatives s’épuiser. 3. Consulter la file de rebut, puis rejouer.
Résultat attendu	La livraison est enregistrée sans attendre ; la commande passe en échec de synchronisation ; l’événement reste rejouable.

5.7.2 Tests automatisés

L’intégration est le composant le plus testé du projet, parce que c’est le plus exposé aux différences entre progiciels. Les tests se répartissent en deux familles : des tests unitaires du mapping et de la résolution de capacités, et une suite d’intégration exécutée contre **deux versions d’Odoo**. Le tableau 5.7 en donne un aperçu ciblé.

TAB. 5.7 : Tests automatisés du sprint 2 (extrait)

Classe de test	Méthodes	Ce qu'elle garantit
Tests unitaires — mapping, types, capacités		
OdooFieldMappingResolverTest	22	La résolution d'un champ Odoo vers le contrat
ErpNextFieldMappingResolverTest	17	La même résolution pour ERPNext
CanonicalConversionContractTest	16	Les verdicts SAFE / LOSSY / UNSUPPORTED
MethodResolverTest	13	Le choix d'une méthode par présence
CapabilityResolverTest	9	Le choix d'un champ par présence, pas par version
Tests d'intégration — Odoo 16 et 19		
Suite bi-version : OdooInfrastructureIT et les trois Workflow*IT	28	Le contrat Odoo réel, éprouvé sur les versions 16 et 19 (100% de réussite)

La suite d'intégration mérite une explication, car elle porte la preuve de l'agnosticité aux versions. Elle réunit **28 tests**, exécutés contre **une instance Odoo 16 et une instance Odoo 19 en même temps**, démarrées par l'intégration continue, tous au vert. Les deux extrémités de la plage supportée étant couvertes, les versions intermédiaires le sont par construction, puisque le connecteur ne teste pas un numéro de version mais la présence des champs et des méthodes. La suite épingle précisément ce qui change d'une version à l'autre : un champ et une méthode qui sont renommés, et la conduite des *assistants de confirmation* d'Odoo, ces fenêtres que valider un transfert fait apparaître et qu'il faut piloter une à une par RPC.

La figure 5.10 montre le job d'intégration continue qui lance cette suite contre les deux versions d'Odoo, toutes deux au vert.

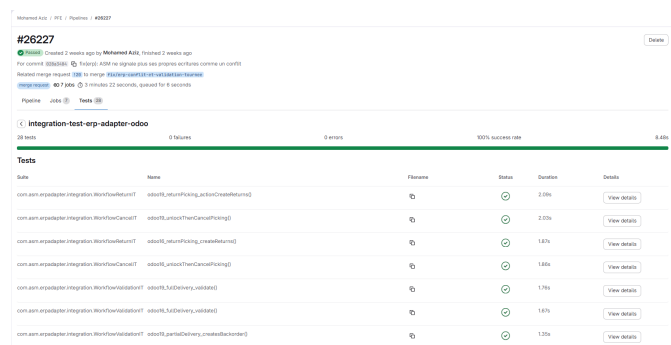


FIG. 5.10 : Exécution en CI de la suite ERP contre deux versions d'Odoo

Un dernier contrôle mérite d'être cité, car il agit avant même les tests. Si un champ canonique déclare un type qu'aucun convertisseur ne sait produire, l'application **ne démarre pas** et liste les champs concernés. Un convertisseur manquant importerait une famille entière de champs

en blanc ; l'échec appartient donc à l'écran d'un développeur, jamais à une livraison.

5.8 Conclusion

Ce sprint a livré l'apport le plus original du projet : une intégration qui ne dépend d'aucun éditeur. Quatre ports tiennent le progiciel à distance du métier, et deux connecteurs, Odoo et ERPNext, les réalisent. Un mapping guidé, appuyé par un contrat canonique et un système de types, garantit qu'un champ relié par un humain produit une donnée exploitable, ou est refusé avant de nuire. Une synchronisation par outbox fait remonter le résultat terrain sans jamais bloquer le livreur, et survit à une panne du progiciel. Enfin, un moteur de capacités absorbe les versions en testant la présence plutôt que le numéro.

Le sprint suivant s'appuie sur ces commandes importées pour construire les tournées et suivre les livraisons en temps réel.

6.1 Introduction

Le chapitre précédent a relié le système au progiciel de gestion de chaque entreprise cliente : les commandes prêtes sont importées avec leurs lignes, leurs adresses et leurs quantités. Une commande importée n'est pourtant pas encore une livraison : il faut la placer dans une tournée, la confier à un chauffeur, suivre son exécution sur le terrain et vérifier que le délai promis au destinataire est tenu.

Ce sprint couvre ce parcours, de la construction de la tournée jusqu'à sa clôture. Il comporte trois volets. Le premier est la **planification** : constituer une tournée à partir des livraisons en attente, en respectant la capacité du véhicule, les horaires du chauffeur et la règle qui veut qu'un colis soit chargé avant d'être livré. Le deuxième est le **dispatch** : affecter la tournée, puis la corriger en cours de journée lorsqu'une livraison doit changer de mains. Le troisième est le **suivi** : l'avancement de chaque livraison apparaît en direct sur l'écran du dispatcheur, un moteur d'échéances signale les retards, et le destinataire consulte l'avancement de son colis par un simple lien, sans compte.

Ces trois volets reposent sur un même mécanisme. Chaque geste du terrain fait avancer une **machine à états** : démarrer la tournée, charger au dépôt, partir vers le client, clôturer l'arrêt. Ce sont ces transitions, et elles seules, qui alimentent le suivi en direct, le calcul des délais et le rapport de fin de journée.

6.2 Objectifs du sprint et planification des tâches

Le périmètre couvre onze *user stories*, US-15 à US-19 et US-28 à US-33, soit **87 points** d'effort. Le tableau 6.1 détaille les tâches du sprint, regroupées par *user story*.

TAB. 6.1 : Sprint backlog — sprint 3

ID	User Story	Tâche	Description	Priorité
US-15	Construction d'une tournée (13 pts)	T3-01	Créer une tournée en brouillon	Haute

ID	User Story	Tâche	Description	Priorité
US-16	Affectation d'un chauffeur et d'un véhicule (8 pts)	T3-02	Ajouter et retirer des livraisons d'une tournée	Haute
		T3-03	Réordonner les arrêts par glisser-déposer	Haute
		T3-04	Affecter un chauffeur et un véhicule à la tournée	Haute
		T3-05	Contrôler la disponibilité du chauffeur et du véhicule	Haute
US-17	Ordre de passage optimisé (13 pts)	T3-06	Interroger le moteur d'itinéraires pour la matrice des durées et le tracé	Haute
		T3-07	Ordonner les arrêts sous contrainte de précedence (charger avant livrer)	Haute
		T3-08	Restituer les gains en distance et en temps par rapport à l'ordre courant	Moyenne
US-18	Validation de la tournée (3 pts)	T3-09	Valider la tournée après contrôle de capacité, de chronologie et de précedence	Haute
US-19	Heure d'arrivée estimée à chaque arrêt (8 pts)	T3-10	Proposer une fenêtre horaire par arrêt à partir des temps de trajet calculés	Moyenne
US-28	Avancement des livraisons en direct (13 pts)	T3-11	Mettre en place la diffusion temps réel cloisonnée par entreprise	Haute
		T3-12	Pousser l'avancement des livraisons sur le centre de dispatch	Haute
US-29	Alerte sur les retards et les échecs (5 pts)	T3-13	Évaluer l'échéance de chaque livraison et signaler les retards	Moyenne
US-30	Réaffectation d'une livraison bloquée (5 pts)	T3-14	Réaffecter une livraison, avec transfert de garde si le colis est déjà en main	Haute
US-31	Suivi public du colis par lien (8 pts)	T3-15	Exposer le suivi par lien, sans authentification et à débit limité	Moyenne
		T3-16	Fermer le lien quatorze jours après la fin de la livraison	Basse
US-32	Demande de retour par le destinataire (3 pts)	T3-17	Enregistrer la demande de retour depuis la page publique	Moyenne
US-33	Approbation d'un retour et reprise en stock (8 pts)	T3-18	Approuver ou refuser une demande de retour	Moyenne

ID	User Story	Tâche	Description	Priorité
		T3-19	Déclencher la remise en stock dans le progiciel	Moyenne

6.3 Concepts et mécanismes utilisés

Ce sprint mobilise quatre notions.

Le moteur d'itinéraires : organiser une dizaine d'arrêts dans le meilleur ordre est un problème de calcul, pas d'intuition [TOT 14]. Le système s'appuie sur *OSRM* [VET 11], un moteur d'itinéraires installé sur le serveur du projet et chargé de la carte routière de la Tunisie. Il fournit trois choses : la durée et la distance de trajet entre chaque paire de points, une proposition d'ordre de visite, et le tracé des routes à afficher sur la carte. Deux raisons ont conduit à l'héberger localement plutôt qu'à appeler un service en ligne : aucun coût par requête, et les adresses des clients ne sortent jamais de l'infrastructure.

Les machines à états : une tournée et une livraison ne changent pas d'état librement. Chacune suit un cycle de vie dont les transitions sont explicites et vérifiées. Une tournée commence en brouillon, devient validée quand le dispatcheur l'engage, passe en cours quand le chauffeur démarre, puis se clôture. Une livraison part du *pool* des non affectées, devient affectée, chargée, en transit, et se termine en livrée, partiellement livrée ou en échec. Chaque transition contrôle sa précondition : on ne démarre pas une tournée qui n'est pas validée, on ne valide pas une tournée dont le chargement dépasse la charge utile du véhicule. L'intérêt n'est pas seulement d'interdire l'incohérence, car chaque passage d'un état à l'autre laisse un horodatage, et ce sont ces horodatages que le calcul des délais et le rapport de tournée exploitent ensuite.

La diffusion en temps réel : le dispatcheur doit voir l'avancement d'une livraison au moment où il se produit, sans recharger sa page. Son navigateur garde donc une connexion *WebSocket* ouverte vers le serveur, et échange dessus des messages *STOMP*, où chaque message porte la destination à laquelle il s'adresse. La diffusion elle-même est déléguée à RabbitMQ. Ce point mérite d'être expliqué, car c'est le seul choix d'architecture réel de ce mécanisme, et la figure 6.1 l'illustre.

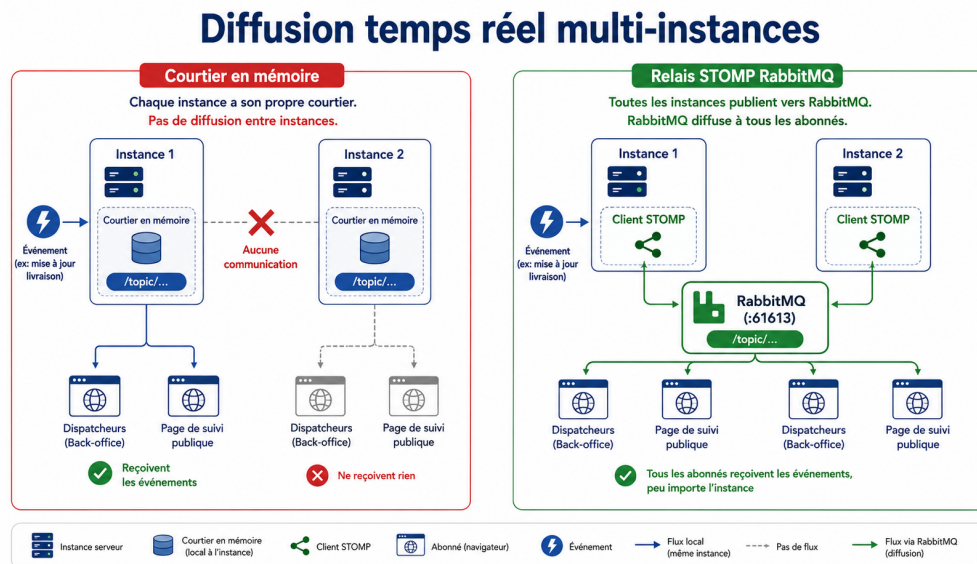


FIG. 6.1 : Diffusion temps réel entre plusieurs instances du service

Le courtier intégré à Spring garde la liste des abonnés dans la mémoire de l'instance qui les a acceptés. Tant qu'une seule instance tourne, cela fonctionne. Dès qu'une seconde est déployée, un dispatcher connecté à la première ne voit plus rien de ce qui se passe sur la seconde. Confier la diffusion à RabbitMQ supprime ce lien à la topologie, puisque toutes les instances publient dans le même courtier, et tout abonné reçoit l'événement quelle que soit l'instance qui le sert.

Le respect des délais : pour chaque livraison, le système compare l'avancement réel à la fenêtre horaire engagée sur l'arrêt et rend un verdict : dans les temps, à risque, ou dépassé. Le calcul dépend de l'étape en cours, car ce qui est promis n'est pas le même selon que la livraison attend d'être planifiée, attend d'être chargée ou est en route. Ce point est développé à la section 6.5.3.

6.4 Analyse et spécification des besoins

6.4.1 Diagramme de cas d'utilisation du sprint

La figure 6.2 présente les cas d'utilisation du sprint : la construction et la validation des tournées, le suivi en temps réel, le traitement des exceptions et le cycle du retour.

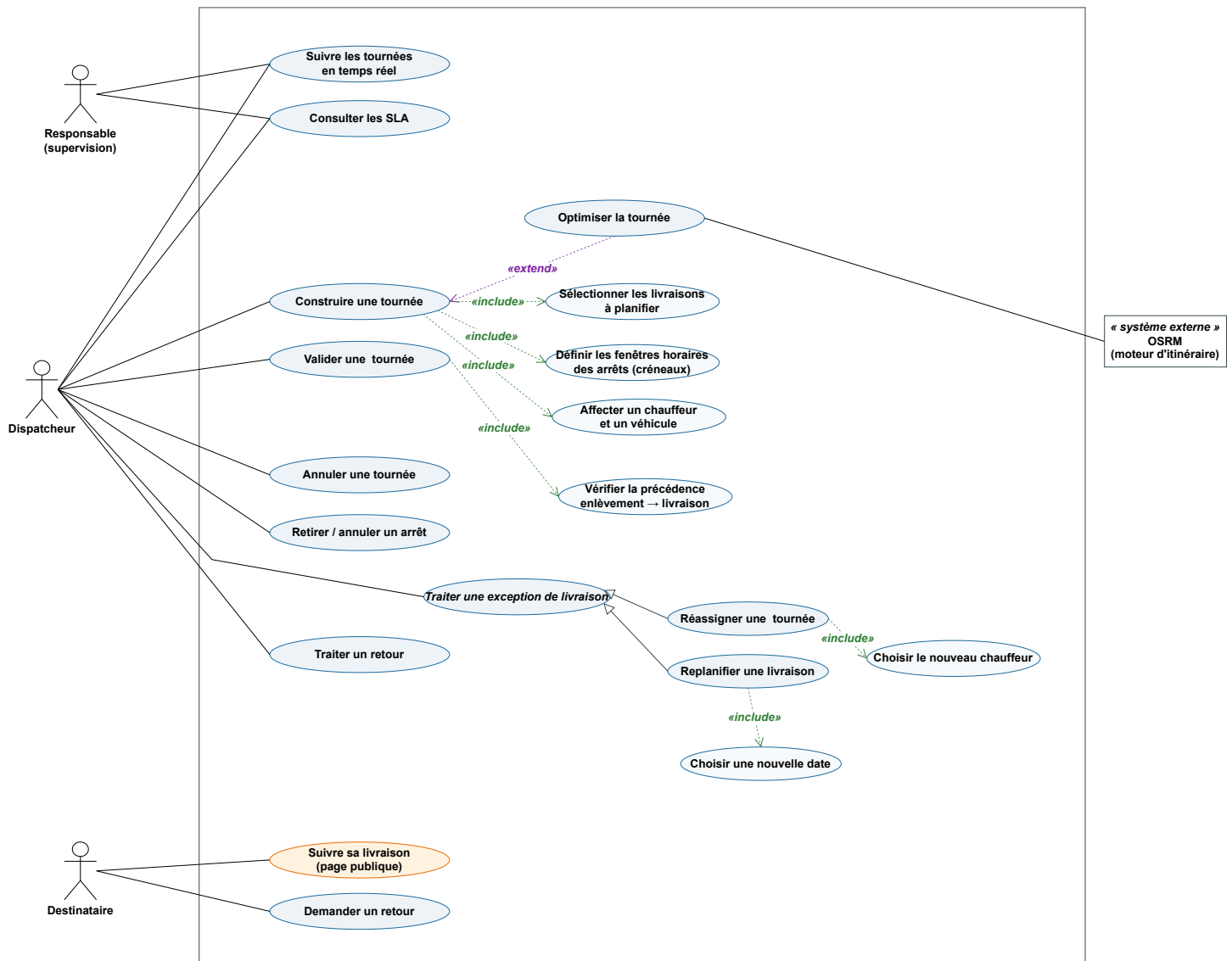


FIG. 6.2 : Diagramme de cas d'utilisation du sprint 3

Trois acteurs se partagent ces cas. Le **dispatcheur** est l'opérateur principal : il construit les tournées, demande une optimisation, valide, annule, retire un arrêt, et traite les exceptions de la journée. Le **responsable** n'agit pas sur les tournées. Il les observe, consulte le respect des délais et lit les rapports de fin de journée. Le **destinataire** n'entre jamais dans l'application, mais suit son colis depuis une page publique où il peut demander un retour. Le moteur d'itinéraires apparaît comme système externe, sollicité par le seul cas d'optimisation.

Deux relations méritent d'être lues attentivement. L'optimisation *étend* la construction, car une tournée se construit sans elle, et c'est le dispatcheur qui décide de la demander. À l'inverse, le choix des livraisons, la définition des fenêtres horaires, l'affectation du chauffeur et le contrôle de la précedence sont *inclus* dans la construction, et aucun ne peut être omis. Enfin, «traiter une exception» est un cas abstrait, réalisé de deux manières selon la situation : réaffecter à un autre chauffeur, ou replanifier à une autre date.

6.4.2 Description détaillée des cas d'utilisation

Trois cas sont détaillés : le plus structurant du sprint, le plus délicat en cours de journée, et celui qui ouvre le système sur l'extérieur.

TAB. 6.2 : Cas d'utilisation « Construire une tournée »

Titre	Construire une tournée
Acteur	Dispatcheur
Résumé	Réunir des livraisons en attente dans une tournée, lui affecter un chauffeur et un véhicule, fixer les fenêtres horaires des arrêts, et éventuellement demander un ordre de passage optimisé.
Préconditions	Le dispatcheur est authentifié et détient la permission <code>perm:route:manage</code> . Des livraisons non affectées existent chez son entreprise. Le chauffeur est actif et le véhicule disponible.
Postconditions	La tournée existe en brouillon, avec ses arrêts ordonnés et leurs fenêtres horaires. Elle n'est pas encore visible du chauffeur ; la validation fait l'objet d'un cas distinct.
Scénario nominal	1. Le dispatcheur ouvre le planificateur. 2. Il crée une tournée en choisissant une date, un dépôt de départ, un chauffeur et un véhicule. 3. Il fait glisser des livraisons du pool vers la tournée. 4. Il fixe la fenêtre horaire de chaque arrêt. 5. Il demande une optimisation, examine l'ordre proposé et les gains annoncés, puis l'applique ou l'ajuste.
Scénarios alternatifs	2a. Le chauffeur est déjà engagé sur une autre tournée au même moment : l'affectation est refusée. 2b. Le véhicule est inactif ou indisponible : l'affectation est refusée. 3a. L'adresse de la livraison n'est pas géolocalisée : l'arrêt ne peut pas être placé sur la carte ni ordonné. 4a. Une fenêtre se termine avant de commencer, ou empiète sur la précédente : le système refuse et désigne l'arrêt en cause. 5a. Le moteur d'itinéraires ne répond pas : l'ordre en place est conservé, sans blocage.

TAB. 6.3 : Cas d'utilisation « Réaffecter une livraison à un autre chauffeur »

Titre	Réaffecter une livraison à un autre chauffeur
Acteur	Dispatcheur
Résumé	Confier une livraison à un autre chauffeur lorsque le titulaire ne peut pas la mener à terme, en distinguant le cas où le colis est encore au dépôt de celui où il est déjà dans le véhicule.

Réaffecter une livraison à un autre chauffeur (suite)

Préconditions	Le dispatcheur détient la permission <code>perm:dispatch:operate</code> . La livraison est encore en jeu : en attente de planification, programmée, chargée ou en transit. La tournée d'accueil, si elle est désignée, appartient bien au chauffeur visé.
Postconditions	La livraison est rattachée à la tournée du nouveau chauffeur, à la place et dans la fenêtre horaire indiquées. Si le colis était déjà en main, elle attend un transfert de garde avant de repartir.
Scénario nominal	1. Le dispatcheur ouvre la livraison en difficulté depuis le centre de dispatch. 2. Il demande la réaffectation. 3. Le système lui recommande les chauffeurs en service, classés par le temps de trajet séparant leur position actuelle de l'adresse de livraison. 4. Il en choisit un, désigne la tournée d'accueil, et fixe la position de l'arrêt ainsi que sa fenêtre horaire. Il peut joindre une note de passation, sans y être tenu ; à défaut, une mention par défaut est enregistrée, de sorte qu'un changement de garde laisse toujours une trace. 5. Si le colis est encore au dépôt, la livraison rejoint directement la tournée d'accueil, qui est recalculée, et le nouveau chauffeur en est averti. 6. S'il est déjà dans le véhicule du chauffeur d'origine, le système ouvre un transfert de garde : la livraison ne repart qu'une fois la remise confirmée entre les deux chauffeurs.
Scénarios alternatifs	2a. La livraison est livrée, annulée ou en échec : la réaffectation est refusée, car un échec se traite par une replanification à une autre date, non par un changement de porteur. 3a. Aucun chauffeur ne transmet de position récente : la recommandation reste vide, et le dispatcheur choisit lui-même dans la liste des chauffeurs en service. 4a. L'arrivée de la livraison dépasse la charge utile du véhicule d'accueil : le système bloque, sauf si le dispatcheur assume explicitement la surcharge. 4b. La fenêtre horaire saisie chevauche celle d'un arrêt voisin, ou se termine avant de commencer : l'insertion est refusée et l'arrêt en cause est désigné.

TAB. 6.4 : Cas d'utilisation « Demander un retour »

Titre	Demander un retour
Acteur	Destinataire

Demander un retour (suite)

Résumé	Signaler depuis la page publique de suivi qu'un article reçu doit être repris, et déclencher le cycle de traitement du retour.
Préconditions	Le destinataire dispose du lien de suivi de sa livraison. Celle-ci est livrée ou partiellement livrée, et le lien n'a pas expiré.
Postconditions	Une demande de retour est enregistrée en attente d'arbitrage, avec les articles concernés et les éventuelles photographies ; le bureau en est averti.
Scénario nominal	1. Le destinataire ouvre son lien de suivi. 2. Il choisit les articles à retourner et en indique le motif. Il peut joindre des photographies, sans y être tenu. 3. La demande est enregistrée en attente. 4. Le dispatcheur l'approuve ; une collecte est créée et insérée dans une tournée comme un arrêt ordinaire.
Scénarios alternatifs	1a. La livraison est terminée depuis plus de quatorze jours : le lien ne répond plus. 3a. Plus de quatre demandes ont été émises en vingt-quatre heures pour cette livraison : la demande est rejetée. 4a. Le dispatcheur refuse la demande en la motivant : aucune collecte n'est créée.

6.5 Modélisation conceptuelle

6.5.1 Diagramme de classes

La figure 6.3 présente la conception du noyau de planification : un contrôleur, quatre services aux responsabilités séparées, les deux entités du domaine et les composants qui les entourent.

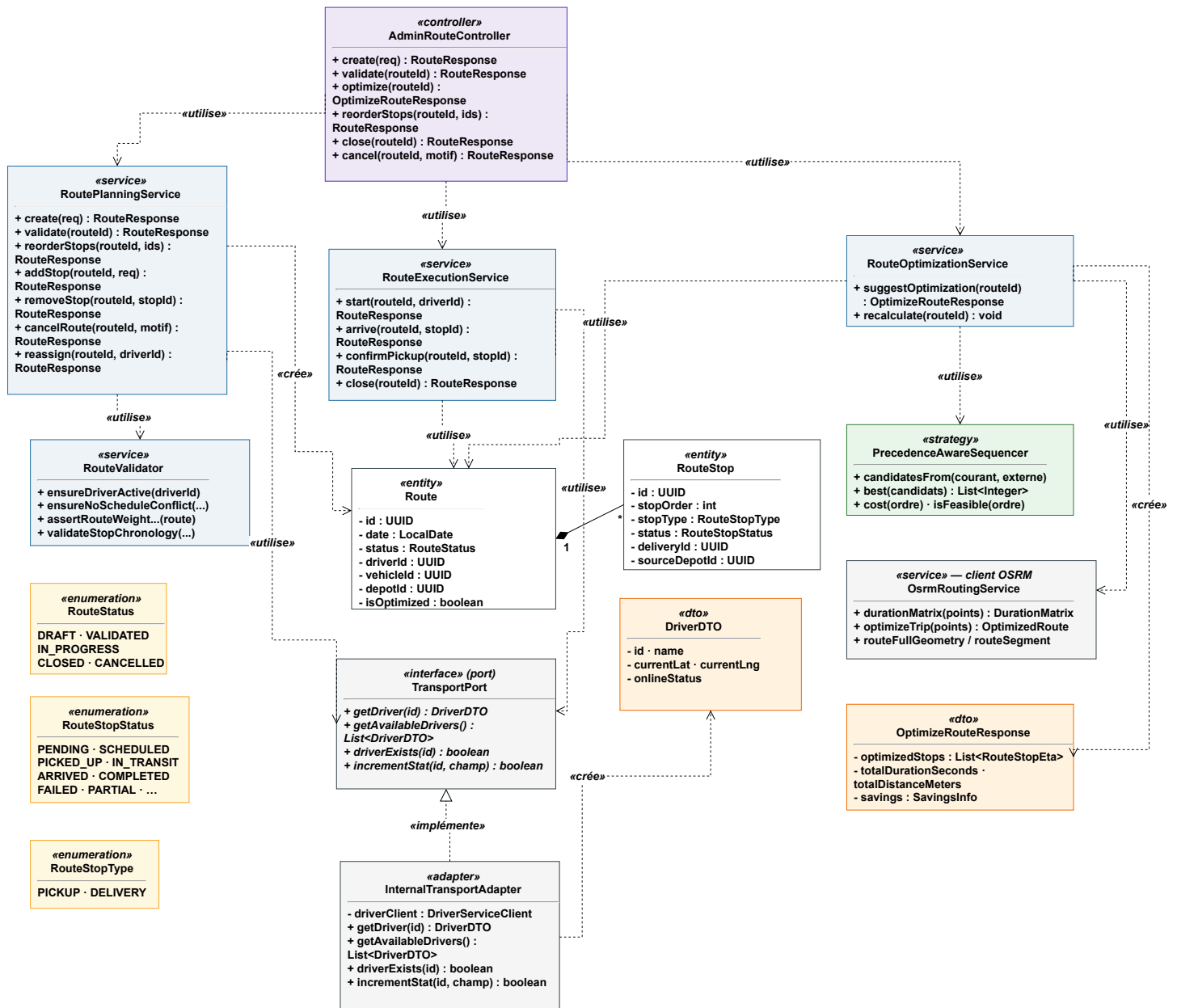


FIG. 6.3 : Classes de conception du sprint 3

Le modèle du domaine tient en deux entités. Une Route porte la date, le dépôt de départ, le chauffeur, le véhicule et l'état du cycle de vie. Elle est composée de RouteStop : un arrêt connaît son rang dans la séquence, son état, la livraison qu'il sert et son type, qui vaut chargement au dépôt ou livraison chez un client. Les trois énumérations en marge donnent les valeurs possibles de ces états.

Le traitement est réparti entre quatre services, et ce découpage suit les moments réels du travail plutôt que les entités. RoutePlanningService porte tout ce que le dispatcheur fait sur une tournée avant qu'elle ne parte. RouteExecutionService porte ce que la journée lui fait subir ensuite : démarrage, chargement confirmé, arrivée à un arrêt, clôture.

RouteOptimizationService n'est appelé que pour proposer un ordre. Enfin, RouteValidator

rassemble les règles qui déterminent si une tournée est légale : chauffeur actif, absence de conflit d'horaire, charge utile respectée, fenêtres horaires cohérentes.

Deux composants complètent l'ensemble. `OsrRoutingService` est le client du moteur d'itinéraires : il fournit la matrice des durées, une proposition d'ordre et les tracés.

`PrecedenceAwareSequencer` est la stratégie qui décide de l'ordre final. Les séparer n'est pas gratuit : le moteur d'itinéraires sait calculer un trajet court, mais il ignore qu'un colis doit être chargé avant d'être livré. Le premier fournit donc les coûts, le second choisit l'ordre en tenant compte de cette contrainte. La section suivante détaille ce partage.

Enfin, `TransportPort` règle une question simple : la planification a besoin de savoir si un chauffeur est actif et où il se trouve, mais ces données appartiennent au service `Chauffeur` et s'obtiennent par le réseau. Plutôt que de disperser cet appel partout, le service de livraison écrit d'abord ce dont il a besoin dans une interface courte, sans dire d'où viendra la réponse.

`InternalTransportAdapter` la réalise en interrogeant le service voisin, et un paramètre de configuration désigne l'adaptateur à employer. Une entreprise qui confierait ses tournées à un transporteur extérieur plutôt qu'à ses propres chauffeurs n'aurait qu'à placer là un adaptateur qui parle à ce transporteur, et la planification, qui ne connaît que l'interface, resterait inchangée. C'est le patron des ports et adaptateurs du sprint précédent, appliqué cette fois à la flotte.

6.5.2 Diagrammes de séquence

La figure 6.4 suit la construction d'une tournée, depuis la sélection des livraisons jusqu'à la validation, en passant par l'optimisation.

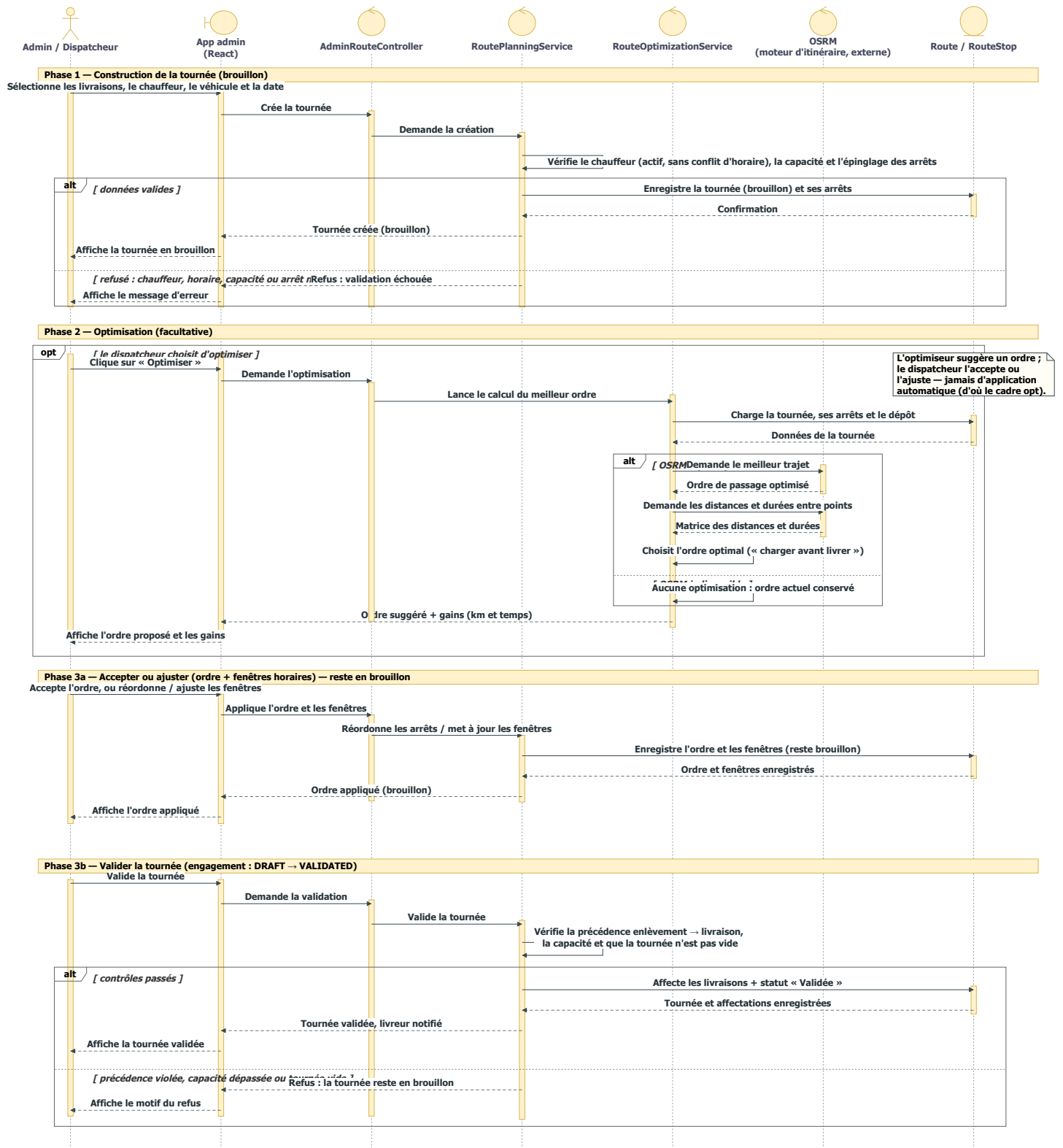


FIG. 6.4 : Séquence de construction, d'optimisation et de validation d'une tournée

La séquence se lit en trois phases. La première crée la tournée en brouillon, après vérification du chauffeur, du véhicule, de la charge et du positionnement des arrêts. La deuxième, facultative, demande une optimisation. La troisième applique l'ordre retenu, puis engage la tournée.

Le cadre *opt* de la phase 2 traduit la décision centrale du sprint : **l'optimiseur propose, le dispatcheur dispose**. Une version antérieure appliquait directement l'ordre calculé, et elle a été retirée. Un algorithme ignore le client qui n'ouvre qu'à quatorze heures ou la rue en travaux ; le dispatcheur les connaît sans pouvoir les saisir. Le cadre *alt* voisin couvre l'indisponibilité du moteur d'itinéraires, auquel cas l'ordre en place est conservé et la tournée reste modifiable. Une panne retire une aide, elle n'empêche pas de planifier la journée.

Le calcul de l'ordre se fait à deux. Le moteur d'itinéraires donne le coût de chaque trajet et propose une tournée, mais sa proposition ignore la précédence et peut placer une livraison avant le chargement du dépôt qui la fournit. Le séquenceur tranche donc lui-même. Il compare trois ordres : celui en place, celui du moteur et un troisième construit de proche en proche. Il écarte ceux qui violent la précédence, et garde le moins coûteux. Comme l'ordre en place est l'un des candidats, la suggestion ne peut pas être moins bonne que ce que le dispatcheur a déjà sous les yeux.

Les heures d'arrivée se règlent au même endroit. Le système part de l'heure de départ, ajoute les durées de trajet et le temps passé sur place, et en déduit une **fenêtre horaire proposée pour chaque arrêt**. Mais il n'enregistre jamais l'heure qu'il a calculée. Ce qui est conservé, et ce sur quoi les retards seront mesurés, c'est la fenêtre que le dispatcheur a acceptée ou corrigée. Une durée lue sur une carte ignore le trafic du jour, et juger un chauffeur là-dessus reviendrait à l'évaluer sur une prévision que personne n'a validée. Une fenêtre qu'un dispatcheur a engagée est, elle, une promesse assumée.

La figure 6.5 donne la machine à états d'une livraison, celle que les gestes du terrain font avancer et sur laquelle tout le suivi s'appuie.

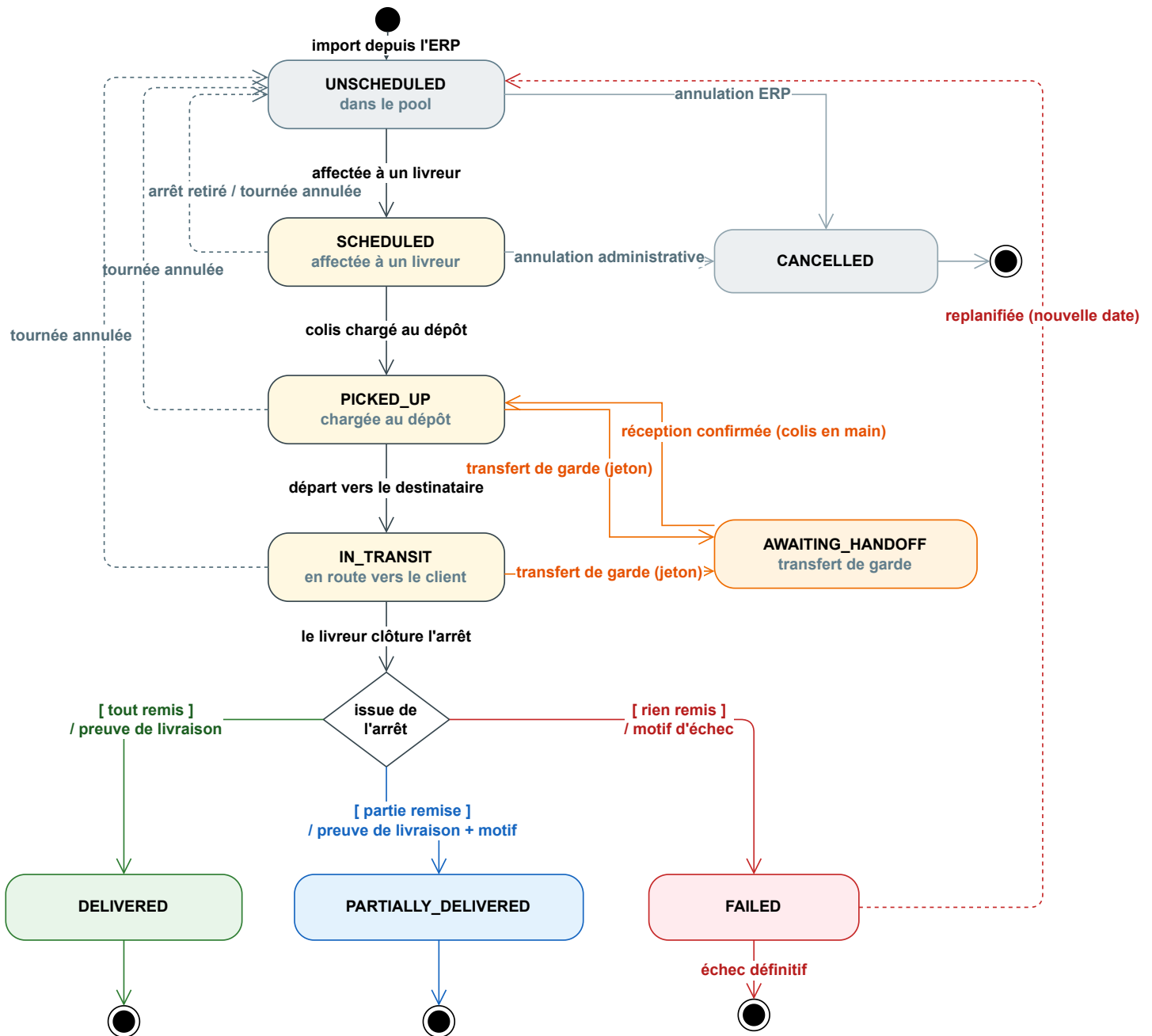


FIG. 6.5 : Machine à états d'une livraison

Une livraison naît dans le *pool* à l'import depuis le progiciel. L'affectation à une tournée la fait passer en programmée, le chargement au dépôt en chargée, le départ en transit. La clôture de l'arrêt par le chauffeur mène à l'un des trois résultats possibles, selon ce qui a réellement été remis : tout, une partie, ou rien.

Deux particularités de ce diagramme portent des décisions de conception.

D'abord, l'issue de l'arrêt est un choix guidé par les quantités, non une déclaration du chauffeur. Le résultat se déduit de ce qui a été remis, et si aucune ligne ne l'a été, c'est un échec, pas une livraison partielle vide. Cette distinction évite de remonter au progiciel un

mouvement de stock sans contenu.

Ensuite, la livraison partielle est un état terminal, au même titre que la livraison complète. Le reliquat ne prolonge pas la livraison en cours, mais donne lieu à une autre livraison, créée à partir de la commande. Une livraison close reste ainsi le compte rendu fidèle d'un passage, et non un dossier qu'on rouvre.

6.5.3 L'évaluation des échéances

Le suivi des retards demande une précision que le diagramme d'états ne montre pas. Une livraison en retard ne l'est pas de la même façon selon l'étape où elle se trouve, et la personne responsable change en cours de route.

La figure 6.6 présente les quatre phases retenues : ce que chacune promet, l'échéance à laquelle elle se mesure, et la personne qui en répond.

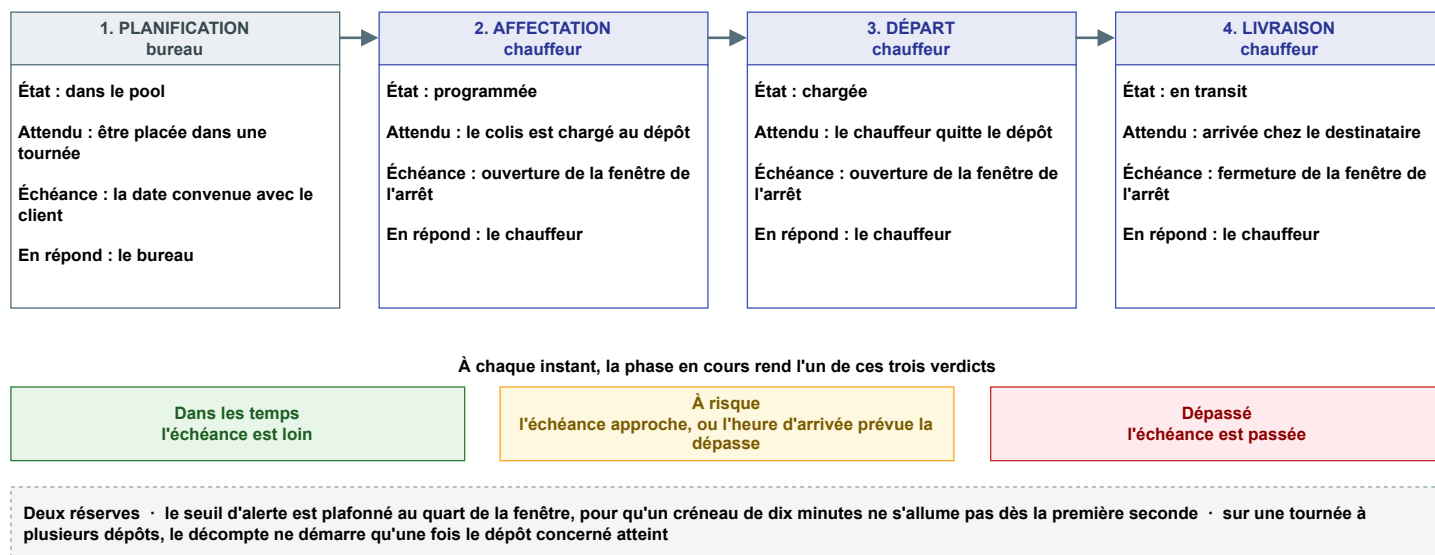


FIG. 6.6 : Les quatre phases d'échéance d'une livraison

Le déplacement de responsabilité se lit sur la dernière ligne des colonnes, où seule la planification engage le bureau quand les trois autres engagent le chauffeur. C'est ce qui permet à un responsable de distinguer une équipe qui planifie trop tard d'une flotte qui roule trop lentement.

Une dernière décision mérite d'être relevée : une livraison en échec reçoit elle aussi un verdict. Une tentative faite dans les temps mais infructueuse est enregistrée comme respectée, tandis qu'une tentative faite hors délai est un dépassement. La distinction sépare ce qui relève de l'organisation de ce qui relève du client absent, alors qu'un tiret dans la colonne aurait mélangé les deux.

6.6 Réalisation

La figure 6.7 montre le planificateur, organisé en trois zones. À gauche, la liste des tournées de la journée. Au centre, la tournée sélectionnée, ici en brouillon, avec ses arrêts dans l'ordre : référence de la commande, client, adresse, dépôt de chargement et fenêtre horaire. À droite, la carte situe ces arrêts, et le tableau placé sous elle donne les commandes restant à planifier. Le bandeau supérieur porte les compteurs et les filtres qui permettent de restreindre cette liste.

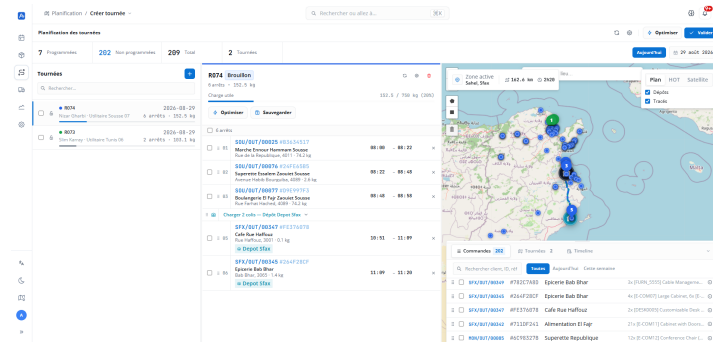


FIG. 6.7 : Planificateur de tournées

La figure 6.8 présente l'aperçu d'optimisation. L'ordre proposé est affiché arrêt par arrêt, avec la fenêtre horaire suggérée pour chacun, et le gain annoncé en distance et en durée par rapport à l'ordre courant, ici sept kilomètres et demi et dix minutes. La carte trace l'itinéraire proposé. Deux boutons closent l'aperçu : appliquer ou rejeter. Rien n'est enregistré avant ce geste, et la tournée reste en brouillon après lui.

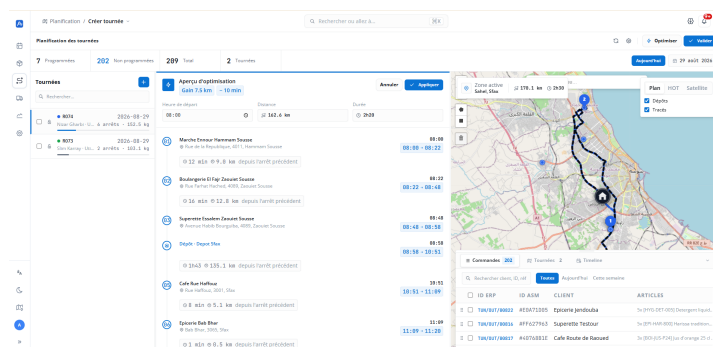


FIG. 6.8 : Aperçu d'optimisation, avec les gains et les fenêtres proposées

La figure 6.9 montre la fiche d'une tournée. Un bandeau d'indicateurs en résumé l'état : avancement, distance, ponctualité, véhicule et durée totale. En dessous, la carte trace l'itinéraire réellement suivi, et la séquence des arrêts est reprise à droite avec, pour chacun, le client et le montant concerné.

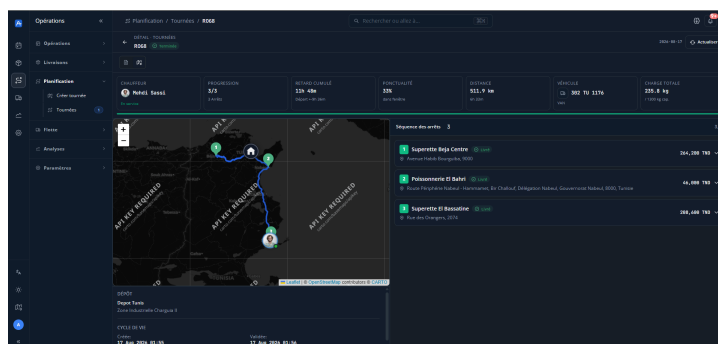


FIG. 6.9 : Fiche d'une tournée en cours

La figure 6.10 présente le centre de dispatch, l'écran sur lequel arrivent les événements poussés en temps réel. La colonne de gauche empile les tournées du jour avec leurs livraisons, chacune portant l'état où elle en est. Le panneau central détaille la livraison sélectionnée : le destinataire, ses coordonnées, et la chronologie de son parcours.

La carte, elle, n'appartient pas à cet écran. C'est une fenêtre flottante, que le dispatcheur déplace, redimensionne, réduit ou ferme, et qui reste ouverte quelle que soit la page consultée tant qu'il ne la ferme pas. Elle suit les positions des chauffeurs en direct et peut se centrer sur une tournée ou sur un chauffeur désigné depuis une autre vue. Ce choix vient du métier : un dispatcheur ne consulte pas une carte, il la garde sous les yeux pendant qu'il travaille ailleurs.

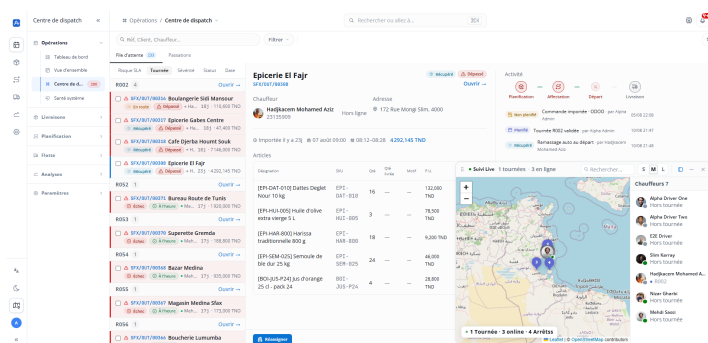


FIG. 6.10 : Centre de dispatch

La figure 6.11 montre le panneau de réaffectation. Les chauffeurs en service sont classés par le temps de trajet qui sépare leur position actuelle de l'adresse de livraison, avec la distance correspondante. Le classement s'appuie sur le moteur d'itinéraires, et non sur la distance à vol d'oiseau, car sur un réseau routier réel, le chauffeur le plus proche sur la carte n'est pas toujours celui qui arrivera le premier. Seuls les chauffeurs dont la position est récente sont proposés, une position ancienne ne disant rien de l'endroit où ils se trouvent maintenant.



FIG. 6.11 : Réaffectation d'une livraison, chauffeurs classés par temps d'accès

La figure 6.12 montre enfin la fiche d'un retour. L'origine y est explicite, « créé par le client depuis le suivi public », ce qui distingue une demande du destinataire d'une reprise décidée au bureau. Viennent ensuite les articles concernés, chacun jugé revendable ou endommagé, les photographies jointes par le client, l'état de la synchronisation avec le progiciel et la chronologie de la demande.

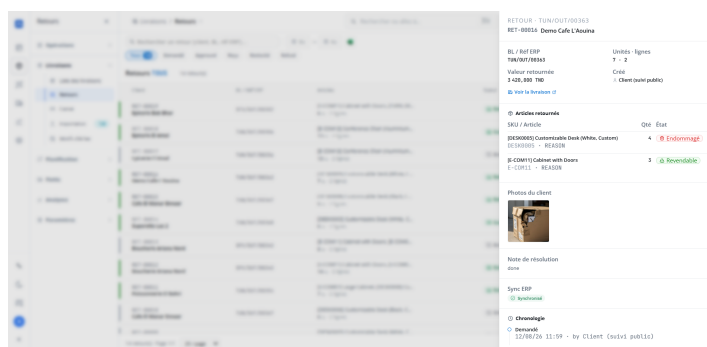


FIG. 6.12 : Traitement d'une demande de retour

Le modèle distingue en revanche deux moments que l'on aurait pu confondre. Un retour reçu signifie que le colis est physiquement revenu au dépôt. Un retour remis en stock signifie qu'il a été jugé revendable et réintégré dans le progiciel. Un article rendu abîmé s'arrête donc au premier moment et ne repart jamais vers le stock. C'est cette séparation qui empêche un article détérioré de réapparaître comme disponible à la vente.

6.7 Tests et validation

6.7.1 Scénarios de test

Trois scénarios couvrent les apports du sprint : l'ordre de passage, la mesure des délais et la réaffectation en cours de journée.

TAB. 6.5 : TC-S3-01 — Respect de la précédence dans l'optimisation

Cas de test	TC-S3-01
Description	L'ordre proposé ne place jamais une livraison avant le chargement du dépôt qui la fournit, et n'est jamais plus coûteux que l'ordre courant.
Précondition	Une tournée en brouillon comportant deux dépôts de chargement et plusieurs livraisons rattachées à chacun.
Scénario	1. Demander une optimisation. 2. Comparer l'ordre proposé et son coût à l'ordre initial.
Résultat attendu	Chaque livraison suit le chargement dont elle dépend ; le coût annoncé est inférieur ou égal à celui de l'ordre initial.

TAB. 6.6 : TC-S3-03 — Mesure des délais et cas du multi-dépôt

Cas de test	TC-S3-03
Description	Le verdict de délai suit la fenêtre engagée, et aucun retard n'est imputé à un chauffeur qui n'a pas encore pu charger.
Précondition	Une tournée validée à deux dépôts ; une livraison rattachée au second dépôt, dont la fenêtre est déjà ouverte.
Scénario	1. Observer le verdict avant l'heure de la fenêtre. 2. Observer le verdict pendant que le second dépôt n'est pas encore atteint. 3. Observer le verdict une fois la fenêtre dépassée, le chargement fait.
Résultat attendu	Étape 1 : dans les temps. Étape 2 : dans les temps, le décompte n'a pas démarré. Étape 3 : dépassé, avec le retard exprimé en minutes.

TAB. 6.7 : TC-S3-04 — Réaffectation d'une livraison déjà chargée

Cas de test	TC-S3-04
Description	Une livraison déjà dans le véhicule ne change pas de chauffeur sans transfert de garde confirmé sur le terrain.
Précondition	Une livraison en transit chez un chauffeur ; un second chauffeur en service.
Scénario	1. Réaffecter la livraison au second chauffeur. 2. Observer l'état avant confirmation de la remise. 3. Confirmer la remise entre les deux chauffeurs.
Résultat attendu	La livraison attend le transfert et n'est portée par personne d'autre tant que la remise n'est pas confirmée ; elle repart ensuite au nom du second chauffeur.

6.7.2 Tests automatisés

Trois mécanismes de ce sprint décident à la place d'un humain : l'ordre de passage, le verdict de délai et la légalité d'une tournée. Ce sont eux que les tests visent en priorité, parce qu'une erreur y passe inaperçue. Elle ne provoque aucune panne, elle produit une tournée un peu plus longue, un retard imputé au mauvais chauffeur, ou un camion chargé au-delà de ce qu'il peut porter.

Le sprint compte **150 méthodes de test**, réparties en deux familles : **141 tests unitaires** sur 13 classes, et **9 tests d'acceptation** sur une classe, qui s'exécutent contre une vraie base PostgreSQL. Le tableau 6.8 sépare les deux.

TAB. 6.8 : Tests automatisés du sprint 3

Classe de test	Méthodes	Ce qu'elle garantit
Tests unitaires — 13 classes, 141 méthodes		
SlaEvaluatorTest	24	Aucun retard n'est imputé à qui ne pouvait pas agir, et un échec reçoit un verdict plutôt qu'un blanc
RouteValidatorTest	23	Une tournée irréalisable est refusée avant d'engager un chauffeur
RmaServiceTest	21	On ne retourne pas plus qu'on n'a reçu, et le stock n'est réalimenté qu'après décision
PickupStopReconcilerTest	12	Une tournée porte exactement les chargements dont ses livraisons ont besoin
ExceptionRetirementTest	10	Une exception réglée disparaît de la liste, une exception en cours y reste
ExceptionResolutionReassignTest	9	Un colis déjà en main ne change de porteur que par transfert de garde
PrecedenceAwareSequencerTest	8	L'ordre respecte la précédence et n'est jamais pire que celui reçu
RouteExecutionServiceTest	8	Un arrêt ne recule pas d'état, et une tournée ne se clôture pas avec des arrêts actifs
PublicRmaServiceTest	8	Le destinataire ne peut ouvrir qu'un retour à la fois, sur ce qu'il a réellement reçu
PublicRmaControllerTest	5	Les points d'entrée publics refusent une demande hors quota
PublicReturnRateLimiterTest	5	La limite de débit, comptée par adresse et par livraison

AssignDriverValidationTest	4	Un chauffeur inexistant est refusé avant toute écriture
DeliveryDepotsTest	4	Une livraison est chargée à chaque dépôt que ses lignes nomment, et n’y prend que ces lignes
Tests d’acceptation (base PostgreSQL réelle) — 1 classe, 9 méthodes		
DeliveryLifecycleAcceptanceIT	9	Le cycle complet d’une livraison, de l’import à la clôture
Total	150	14 classes

Deux classes méritent un mot.

PrecedenceAwareSequencerTest ne vérifie pas qu’un ordre précis est produit, ce qui figerait une heuristique, mais des propriétés qui doivent tenir quel que soit le résultat : aucune livraison avant son chargement, et un coût jamais supérieur à celui de l’ordre reçu.

RouteValidatorTest oppose volontairement deux cas, et c’est là que se joue la justesse du contrôle de charge : six cents kilogrammes dans un véhicule de quatre cents. Accepté si le premier colis est livré avant que le second ne soit chargé au dépôt distant, refusé s’ils voyagent ensemble. Une simple somme des poids aurait rejeté les deux, et interdit une tournée réalisable.

6.8 Conclusion

Ce sprint a transformé un stock de commandes importées en journées de travail organisées et suivies. Le dispatcheur construit ses tournées dans un planificateur qui contrôle la charge, les horaires et la cohérence des fenêtres, et qui lui propose un ordre de passage sans jamais le lui imposer. La répartition entre le moteur d’itinéraires, qui chiffre les trajets, et le séquenceur, qui décide de l’ordre sous contrainte de précédence, permet d’obtenir des tournées réalisables au lieu de tournées seulement courtes.

L’exécution est encadrée par une machine à états dont les transitions servent trois usages à la fois : elles alimentent le suivi en direct, fournissent les horodatages sur lesquels les délais sont mesurés, et constituent le rapport de fin de journée. Le moteur d’échéances rend un verdict par phase, en refusant d’imputer à un chauffeur un retard qu’il ne pouvait pas éviter. Enfin, le destinataire entre dans le dispositif par un lien de suivi qui ne demande aucun compte, se ferme quatorze jours après la livraison, et depuis lequel il peut demander un retour.

Le bureau dispose désormais de tout ce qu’il faut pour organiser et surveiller la journée. Reste l’autre extrémité de la chaîne : le chauffeur, son téléphone, et les zones où le réseau ne passe pas. C’est l’objet du sprint suivant.

Sprint 4 — Application mobile chauffeur et mode hors ligne

7.1 Introduction

Tout ce que le bureau observe vient d'un geste fait ailleurs : un chauffeur qui confirme un enlèvement, photographie un bon signé, encaisse un montant. Il les fait sur une autoroute entre deux villes ou devant une porte que le réseau dessert mal. Le bureau aussi peut perdre sa connexion. La différence est qu'il peut attendre qu'elle revienne, puisque rien ne se produit pendant ce temps ; le chauffeur, lui, a un client en face de lui et un colis qui change de main à cet instant.

Ce sprint construit l'application du chauffeur, et il se lit en trois volets. Le premier est l'**exécution** : le chauffeur consulte sa tournée du jour et le calendrier de celles qui viennent, la démarre, confirme son chargement au dépôt, part en transit, puis clôture chaque arrêt. Cette clôture n'a pas une forme unique : il peut avoir tout remis, n'avoir remis qu'une partie de la commande, avoir échoué en indiquant un motif, ou au contraire avoir récupéré un colis que le destinataire renvoie. Le deuxième volet est la **preuve**, car ce qui ferme un arrêt engage l'entreprise : le chauffeur photographie le bon de livraison signé et le colis remis, détaille les manques ligne par ligne quand la livraison est partielle, enregistre le montant encaissé, puis déclare sa caisse au dépôt, où quelqu'un d'autre la compte. Le troisième volet est le **hors-ligne**, qui conditionne les deux autres, car sans lui tout ce qui précède s'arrête à la première zone sans couverture.

7.2 Objectifs du sprint et planification des tâches

Le périmètre couvre neuf *user stories*, US-20 à US-27 et US-34, soit **87 points** d'effort. Le tableau 7.1 détaille les tâches du sprint, regroupées par *user story*.

TAB. 7.1 : Sprint backlog — sprint 4

ID	User Story	Tâche	Description	Priorité
US-20	Consultation de la tournée du jour (8 pts)	T4-01	Afficher la tournée du jour et ses arrêts dans l'ordre de passage	Haute

ID	User Story	Tâche	Description	Priorité
US-21	Déclaration d'une livraison réussie ou échouée (8 pts)	T4-02	Consulter le calendrier des tournées planifiées	Moyenne
		T4-03	Exécuter la tournée : démarrage, enlèvement au dépôt, transit, arrivée	Haute
		T4-04	Clôturer un arrêt en livraison complète ou partielle	Haute
		T4-05	Déclarer un échec avec un motif issu du référentiel	Haute
		T4-06	Détailler les manques par unité : manquant, refusé, abîmé	Moyenne
		T4-07	Photographier le bon de livraison signé et le colis remis	Haute
US-22	Preuve de livraison avec photographies (13 pts)	T4-08	Déposer les photographies dans le stockage objet avant d'écrire la preuve	Haute
		T4-09	Consulter le bon de livraison même hors ligne	Moyenne
		T4-10	Mettre les écritures en file sur le téléphone et les rejouer au retour du réseau	Haute
		T4-11	Rendre l'écriture idempotente par une clé stable par livraison	Haute
		T4-12	Horodater l'action au moment du geste, non au moment du jeu	Haute
		T4-13	Projeter localement l'effet d'une écriture en attente	Haute
US-23	Travail hors réseau et transmission différée (21 pts)	T4-14	Mettre en cache la tournée, les livraisons et les bons de livraison	Haute
		T4-15	Centre de synchronisation : relancer ou abandonner une écriture en échec	Moyenne
		T4-16	Générer un code à usage unique et l'afficher sous forme de QR	Moyenne
		T4-17	Confirmer la réception par scan, avec la position des deux chauffeurs	Moyenne
		T4-18	Placer la livraison en attente de transfert jusqu'à confirmation de la remise	Haute
		T4-19	Saisir le montant reçu et son moyen : espèces, chèque, ou rien	Haute
US-24	Transfert de garde d'un colis (13 pts)	T4-16	Générer un code à usage unique et l'afficher sous forme de QR	Moyenne
US-25	Enregistrement du montant encaissé (8 pts)	T4-17	Confirmer la réception par scan, avec la position des deux chauffeurs	Moyenne
		T4-18	Placer la livraison en attente de transfert jusqu'à confirmation de la remise	Haute
US-25	Enregistrement du montant encaissé (8 pts)	T4-19	Saisir le montant reçu et son moyen : espèces, chèque, ou rien	Haute

ID	User Story	Tâche	Description	Priorité
		T4-20	Exiger un motif lorsque le montant reçu est inférieur à l'attendu	Moyenne
US-26	Déclaration de la remise de caisse (3 pts)	T4-21	Consulter l'encaisse détenue et déclarer le montant remis au dépôt	Haute
US-27	Comptage de la caisse et arbitrage de l'écart (8 pts)	T4-22	Compter la remise au bureau, constater l'écart et le solder	Haute
US-34	Collecte d'un retour chez le destinataire (5 pts)	T4-23	Collecter un retour comme un arrêt de tournée ordinaire, sans bon à photographier	Moyenne

7.3 Concepts et mécanismes utilisés

Ce sprint mobilise quatre notions. Les trois dernières se répondent, car elles décrivent le trajet d'une écriture depuis le geste du chauffeur jusqu'à son enregistrement, et chacune règle un problème que la précédente laisse ouvert.

L'architecture hors-ligne d'abord : une application de terrain ne peut pas traiter le réseau comme une condition de fonctionnement, elle inverse donc la dépendance habituelle. Ce que le chauffeur voit vient d'une copie locale, que le réseau rafraîchit quand il le peut ; ce qu'il fait devient un fait enregistré sur le téléphone, que le réseau transmet ensuite. La couverture ne décide plus de ce qu'il a le droit de faire. L'idée n'est pas neuve : elle a été formulée dès 1992 sous le nom de *fonctionnement déconnecté*, en observant qu'un cache tenu pour un gain de performance pouvait aussi servir à rester disponible quand la ressource distante ne répond plus [KIS 92].

Cette copie est préparée au dépôt, avant le départ : la tournée du jour, le détail de ses livraisons, et les bons de livraison au format PDF. Ce dernier point compte, car le bon est produit par le progiciel de l'entreprise cliente, donc récupéré en direct, et c'est le document que le destinataire signe. La connectivité se mesure enfin à ce que le serveur répond et non à ce que le téléphone affiche, de sorte qu'un réseau qui ne mène nulle part est traité comme une absence de réseau.

La file de rejeu : une écriture que le réseau n'a pas pu transmettre reste sur le téléphone jusqu'à ce que le serveur l'accepte. La file est durable, chiffrée sur le disque, et survit au redémarrage de l'appareil. Ce qui la distingue d'un simple tampon est le traitement de l'échec : un refus définitif du serveur ne repart pas, une erreur passagère donne droit à **trois tentatives**, et une écriture jamais partie expire au bout de **vingt-quatre heures**. Dans les trois cas elle

n'est pas supprimée mais marquée, et reste visible dans un centre de synchronisation où le chauffeur la relance ou l'abandonne. Une preuve de livraison effacée sans que personne ne l'apprenne serait un litige perdu d'avance.

L'idempotence : une écriture reçue plusieurs fois ne doit agir qu'une seule. Côté téléphone, chaque action reçoit une **clé stable** construite sur la livraison qu'elle concerne :

pod-3f2a9c14... pour l'enregistrement d'une preuve, fail-3f2a9c14... pour un échec sur cette même livraison. Réenfiler la même action remplace donc l'écriture en attente au lieu d'en ajouter une seconde, et un chauffeur qui change de motif avant de retrouver le réseau règle son hésitation là où il la voit encore. Côté serveur, un aspect mémorise la réponse rendue et la restitue si la même écriture se représente, sans rien réexécuter.

La clé seule ne désigne pourtant pas une écriture. Le serveur l'associe donc à une **portée** réunissant la méthode, l'adresse appelée et l'identité de l'appelant, que la passerelle extrait du jeton et que le client ne peut pas forger. C'est elle qui empêche un rejeu de profiter au mauvais acteur, un chauffeur ne récupérant jamais la réponse donnée à son collègue sur une livraison qu'on lui a retirée. Cette portée est enfin ramenée à une **empreinte** de taille fixe, ici bd22134f..., qui sert de clé primaire, ce qui retire au client la maîtrise de celle-ci.

L'horloge de l'action : lorsqu'une écriture attend le réseau, l'instant où le chauffeur a agi et celui où le serveur l'apprend s'écartent. Le système retient le premier, dont l'heure est prise sur le téléphone avant toute tentative de connexion, puis voyage avec l'écriture, sur le chemin direct comme sur le rejeu. Ce détail décide de la justesse de tout ce qui se mesure ensuite. Un colis remis à quatorze heures dans une rue sans réseau, dont l'écriture ne part qu'à dix-sept heures cinquante, serait sinon enregistré comme livré à dix-sept heures cinquante ; le moteur d'échéances du chapitre précédent imputerait alors au chauffeur près de quatre heures de retard qu'il n'a pas eues.

7.4 Analyse et spécification des besoins

7.4.1 Diagramme de cas d'utilisation du sprint

La figure 7.1 présente les cas d'utilisation du sprint : l'exécution de la tournée, l'enregistrement de la preuve, le travail hors réseau, le transfert de garde et le circuit de la caisse.

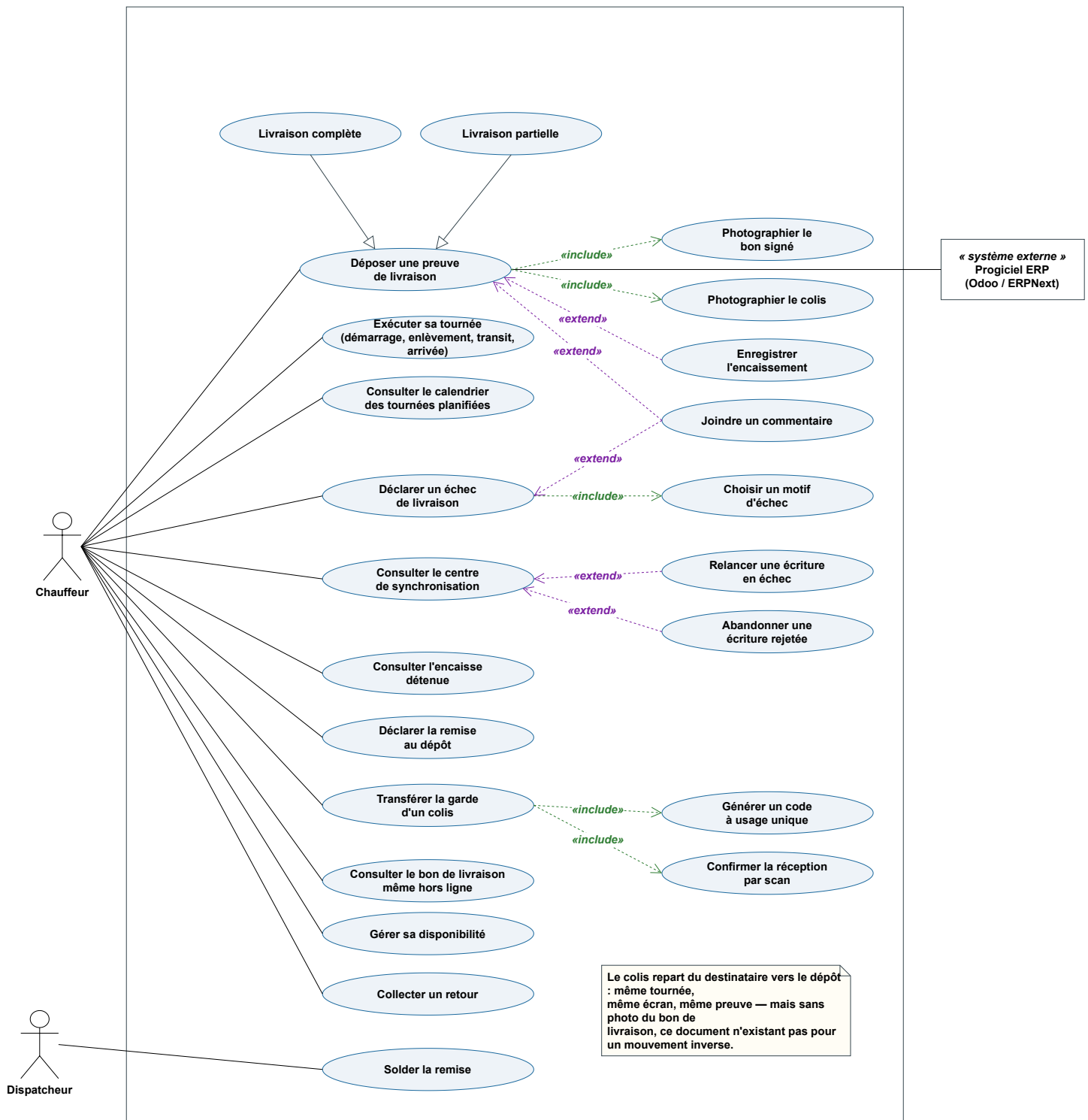


FIG. 7.1 : Diagramme de cas d'utilisation du sprint 4

Le déséquilibre entre les deux acteurs est le premier fait à relever. Le **chauffeur** porte presque tous les cas ; le **dispatcheur** n'en détient qu'un seul, solder la remise de caisse. Cette solitude n'est pas un oubli de découpage, c'est le mécanisme lui-même. Celui qui remet l'argent n'est pas celui qui le compte. Le progiciel apparaît en système externe, sollicité par le seul enregistrement de la preuve, dont il reçoit le mouvement de stock.

Les relations méritent une lecture attentive. La livraison complète et la livraison partielle sont deux *spécialisations* du même cas. Une livraison partielle n'est pas un parcours à part, c'est le même geste avec un décompte ligne par ligne. Les deux photographies sont *incluses* dans l'enregistrement de la preuve, donc obligatoires. Une collecte de retour fait seule exception, car elle n'a pas de bon de livraison à photographier, ce document n'existant pas pour un mouvement inverse. L'encaissement et le commentaire *étendent* l'enregistrement de la preuve, le premier n'ayant lieu que si la commande est à encaisser, le second restant facultatif. Enfin, la relance et l'abandon d'une écriture étendent le centre de synchronisation, puisqu'ils ne se présentent que si une écriture a échoué.

7.4.2 Description détaillée des cas d'utilisation

Trois cas sont détaillés : celui qui clôt une livraison, celui qui la rend possible sans réseau, et celui qui règle la garde d'un colis sur le terrain.

TAB. 7.2 : Cas d'utilisation « Déposer une preuve de livraison »

Titre	Déposer une preuve de livraison
Acteur	Chauffeur
Résumé	Clore un arrêt en enregistrant ce qui a réellement été remis, les photographies qui l'attestent et, le cas échéant, le montant encaissé.
Préconditions	Le chauffeur est authentifié et la livraison lui est affectée. Elle est chargée ou en transit. Le réseau n'est pas requis.
Postconditions	La preuve est enregistrée, horodatée à l'heure du geste, avec ses photographies dans le stockage objet. La livraison est livrée ou partiellement livrée. Le mouvement de stock part vers le progiciel.
Scénario nominal	1. Le chauffeur ouvre l'écran de preuve depuis l'arrêt en cours. 2. Il consulte le bon de livraison, servi depuis la copie locale si le réseau manque. 3. Il photographie le bon signé par le destinataire, puis le colis remis. 4. Si la commande est à encaisser, il saisit le montant reçu et son moyen de règlement. 5. Il valide. L'écran affiche la livraison comme livrée sans attendre le serveur.

Déposer une preuve de livraison (suite)

Scénarios alternatifs	3a. La livraison est partielle : le chauffeur bascule l'interrupteur prévu et répartit chaque ligne entre ce qui est remis et ce qui manque, en indiquant pour chaque manque sa quantité et son motif. La photographie du bon déjà prise est alors écartée, car elle montre les quantités imprimées et non celles remises. 3b. Il s'agit d'une collecte de retour : aucun bon n'est demandé, seule la photographie du colis récupéré est exigée, et le décompte partiel n'est pas proposé. 4a. Le montant reçu est inférieur à celui attendu : un motif devient obligatoire, dès lors que le référentiel en propose un. 5a. Le réseau est absent : l'écriture est mise en attente et la suite se déroule comme au cas « Travailler hors réseau ». 5b. Une preuve existe déjà pour cette livraison : le serveur renvoie l'état courant sans rien réécrire. Le cas survient quand la requête a bien été traitée mais que sa réponse s'est perdue au retour : l'application la croit échouée et la rejoue. Sans ce contrôle, le progiciel recevrait un second mouvement de stock.
------------------------------	---

TAB. 7.3 : Cas d'utilisation « Travailler hors réseau »

Titre	Travailler hors réseau et rejouer ses écritures
Acteur	Chauffeur
Résumé	Poursuivre la tournée sans couverture réseau, puis voir les actions accumulées transmises au serveur dès qu'elle revient.
Préconditions	La tournée et ses livraisons ont été chargées pendant que le réseau était disponible, en principe au dépôt.
Postconditions	Chaque action est parvenue au serveur exactement une fois, datée de l'instant où le chauffeur l'a faite. Toute écriture non transmise reste visible et actionnable.
Scénario nominal	1. Le chauffeur agit sur une livraison alors que le réseau est absent. 2. L'application enregistre l'écriture sur le téléphone avec sa clé et l'heure du geste. 3. L'écran avance immédiatement, comme si le serveur avait répondu. 4. Le réseau revient. La file part, écriture par écriture, dans l'ordre. 5. Le serveur accepte ; l'écriture quitte la file.

Travailler hors réseau et rejouer ses écritures (suite)

Scénarios alternatifs	1a. Rien n’a été chargé au préalable et la livraison n’est pas en cache : l’action est mise en file, mais l’écran ne peut rien afficher de nouveau. 2a. Le chauffeur revient sur son action, par exemple pour changer un motif d’échec : la clé étant identique, l’écriture en attente est remplacée au lieu d’être doublée. 4a. Une écriture échoue sur une erreur passagère : elle est retentée trois fois, puis signalée. 4b. Le serveur la refuse définitivement : elle est signalée sans nouvelle tentative. 4c. Elle attend depuis plus de vingt-quatre heures : elle expire et est signalée. 5a. Dans les trois cas précédents, l’écriture apparaît dans le centre de synchronisation, où le chauffeur la relance ou l’abandonne. Elle n’est jamais supprimée sans son accord.
------------------------------	---

TAB. 7.4 : Cas d’utilisation « Transférer la garde d’un colis »

Titre	Transférer la garde d’un colis
Acteur	Chauffeur cédant, chauffeur receveur
Résumé	Remettre à un collègue un colis déjà chargé, de sorte que le système enregistre le changement de main au moment où il a physiquement lieu.
Préconditions	Le bureau a réaffecté la livraison alors qu’elle était déjà en main. Elle est en attente de transfert. Les deux chauffeurs sont en service.
Postconditions	La livraison repart au nom du receveur à l’état <i>chargée</i> , quel que soit son état avant le transfert : le colis est en main, mais le trajet vers le client reste à engager par celui qui le porte désormais. La remise est datée et localisée, et les deux chauffeurs y sont nommés.
Scénario nominal	1. Le chauffeur cédant ouvre la livraison et demande un code de remise. 2. Le serveur émet un code à usage unique, valable deux heures, affiché sous forme de QR avec son compte à rebours. 3. Les deux chauffeurs se rejoignent et le colis change de main. 4. Le receveur scanne le code. L’application lui demande de le maintenir cadré quelques secondes avant de valider. 5. Le serveur confirme la remise et la livraison repart au nom du receveur.

*Transférer la garde d'un colis (suite)***Scénarios alternatifs**

- 2a. Le code expire avant la rencontre : il est verrouillé à l'écran et le cédant en demande un nouveau.
- 4a. Le code est déjà utilisé, ou ne correspond pas à ce receveur : la confirmation est refusée avec un message explicite.
- 4b. Cinq tentatives infructueuses ont été faites : le code est bloqué.
- 5a. La remise n'est pas confirmée dans le délai que l'entreprise s'est fixé, un quart d'heure par défaut : le bureau et les deux chauffeurs reçoivent une alerte, envoyée une seule fois.
- 5b. Elle ne l'est toujours pas au terme du délai d'annulation, deux heures par défaut : le transfert est annulé, le colis revient au chauffeur qui l'a toujours en main, son arrêt est replacé sur la tournée de celui-ci, et le bureau est averti. Ce délai vaut exactement la durée de validité du code : un QR ne peut donc jamais afficher un compte à rebours vivant sur un transfert déjà mort. Une entreprise qui préfère garder la main peut désactiver cette annulation automatique.

7.5 Modélisation conceptuelle

7.5.1 Diagramme de classes

La figure 7.2 présente la conception de l'enregistrement d'une preuve, de bout en bout : le téléphone, le service qui le reçoit, et les entités écrites en base.

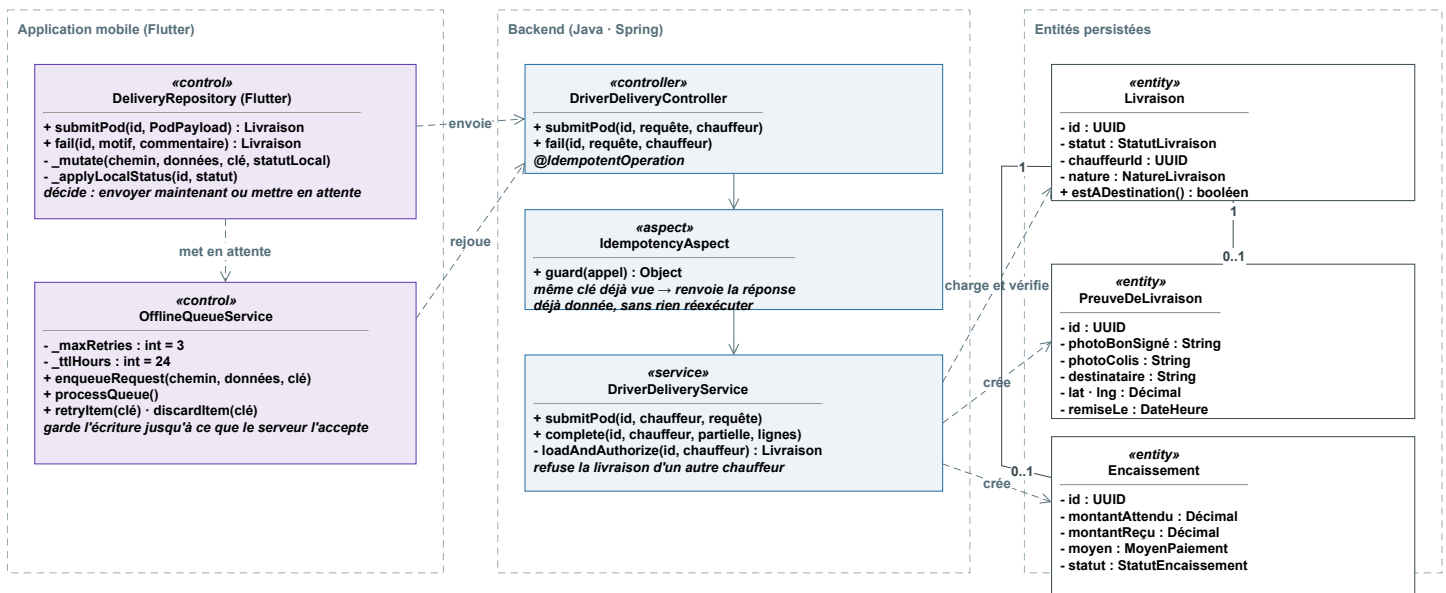


FIG. 7.2 : Classes de conception du sprint 4

Le diagramme se lit en trois bandes, qui sont trois responsabilités et non trois couches techniques. Sur le téléphone, DeliveryRepository **décide** et OfflineQueueService **garantit**.

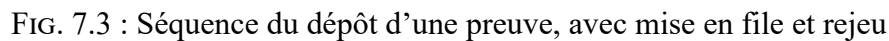
Le premier tranche une question immédiate, celle de savoir si le réseau répond, et si oui l'écriture part tout de suite, sinon il la confie au second. Celui-ci porte alors une promesse longue, conserver cette écriture jusqu'à ce que le serveur l'accepte, avec ses trois tentatives et son expiration à vingt-quatre heures. Les séparer suit la durée de vie de ces deux tâches, le choix se faisant en un instant quand la promesse peut tenir toute la journée.

Côté serveur, `DriverDeliveryController` porte l'annotation qui déclenche la garde d'idempotence. `IdempotencyAspect` s'intercale avant le service et rend la réponse déjà donnée si la clé a déjà été vue. `DriverDeliveryService` n'est donc atteint que par une écriture réellement nouvelle, et sa première action est de charger la livraison en vérifiant qu'elle appartient bien au chauffeur qui la présente.

Trois entités closent la chaîne. Une `Livraison` porte son état, son chauffeur et sa nature, cette dernière distinguant une livraison ordinaire d'une collecte de retour. Elle est associée à au plus une `PreuveDeLivraison`, qui garde les chemins des deux photographies, le nom du destinataire, la position et l'heure de la remise, et à au plus un `Encaissement`, qui compare le montant attendu au montant reçu. Les deux cardinalités valent 0..1 pour la même raison : une livraison en échec, chez un client absent, n'a ni preuve ni encaissement.

7.5.2 Diagramme de séquence

La figure 7.3 suit l'enregistrement d'une preuve depuis l'écran du chauffeur jusqu'à l'enregistrement, en passant par le cas où le réseau est absent.



La dernière branche est la plus importante pour le chauffeur. En cas de refus définitif ou de tentatives épuisées, l'écriture n'est pas jetée, mais signalée dans le centre de synchronisation, où il la relance ou l'abandonne lui-même.

7.6 Réalisation

La figure 7.4 montre la tournée du jour telle que le chauffeur la voit. L'en-tête porte son nom et sa disponibilité, ici «en service». La carte du haut résume la tournée : son numéro, son état, le créneau de la journée, la ville, le nombre d'arrêts, le véhicule attribué avec son immatriculation, le dépôt de départ, et une barre d'avancement qui compte les arrêts clos. En dessous vient la liste des arrêts dans l'ordre de passage. Chaque arrêt donne le nom du client, son adresse, la référence de la commande, le montant à encaisser et le créneau promis ; l'arrêt en cours porte l'action qui lui reste, ici confirmer la livraison. La barre du bas ne propose que trois destinations — la tournée, le calendrier, le profil — parce qu'un écran de téléphone tenu d'une main dans un camion ne supporte pas davantage.



FIG. 7.4 : Tournée du jour sur l'application chauffeur

La figure 7.5 présente l'écran de preuve dans ses deux moments. À gauche, à l'ouverture : un rappel en trois étapes dit ce qui est attendu, le bon de livraison reste consultable et imprimable depuis l'écran, et l'emplacement de la photographie du bon signé est marqué obligatoire. Le bouton de validation est là mais grisé, et le motif de son indisponibilité est écrit juste au-dessus, en rouge, plutôt que laissé à deviner : les deux photographies manquent. À droite, le décompte ligne par ligne, qui n'apparaît que si la livraison est partielle. L'article est nommé avec sa référence, la quantité effectivement remise est comparée à celle attendue, et le reste doit être ventilé : manquant, refusé, abîmé. Chaque catégorie ouvre la liste des motifs du référentiel, si

bien que le chauffeur choisit une raison au lieu d'en écrire une.

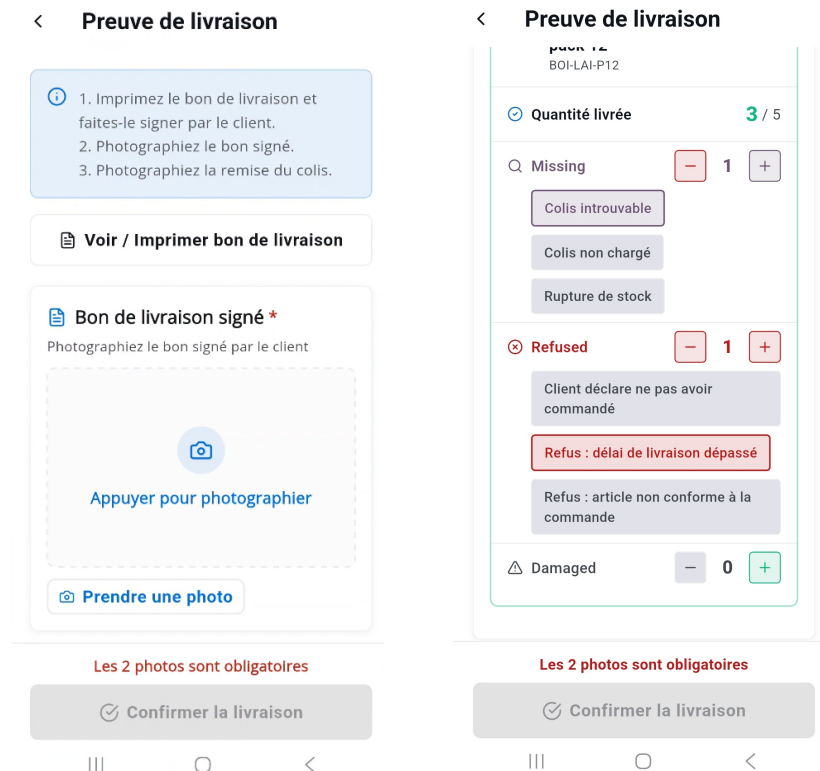


FIG. 7.5 : Preuve de livraison : le formulaire, puis le décompte des manques

La figure 7.6 montre le centre de synchronisation dans ses deux états. À gauche, hors réseau, une preuve de livraison attend son tour ; le bandeau ambré annonce le nombre d'écritures en attente sans que le chauffeur ait à ouvrir quoi que ce soit. À droite, la même écriture après un rejet définitif du serveur : le bandeau passe au rouge et compte les actions à revoir, la ligne porte la mention «refusé par le serveur», et trois issues sont offertes — relancer, ouvrir la livraison concernée, ou abandonner l'écriture. Dans les deux états, la ligne nomme l'action et la livraison à laquelle elle appartient, et rappelle l'heure à laquelle le chauffeur l'a faite. C'est le seul endroit où il reprend la main sur ce que le réseau n'a pas transmis.

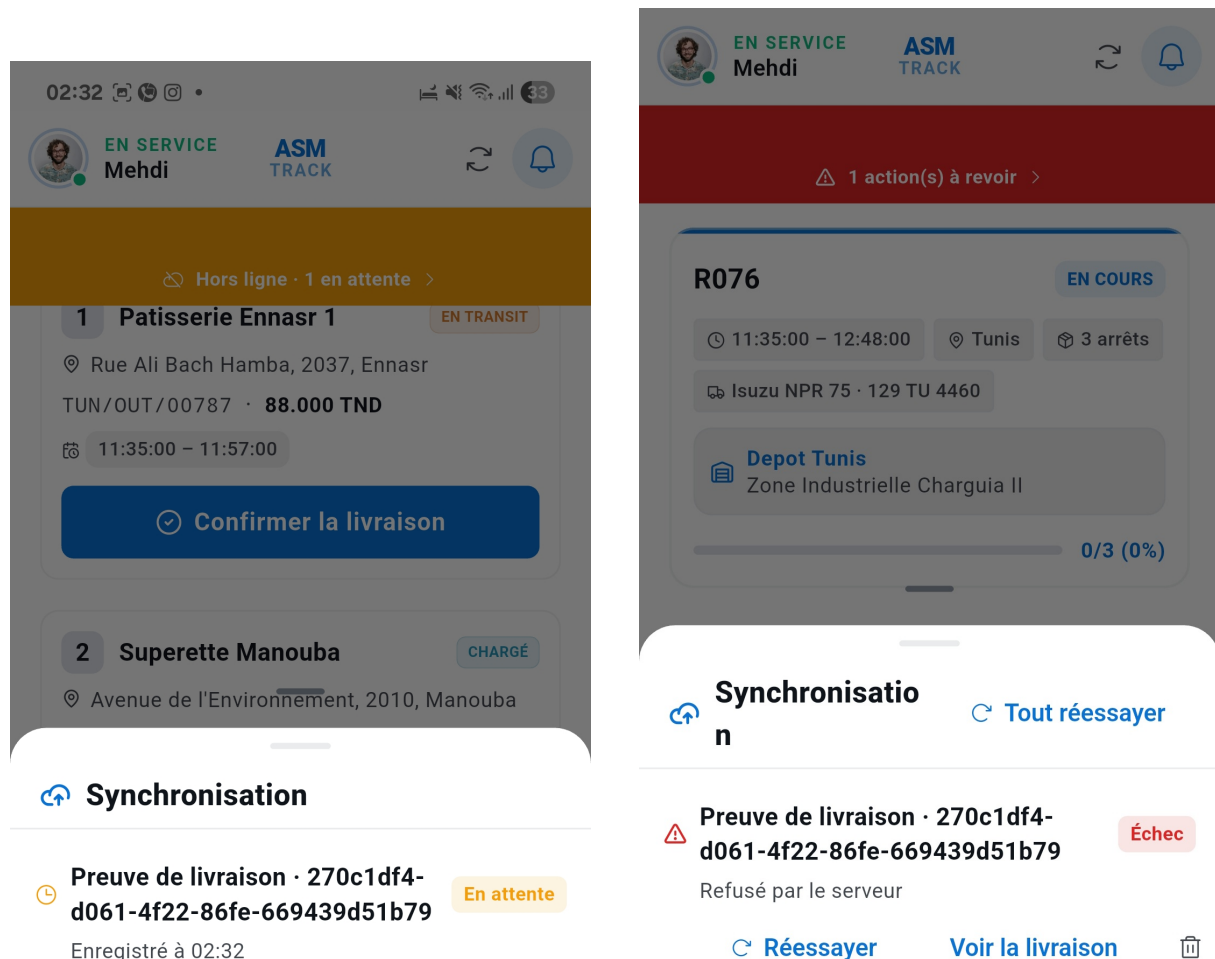


FIG. 7.6 : Centre de synchronisation : une écriture en attente, puis la même refusée par le serveur

La figure 7.7 montre le code affiché par le chauffeur qui cède le colis, ouvert au-dessus de son écran de transferts. Une phrase dit quoi faire : demander à l'autre chauffeur de scanner le code pour confirmer le transfert. Le code apparaît sous deux formes, le QR à scanner et les six caractères qu'il porte, pour le cas où l'appareil d'en face ne parvient pas à lire l'image. Un compte à rebours indique le temps qu'il reste avant expiration, car le code ne vaut que pour cette remise-là et pour un temps borné. La capture est prise en thème sombre, que l'application propose comme le reste du produit.

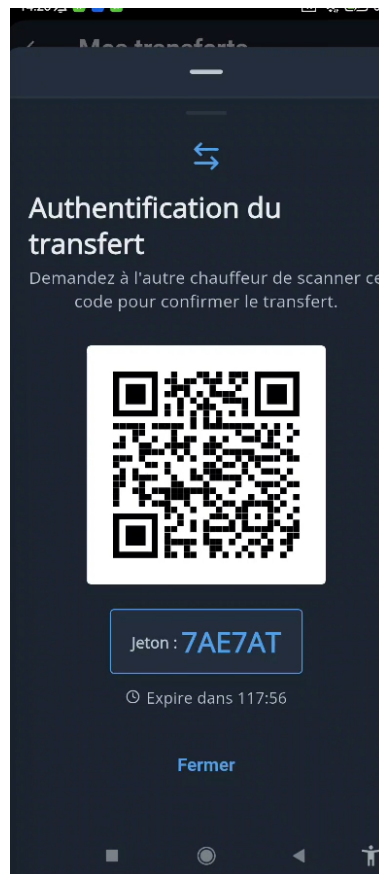


FIG. 7.7 : Code de transfert de garde : QR, jeton et compte à rebours

La figure 7.8 montre la remise de caisse. L'écran s'ouvre sur ce que la plateforme attribue au chauffeur — «encaissé par vous», total calculé à partir des encaissements enregistrés pendant la tournée — et non sur un champ vide. Vient ensuite le seul champ à remplir, le montant qu'il déclare rendre, prérempli avec ce total puisque c'est le cas courant, et modifiable puisque l'écart est justement ce qu'il faut pouvoir déclarer. Une note sous le champ dit la suite : le dépôt comptera l'argent devant lui, et la remise n'est pas terminée avant. L'écran ne confirme donc jamais une remise : il annonce une attente de comptage.

← **Remise de caisse**

Encaissé par vous
3300,000 TND

Montant que vous remettez
3300,000 TND

ⓘ Le dépôt comptera le montant devant vous. La remise n'est pas terminée avant.

Déclarer le montant

FIG. 7.8 : Déclaration de la remise de caisse par le chauffeur

La figure 7.9 montre enfin le comptage côté bureau. En haut, le total qui circule — l'argent encaissé par les livreurs et pas encore remis au dépôt — puis sa décomposition par chauffeur. Les remises sont classées par état : à traiter, en écart, clôturées. Le tableau met côte à côte les trois montants qui comptent, l'attendu calculé par la plateforme, le déclaré saisi par le chauffeur et le compté saisi au guichet, et n'affiche l'écart que lorsqu'il existe. Une ligne dépliée montre l'arbitrage : l'explication écrite par celui qui a compté, et les commandes concernées avec leur montant. C'est là que se règle le cas de la capture — cinq dinars de frais de parking payés en espèces pendant la tournée, sans justificatif, retenus sur la remise suivante.

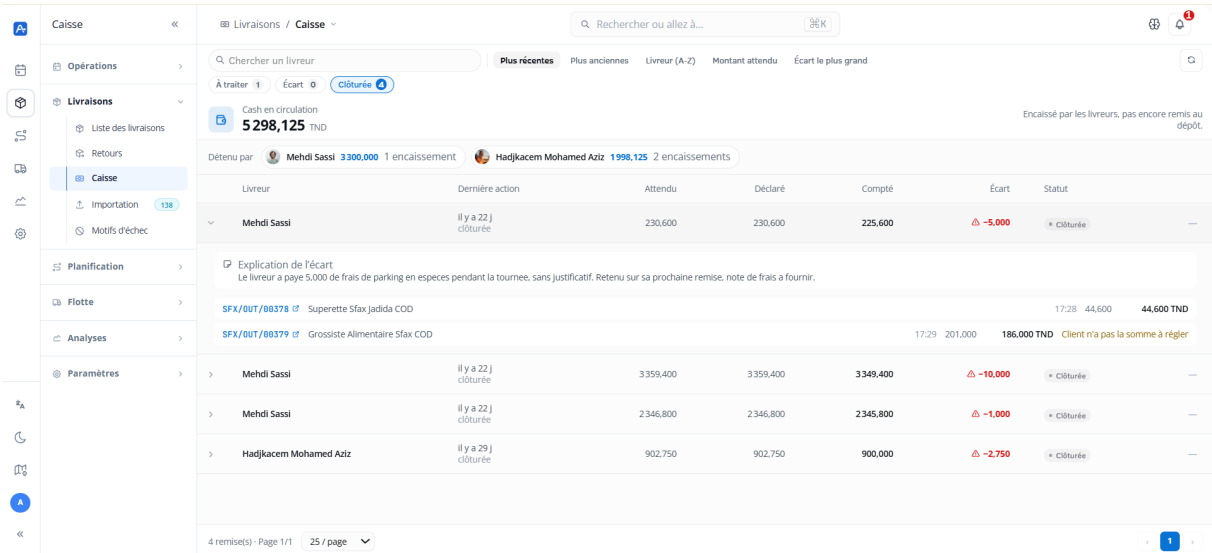


FIG. 7.9 : Comptage de la remise et arbitrage de l'écart au bureau

7.7 Tests et validation

7.7.1 Scénarios de test

Trois scénarios couvrent les apports du sprint : la continuité du travail sans réseau, l'unicité de l'écriture et le circuit de la caisse. Le sort d'une écriture refusée et la garde d'un colis sont décrits par les cas d'utilisation détaillés plus haut.

TAB. 7.5 : TC-S4-01 — Dépôt de preuve sans réseau, puis jeu

Cas de test	TC-S4-01
Description	Une preuve déposée sans couverture réseau est enregistrée au retour de celle-ci, à l'heure du geste et non à l'heure du jeu.
Précondition	Une tournée chargée au dépôt, une livraison en transit, le mode avion activé.
Scénario	1. Déposer la preuve, photographies comprises, hors réseau. 2. Observer l'écran immédiatement après la validation. 3. Poursuivre la tournée, puis rétablir le réseau. 4. Consulter la livraison depuis le back-office.
Résultat attendu	Étape 2 : la livraison apparaît comme livrée et le chauffeur peut passer à l'arrêt suivant. Étape 4 : la preuve existe côté serveur, ses deux photographies sont accessibles, et l'heure enregistrée est celle de l'étape 1, non celle de l'étape 3.

TAB. 7.6 : TC-S4-02 — Une même preuve envoyée deux fois

Cas de test	TC-S4-02
Description	Une preuve reçue plusieurs fois ne produit ni doublon, ni second mouvement de stock, ni second encaissement.
Précondition	Une livraison en transit, à encaisser.
Scénario	1. Déposer la preuve avec un montant encaissé. 2. Rejouer la même requête avec la même clé. 3. Compter les preuves, les encaissements et les mouvements envoyés au progiciel.
Résultat attendu	La seconde requête réussit et renvoie la même réponse que la première. Une seule preuve, un seul encaissement et un seul mouvement de stock existent.

TAB. 7.7 : TC-S4-05 — Encaissement, remise et arbitrage de l'écart

Cas de test	TC-S4-05
Description	Le montant encaissé à la porte remonte jusqu'à la caisse du dépôt, et un écart entre le déclaré et le compté est constaté puis soldé par une autre personne que le chauffeur.
Précondition	Deux livraisons à encaisser affectées au même chauffeur.
Scénario	1. Encaisser le montant attendu sur la première livraison. 2. Encaisser un montant inférieur sur la seconde, sans indiquer de motif, puis avec un motif. 3. Consulter l'encaisse détenue, puis déclarer un montant différent de celle-ci. 4. Compter la remise au bureau et solder l'écart.
Résultat attendu	Étape 2 : la validation est refusée sans motif et acceptée avec. Étape 3 : l'encaisse affichée correspond à la somme des deux encaissements, et la déclaration ne vaut pas remise. Étape 4 : l'écart entre le déclaré et le compté est affiché, et le chauffeur ne peut pas le solder lui-même.

7.7.2 Tests automatisés

Deux mécanismes de ce sprint décident sans qu'un humain puisse les contrôler : la file hors ligne et le calcul de la caisse. Ce sont eux que les tests visent en priorité, parce qu'une erreur y est silencieuse. Une écriture perdue ne provoque aucune panne, mais produit une livraison qui n'a jamais eu lieu selon le serveur, et un chauffeur qui jure du contraire.

Le sprint compte **71 tests**, répartis entre le serveur et l'application mobile. Le tableau 7.8 sépare les trois familles.

TAB. 7.8 : Tests automatisés du sprint 4

Classe ou fichier de test	Cas	Ce qu'il garantit
Tests unitaires du serveur — 4 classes, 38 cas		
CashRemittanceServiceTest	15	Une remise déclarée n'est jamais soldée par celui qui la déclare
CashCollectionServiceTest	10	Le montant pris à la porte est enregistré avec la preuve, pas séparément
CashExpectedAmountTest	7	Le montant attendu est celui de la commande, jamais recalculé sur le téléphone
ErpLookupServiceCodTest	6	Le montant à encaisser vient du progiciel et suit ses conventions
Tests d'acceptation (base PostgreSQL réelle) — 1 classe, 9 cas		
DriverJourneyAcceptanceIT	9	La journée complète d'un chauffeur, de l'enlèvement à la clôture
Tests de l'application mobile — 4 fichiers, 24 cas		
offline_queue_service_test	8	Aucune écriture n'est perdue, aucune n'est doublée, aucune n'est jetée en silence
pod_widgets_test	7	L'écran de preuve dit dans les trois langues ce qu'il attend du chauffeur
route_multi_depot_test	5	Un colis n'est chargé qu'au dépôt qui le fournit
driver_copy_parity_test	4	Aucun libellé ne manque dans une des langues de l'application
Total	71	9 classes et fichiers

Deux fichiers méritent un mot.

`offline_queue_service_test` vérifie les quatre issues d'une écriture plutôt qu'un chemin heureux : le jeu qui réussit et disparaît de la file, le refus définitif qui la marque sans la supprimer, l'erreur réseau qui la conserve pour plus tard, et l'expiration qui la signale sans jamais toucher au réseau. Il vérifie aussi que réenfiler la même clé remplace l'entrée au lieu d'en ajouter une seconde.

`CashRemittanceServiceTest` est le plus fourni du sprint, et ce n'est pas un hasard, puisqu'il couvre le seul mécanisme où une erreur se traduit directement en argent manquant. Il porte notamment sur ce qu'un chauffeur ne peut pas faire, à savoir clore sa propre remise.

Les 47 tests du serveur s'exécutent dans le pipeline d'intégration continue à chaque *commit*,

les neuf cas d'acceptation contre une base PostgreSQL réelle. L'application chauffeur est versionnée dans un dépôt distinct et possède donc sa propre chaîne, qui lance ses 24 tests à chaque *commit* elle aussi, après un contrôle de format et une analyse statique.

7.8 Conclusion

Ce sprint a donné au terrain l'outil qui manquait au dispositif du chapitre précédent. Le chauffeur consulte sa tournée, la démarre, charge au dépôt, livre, et clôture chaque arrêt par une preuve qui engage l'entreprise : le bon de livraison signé, le colis remis, le décompte des manques et le montant encaissé. Le mouvement inverse, la collecte d'un retour, emprunte la même tournée, le même écran et la même preuve, sans qu'il ait fallu redévelopper quoi que ce soit.

Le mode hors ligne est l'apport structurant, et il ne se voit pas quand il fonctionne. Une écriture faite sans réseau est conservée sur le téléphone, projetée immédiatement à l'écran, rejouée au retour de la couverture avec la même clé et la même heure, puis reconnue par le serveur comme déjà traitée si elle arrive deux fois. Ce qui échoue malgré tout n'est pas perdu, car le chauffeur le retrouve dans son centre de synchronisation et décide lui-même de le relancer ou de l'abandonner. Cette dernière propriété est celle qui distingue un mode dégradé utilisable d'un mode dégradé sur lequel on ne peut pas compter.

Deux mécanismes complètent l'ensemble en réglant des questions de responsabilité plutôt que de technique. Le transfert de garde interdit qu'un colis soit attribué à quelqu'un qui ne le tient pas. La séparation entre la déclaration du chauffeur et le comptage du bureau impose qu'une autre personne que le chauffeur compte l'argent et clôture la remise.

Le système couvre désormais la chaîne complète, de la commande importée du progiciel jusqu'à la preuve rapportée du terrain. Reste à le rendre exploitable au quotidien : savoir ce qui se passe quand un composant tombe, réparer sans intervention technique, et rendre les données lisibles à ceux qui pilotent. C'est l'objet du sprint suivant.

8.1 Introduction

Les quatre sprints précédents ont équipé ceux qui font le travail : le dispatcheur qui planifie, le chauffeur qui livre. Chacun voit ce qu'il a devant lui, et rien de plus. Personne, jusqu'ici, ne voit l'activité dans son ensemble.

C'est ce que ce sprint construit, pour deux besoins qui n'ont pas le même horizon. Le responsable veut savoir ce que vaut son exploitation : taux de réussite, motifs d'échec, respect des délais, performance par chauffeur et par zone, et de quoi en sortir un document. L'administrateur veut savoir si le système tient debout, et pouvoir le remettre en route sans appeler un développeur.

Ces deux besoins ont un point commun : ils ne demandent presque aucune donnée nouvelle. Les indicateurs se calculent sur les livraisons, les tournées et les échéances que les sprints précédents ont déjà écrites. La console de santé, elle, lit l'état des composants à l'instant où on la consulte. Le travail a donc porté sur la lecture de ces données, pas sur leur collecte.

S'y ajoute un assistant interne, à qui un opérateur pose sa question plutôt que d'aller la chercher d'écran en écran. Il n'a pas de compte à lui : il emprunte les droits de celui qui l'interroge, et il ne peut rien écrire.

8.2 Objectifs du sprint et planification des tâches

8.2.1 Objectifs attendus

Ce sprint outille ceux qui répondent de l'activité. Sept résultats sont attendus :

- un **tableau de bord d'indicateurs** : taux de réussite, motifs d'échec, respect des délais, performance par chauffeur et par zone, et densité des commandes par zone.
- des **rapports exportables en PDF**, à l'échelle de l'activité ou d'un chauffeur.
- une **console Santé Système** qui expose l'état de chaque composant et le journal des

synchronisations avec le progiciel.

- des **seuils de retard réglables** par l’administrateur depuis l’écran de paramétrage, et appliqués sans redémarrage.
- une **reprise sans développeur** : relancer une synchronisation restée en échec, rejouer les messages écartés vers une file de rebut.
- un **journal d’audit** qui dit qui a fait quoi, quand, et sur quel objet.
- un **assistant interne** qui répond en langage naturel sur l’activité de l’entreprise, dans la limite de ce que son interlocuteur a le droit de consulter.

Le périmètre couvre dix *user stories*, US-35 à US-44, soit **73 points** d’effort. Le tableau 8.1 en donne la décomposition en tâches.

8.2.2 Sprint backlog

Le tableau 8.1 détaille les tâches du sprint, regroupées par *user story*.

TAB. 8.1 : Sprint backlog — sprint 5

ID	User Story	Tâche	Description	Priorité
US-35	Indicateurs de performance (13 pts)	T5-01	Construire le moteur d’analyse sur les livraisons, les tournées et les échéances	Haute
		T5-02	Résoudre la période demandée et comparer à la précédente	Moyenne
		T5-03	Filtrer côté serveur par zone, chauffeur, statut, motif, ville, source et dépôt	Moyenne
		T5-04	Classer les échecs par motif et retirer les exceptions résolues	Moyenne
US-43	Densité des commandes par zone (5 pts)	T5-05	Agréger les commandes par code postal, avec leur zone de rattachement	Moyenne
US-36	Export d’un rapport de tournée (5 pts)	T5-06	Écrire le socle PDF commun : gabarit, polices, en-tête et pied de page	Moyenne
US-44	Export de la performance d’un chauffeur (3 pts)	T5-07	Produire la fiche individuelle d’un chauffeur sur une période	Basse

ID	User Story	Tâche	Description	Priorité
US-37	Console d'état de l'intégration (13 pts)	T5-08	Mesurer l'état des composants en arrière-plan et garder le résultat	Haute
		T5-09	Séparer ce qui est commun à tous de ce qui appartient à une entreprise	Haute
		T5-10	Conserver une heure d'historique pour suivre l'évolution	Basse
		T5-11	Afficher la console et le journal des synchronisations	Moyenne
US-41	Réglage des seuils de retard (3 pts)	T5-12	Stocker les seuils par entreprise et les appliquer au moteur d'échéances	Moyenne
US-38	Relance d'une synchronisation en échec (5 pts)	T5-13	Rejouer l'événement d'origine, ou le reconstruire depuis la livraison	Haute
		T5-14	Balayer périodiquement les commandes bloquées en attente de résultat	Moyenne
US-42	Rejeu des messages bloqués d'une file (5 pts)	T5-15	Compter les messages bloqués dans chaque file de rebut	Moyenne
		T5-16	Republier un message écarté vers son échange d'origine	Moyenne
US-39	Piste d'audit des données sensibles (8 pts)	T5-17	Alimenter une piste unique depuis tous les services par le bus de messages	Haute
		T5-18	Nommer l'acteur et la cible plutôt que leur identifiant technique	Moyenne
		T5-19	Afficher le journal filtrable par acteur, rôle, entité et période	Moyenne
US-40	Assistant en langage naturel (13 pts)	T5-20	Indexer le corpus documentaire en fragments vectorisés	Haute
		T5-21	Combiner recherche vectorielle et recherche lexicale par fusion de rangs	Haute
		T5-22	Router la question vers la documentation, la donnée en direct ou le moteur d'échéances	Haute
		T5-23	Ancrer la réponse sur les extraits retrouvés et refuser à défaut	Haute
		T5-24	Faire hériter l'assistant des droits de son interlocuteur	Haute

ID	User Story	Tâche	Description	Priorité
		T5-25	Journaliser chaque interaction : question, route, sources, refus, latence	Moyenne

8.3 Concepts et mécanismes utilisés

Ce sprint mobilise trois notions. La première sert à dire l'état du système sans le deviner, les deux autres à répondre à une question sans inventer la réponse.

Le disjoncteur : un service qui ne répond plus ne doit pas être appelé. Chaque appel sortant traverse un compteur qui observe le taux d'échec sur les dernières tentatives. Au-delà d'un seuil, le disjoncteur *s'ouvre* et les appels suivants échouent immédiatement, sans attendre l'expiration du délai réseau. Après une pause, il laisse passer quelques requêtes d'essai, état intermédiaire dit *semi-ouvert*, et se referme si elles aboutissent. Le gain n'est pas seulement de temps : le service en difficulté cesse de recevoir le trafic qui l'achève, et celui qui l'appelle n'attend plus une réponse qui ne viendra pas.

Ces compteurs valent aussi comme signal. La console lit l'état du disjoncteur de chaque dépendance, donc une mesure prise sur le trafic réel, plutôt que de sonder à l'aveugle au moment où on l'ouvre.

La génération augmentée par la recherche : un modèle de langue interrogé seul répond avec ce que ses paramètres contiennent, et il répond toujours, y compris quand il ne sait pas. Le procédé retenu inverse l'ordre : la question sert d'abord à retrouver des extraits dans un corpus maîtrisé, et le modèle ne rédige qu'à partir de ces extraits, qu'il cite. Il ne devient pas une source, il devient un rédacteur.

Cette contrainte a une conséquence que le projet a traitée comme une fonctionnalité et non comme un défaut : quand la recherche ne ramène rien d'exploitable, l'assistant refuse. Un refus explicite est une réponse utile dans un contexte d'exploitation, là où une réponse plausible mais fausse sur l'état d'une livraison coûterait un appel client. La même règle vaut lorsque le modèle est injoignable : l'assistant annonce l'indisponibilité au lieu de répondre de mémoire.

La recherche hybride et la fusion des rangs : deux recherches cohabitent parce qu'elles échouent sur des questions différentes. La recherche **vectorielle** compare le sens : la question et chaque fragment du corpus sont projetés en vecteurs, et la proximité entre eux fait le classement. Elle retrouve une reformulation, mais rate un identifiant exact, qui ne porte aucun sens à projeter. La recherche **lexicale** fait l'inverse : elle retrouve le terme précis et ignore la reformulation.

Reste à combiner deux classements dont les scores ne sont pas comparables. La *fusion réciproque des rangs* ignore les scores : chaque fragment reçoit, pour chaque liste où il apparaît, une part qui décroît avec son rang, et les parts s'additionnent. Un fragment bien placé dans les deux listes passe donc devant un fragment excellent dans une seule.

8.4 Analyse et spécification des besoins

8.4.1 Diagramme de cas d'utilisation du sprint

La figure 8.1 présente les cas d'utilisation du sprint : la consultation des indicateurs, les exports, la console de santé, la reprise après incident, le journal d'audit et l'assistant.

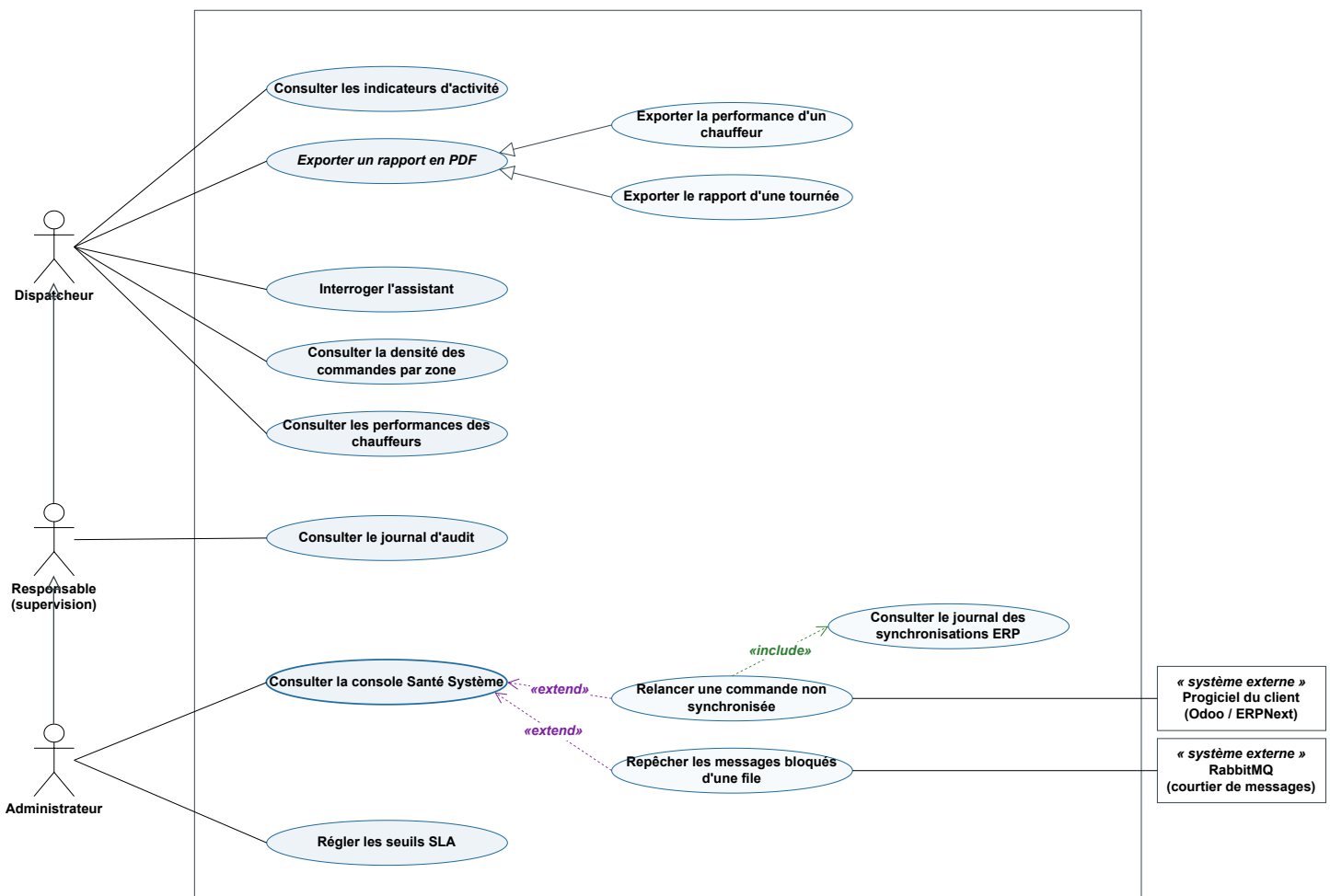


FIG. 8.1 : Diagramme de cas d'utilisation du sprint 5

Les cas se répartissent selon ce que chaque acteur a le droit de faire. Le **dispatcheur** consulte les indicateurs, la densité des commandes par zone et la performance des chauffeurs, exporte un rapport et interroge l'assistant : tout ce qui l'aide à décider de sa journée. Le **responsable** n'a qu'un cas propre, le journal d'audit, ce qui est cohérent avec son rôle défini au chapitre 4 : il relit ce que font les autres sans agir lui-même. L'**administrateur** détient les cas qui touchent

au fonctionnement du système, la console de santé et le réglage des seuils.

Deux systèmes externes apparaissent, et ils ne jouent pas le même rôle que dans les sprints précédents. Le progiciel du client n'est plus une source de commandes mais l'objet d'une surveillance : on regarde s'il répond. Le courtier de messages entre pour la première fois dans un cas d'utilisation, parce qu'un opérateur peut désormais agir sur ses files.

Les relations sont peu nombreuses mais parlantes. L'export d'une performance de chauffeur et l'export d'un rapport de tournée sont deux *spécialisations* du même export PDF, puisqu'ils partagent le gabarit et ne diffèrent que par leur contenu. Consulter le journal des synchronisations est *inclus* dans la relance d'une commande : on ne relance pas à l'aveugle, on relance ce que le journal désigne. Enfin, la relance d'une commande et le repêchage des messages bloqués *étendent* la console de santé, car ces deux gestes ne se présentent que lorsque la console signale un problème.

8.4.2 Description détaillée des cas d'utilisation

Deux cas sont détaillés : celui qui dit à l'administrateur si le système va bien, et celui qui répond à une question posée en français.

TAB. 8.2 : Cas d'utilisation « Consulter la console Santé Système »

Titre	Consulter la console Santé Système
Acteur	Administrateur
Résumé	Savoir, en un écran, si chaque composant du système répond, et si des commandes attendent d'être synchronisées avec le progiciel.
Préconditions	L'administrateur est authentifié et rattaché à une entreprise.
Postconditions	Aucune. Le cas ne modifie rien, il donne à voir.
Scénario nominal	1. L'administrateur ouvre la console. 2. Le serveur renvoie l'instantané déjà calculé en tâche de fond, sans ouvrir de connexion au moment de la demande. 3. L'écran affiche l'état de la base, des files, des disjoncteurs et de la dernière sauvegarde, puis le nombre de commandes en échec de synchronisation. 4. L'administrateur ouvre le journal des synchronisations pour voir quelles commandes ont échoué et pourquoi.
Scénarios alternatifs	2a. Aucun instantané n'existe encore, au tout premier appel qui suit un démarrage : il est calculé à la demande, puis mis en cache. 4a. Des commandes sont en échec : l'administrateur les relance, ce qui relève du cas « Relancer une commande non synchronisée ».

TAB. 8.3 : Cas d'utilisation « Interroger l'assistant »

Titre	Interroger l'assistant
Acteur	Dispatcheur, responsable ou administrateur
Résumé	Poser une question en français sur l'activité de l'entreprise et obtenir une réponse appuyée sur des sources, sans parcourir plusieurs écrans.
Préconditions	L'utilisateur est authentifié. Sa question part avec son propre jeton.
Postconditions	La réponse est affichée avec ses sources. L'interaction est journalisée : question, route empruntée, sources, refus éventuel et durée. Rien n'est modifié dans le système.
Scénario nominal	1. L'utilisateur pose sa question. 2. Le système décide d'où doit venir la réponse : la documentation, une lecture en direct, ou le moteur d'échéances. 3. Il rassemble les éléments correspondants. 4. Le modèle rédige à partir de ces seuls éléments et cite ce sur quoi il s'appuie.
Scénarios alternatifs	2a. La question porte sur un objet précis, une livraison ou une tournée : la lecture en direct est faite avec le jeton de l'utilisateur, donc limitée à ce qu'il aurait pu consulter lui-même. 3a. Rien n'est trouvé dans la documentation : l'assistant le dit et s'arrête. Une réponse inventée sur l'état d'une livraison coûterait un appel au client. 3b. La référence citée n'existe pas : l'assistant répond qu'aucun élément ne correspond, ce qui n'est pas la même chose qu'une panne. 4a. Le modèle est injoignable : l'assistant annonce l'indisponibilité au lieu de répondre de mémoire. 1a. L'utilisateur dépasse trente questions par minute : les suivantes sont refusées jusqu'à la fenêtre suivante. 1b. La question contient une consigne destinée à détourner le modèle de son rôle : elle est comptée dans un indicateur dédié, et le contexte reste déclaré comme donnée dans la consigne système, jamais comme instruction.

8.5 Modélisation conceptuelle

8.5.1 Diagramme de classes

La figure 8.2 montre les classes que ce sprint ajoute ou exploite.

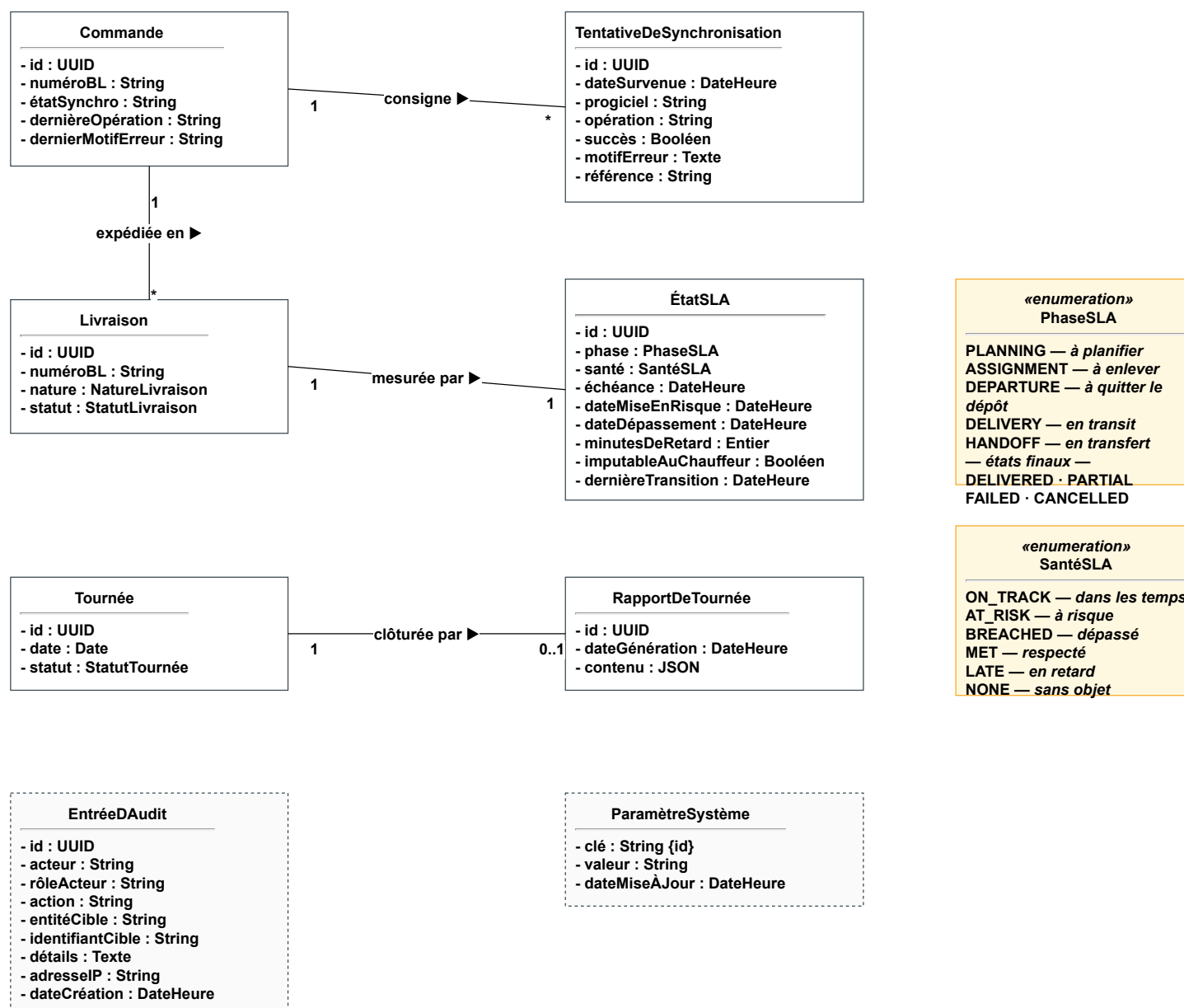


FIG. 8.2 : Diagramme de classes du sprint 5

La partie haute décrit la synchronisation avec le progiciel. La **Commande** garde son état actuel : où en est sa synchronisation, quelle opération a été tentée en dernier, et pourquoi elle a échoué. La **TentativeDeSynchronisation**, elle, garde une ligne par essai.

Les deux sont nécessaires parce qu'ils répondent à deux questions différentes. L'état répond à «combien de commandes sont en échec», que la console affiche. L'historique répond à «qu'est-ce qui s'est passé sur celle-ci», que l'administrateur va lire.

L'**ÉtatSLA** est la classe qui alimente la plupart des indicateurs. Chaque livraison en possède un, qui retient la phase en cours, la santé du délai, l'échéance visée et le retard constaté. Un attribut mérite d'être signalé : le retard est marqué comme imputable ou non au chauffeur.

Deux classes en bas ne sont reliées à rien, et c'est volontaire. L'**EntréeDAudit** enregistre un

8.5.2 Diagramme de séquence

Le diagramme de séquence UML est divisé en quatre phases principales, impliquant les acteurs suivants : Livreur, Application livreur (Flutter), Service Livraison, Livraison, Événement à synchroniser (outbox), Publieur d'événements (toutes les 20 s), RabbitMQ, Adaptateur ERP, et Admin.

Phase 1 — Enregistrement de la livraison : les deux écritures dans une seule transaction
 Valide la livraison (photo du colis + photo du bon signe)
 - Le livreur transmet la preuve de livraison à l'application livreur.
 - L'application livreur transmet la preuve de livraison au service livraison.
 - Le service livraison enregistre le statut « Livrée » et crée l'événement à synchroniser (en attente).
 - L'événement à synchroniser est enregistré dans l'outbox.
 - Une seule transaction est utilisée : les deux écritures réussissent, ou aucune.
 - L'application livreur reçoit une confirmation immédiate.
 - L'écran suivant est affiché : le livreur repart (il n'attend jamais l'ERP).

Phase 2 — Publication du message (asynchrone)
 - Une boucle toutes les 20 secondes, le publieur d'événements réclame un lot d'événements en attente (verrou, 10 max).
 - Les événements sont marqués « en cours de traitement ».
 - Le publieur publie la commande de synchronisation sur RabbitMQ.
 - RabbitMQ marque l'événement « publié » (le travail de l'outbox est terminé).
 - RabbitMQ remet en attente, compteur +1, délai doublé à chaque essai (jusqu'à 4 h).
 - Note : Au-delà de 16 essais (≈ 22 h), l'événement est marqué en échec définitif : visible et rejouable, jamais perdu.

Phase 3 — Application dans l'ERP du client
 - L'adaptateur ERP reçoit la commande de synchronisation.
 - L'adaptateur ERP lit l'état actuel du transfert.
 - L'adaptateur ERP valide le transfert de stock.
 - L'adaptateur ERP publie le résultat (succès).
 - Le service livraison marque la livraison « synchronisée ».
 - Note : Avant chaque tentative, l'adaptateur ERP lit l'état dans l'ERP. Un transfert déjà valide n'est jamais révalidé : le stock ne sort pas deux fois, même si une réponse s'est perdue.
 - Une boucle de 0 à 4 essais est exécutée : si aucune réponse, l'adaptateur ERP rélit l'état, puis tente la validation.
 - L'adaptateur ERP écarte le message vers le file de rebut.
 - Le service livraison publie le résultat (échec).
 - Le service livraison marque la livraison « échec de synchronisation ».
 - Note : Notifie l'opérateur — l'échec devient visible, il n'est jamais silencieux.

Phase 4 — Reprise manuelle
 - Note : L'opérateur relance une synchronisation en échec.
 - L'opérateur clique sur « Resynchroniser ».
 - Le service livraison remet l'événement en attente.
 - Note : Reprise confirmée — la boucle de la phase 2 recommence.

FIG. 8.3 : Séquence de synchronisation avec le progiciel et reprise après échec

Le diagramme se lit comme une suite de filets tendus sous la même écriture. Un publieur reprend les ordres de synchronisation **toutes les vingt secondes**, par lots de dix, en verrouillant ce qu'il prend pour que deux instances ne traitent pas le même. Si le courtier ne répond pas, l'ordre retourne en attente avec un délai qui double à chaque essai, **jusqu'à quinze fois**, soit une vingtaine d'heures : de quoi traverser une panne de nuit. Au-delà, il est marqué en échec définitif, ce qui le rend visible et rejouable plutôt que perdu. Si c'est le progiciel qui ne répond

pas, l'adaptateur réessaie cinq fois en doublant l'attente, puis écarte le message vers la file de rebut.

Une note du diagramme porte tout le reste : avant chaque tentative, l'adaptateur relit l'état du transfert dans le progiciel, et ne revalide jamais ce qui l'est déjà. C'est la propriété qui rend la quatrième phase sans danger. L'administrateur clique sur « Resynchroniser », l'ordre retourne en attente, la boucle du publieur le reprend, et s'il clique deux fois, le stock ne sort pas deux fois. Sans cette relecture, offrir un bouton de relance aurait été offrir un moyen de se tromper.

8.5.3 Le chemin d'une question

La figure 8.4 suit une question de bout en bout. Elle distingue deux temps : l'indexation du corpus, faite hors ligne à chaque mise à jour de la documentation, et le trajet de la question elle-même.

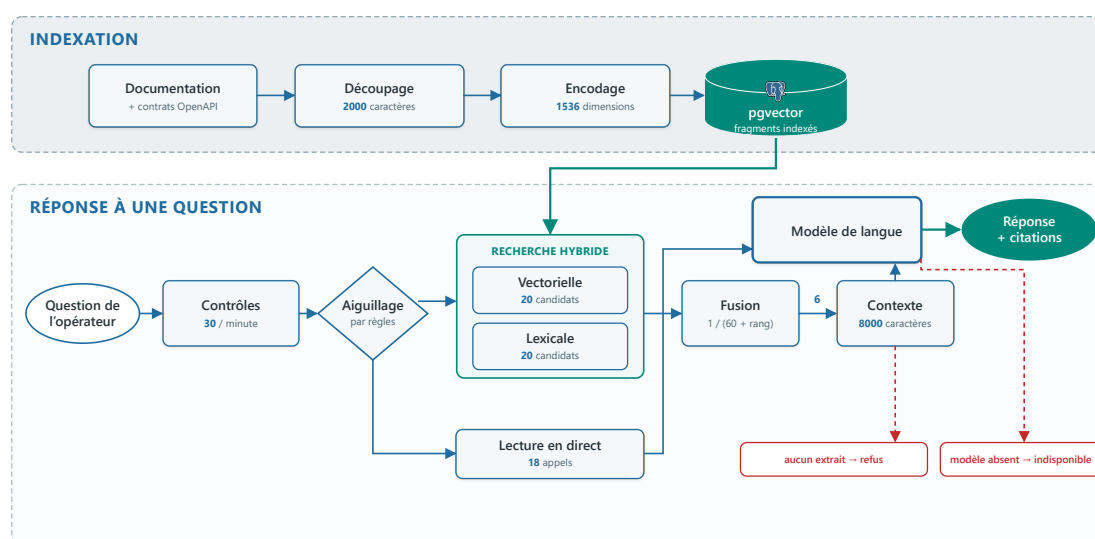


FIG. 8.4 : Le chemin d'une question, de l'indexation à la réponse citée

Trois décisions se lisent mal sur un schéma et méritent d'être dites.

L'aiguillage se fait par règles, sans modèle. C'est la pièce qui empêche l'assistant de devenir un agent conversationnel branché sur la base. Une question sur le respect d'un délai part vers le moteur d'échéances, dont la réponse est calculée et non rédigée. Une question sur l'état courant d'une livraison part vers une lecture en direct. Tout le reste part vers la documentation. Le routeur reconnaît au passage les références telles que les progiciels les impriment, dont les formats diffèrent d'un éditeur à l'autre.

L'assistant n'a pas de droits propres. Les dix-huit lectures en direct sont toutes en consultation seule, et chacune part avec le jeton de celui qui pose la question. Deux utilisateurs

de rôles différents n’obtiennent donc pas la même réponse à la même question, et aucun ne peut lire ce qu’il n’aurait pas pu consulter lui-même.

Le refus est une issue prévue. Quand la fusion ne retient aucun fragment, l’assistant refuse sans même appeler le modèle. Quand aucun fournisseur ne répond, il annonce l’indisponibilité plutôt que de rédiger sans matière. Ces deux sorties sont des résultats, pas des pannes : une réponse plausible mais fausse sur l’état d’une livraison coûterait un appel au client.

8.6 Réalisation

La figure 8.5 montre le tableau de bord. Quatre indicateurs ouvrent l’écran avec leur courbe et leur écart sur la période précédente, ici un respect des délais de 59 pour cent. En dessous viennent ce qui reste en souffrance, les retours, la décomposition du cycle en étapes, et une colonne d’activité en direct sur le bord droit.

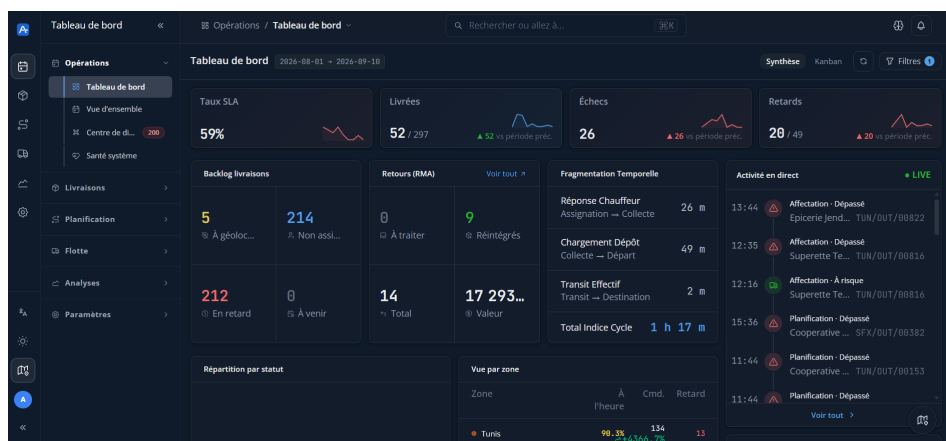


FIG. 8.5 : Tableau de bord : indicateurs de la période et comparaison à la précédente

La figure 8.6 présente la console Santé Système. La première ligne annonce deux points d’attention, l’intégration Odoo et la sauvegarde, dont l’âge est affiché plutôt que son seul succès. La section « À traiter » liste deux commandes jamais arrivées dans le progiciel, avec leur motif et leur bouton de relance.

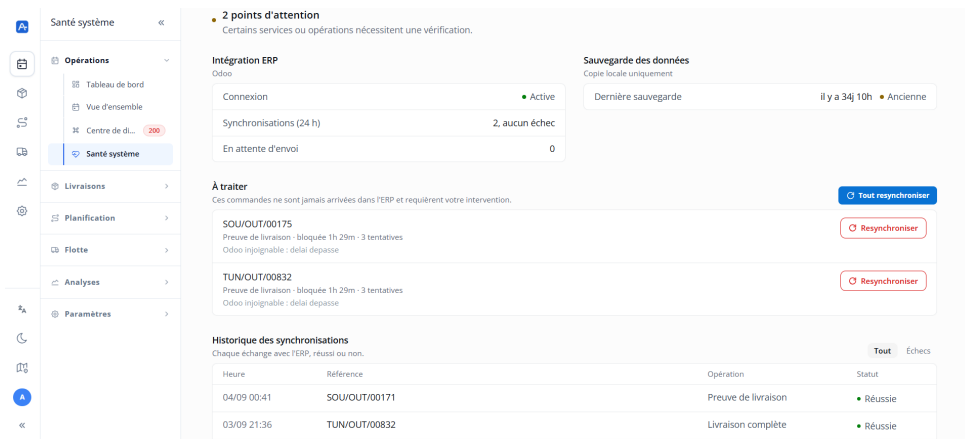


FIG. 8.6 : Console Santé Système : points d'attention, commandes à reprendre et journal des synchronisations

Un volet «Détails techniques», replié par défaut, complète la console : l'état de chaque composant, les disjoncteurs avec leur taux d'échec, et les files de rebut.

La figure 8.7 montre l'assistant, ouvert en panneau latéral par-dessus l'écran en cours. La question porte sur les dépôts actifs, et la réponse est précédée de la mention *données en direct*, qui signale une lecture de l'API plutôt qu'une recherche dans la documentation. Chaque dépôt est rendu avec son adresse, son code d'entrepôt et ses coordonnées.

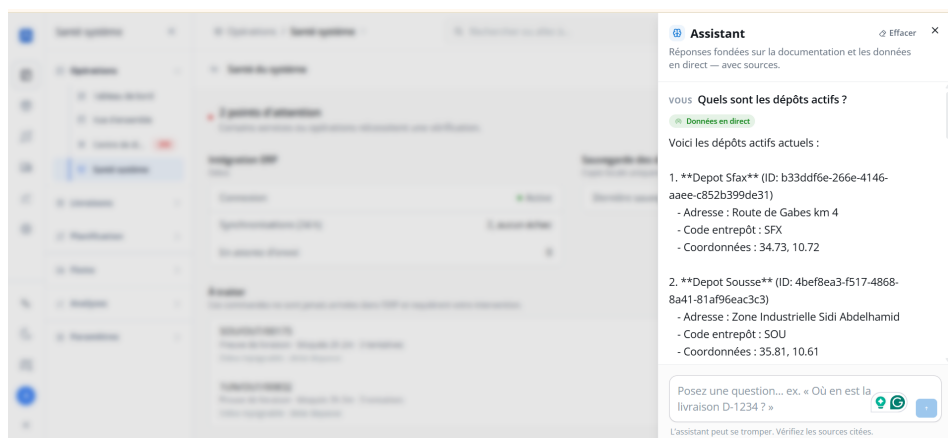


FIG. 8.7 : Assistant : une réponse tirée des données en direct

8.7 Tests et validation

8.7.1 Scénarios de test

Deux scénarios sont retenus, ceux dont l'échec se paie : l'un en stock sorti deux fois, l'autre en données montrées à quelqu'un qui n'y a pas droit.

TAB. 8.4 : TC-S5-01 — Une commande en échec repart sans doubler le mouvement de stock

Cas de test	TC-S5-01
Description	Une relance manuelle doit rester sans effet si le progiciel a déjà enregistré le mouvement.
Précondition	Une commande en échec de synchronisation, dont le transfert a en réalité été validé côté progiciel.
Scénario	1. Relancer la commande depuis la console. 2. Observer le progiciel. 3. Relancer une seconde fois.
Résultat attendu	La commande passe en synchronisée. Le stock ne sort qu'une fois, la seconde relance étant reconnue sans effet.

TAB. 8.5 : TC-S5-02 — L'assistant ne dépasse pas les droits de celui qui l'interroge

Cas de test	TC-S5-02
Description	La même question, posée par deux comptes différents, ne doit pas donner la même réponse.
Précondition	Deux utilisateurs de la même entreprise, l'un dispatcheur, l'autre responsable, et un troisième d'une autre entreprise.
Scénario	1. Poser une question sur les espèces en circulation avec chacun des deux premiers comptes. 2. Poser la même question avec le compte de l'autre entreprise. 3. Demander l'état d'une livraison dont la référence n'existe pas.
Résultat attendu	Chaque réponse se limite à ce que le rôle autorise. L'autre entreprise ne voit rien des données de la première. La référence inconnue donne «aucun élément ne correspond», et non un message de panne.

8.7.2 Tests automatisés

Ce sprint met en jeu deux mécanismes qui décident sans qu'un humain contrôle la décision : la reprise d'une synchronisation et la réponse de l'assistant. Ce sont eux que les tests visent, parce qu'une erreur y est silencieuse. Une relance mal conduite écrit une seconde fois dans le progiciel du client. Une réponse inventée circule comme si elle était vraie.

Le sprint compte **82 tests**, répartis entre le service de livraison et le service assistant. Le tableau 8.6 les détaille.

TAB. 8.6 : Tests automatisés du sprint 5

Classe de test	Cas	Ce qu'elle garantit
Service de livraison — 3 classes, 28 cas		

Classe de test	Cas	Ce qu'elle garantit
ErpResyncServiceTest	10	Une relance rejoue la charge d'origine, ou refuse quand elle devrait deviner
ErpResyncFreezeTest	8	Une commande déjà traitée par le progiciel n'est pas rejouée
ExceptionRetirementTest	10	Une exception résolue disparaît des indicateurs
Service assistant — 8 classes, 54 cas		
IntentRouterTest	9	Chaque question part vers la bonne source
ReferenceResolverTest	9	Les références des deux progiciels sont reconnues malgré leurs formats
InjectionRegressionTest	10	Les consignes cherchant à détourner le modèle sont signalées, et les questions ordinaires ne le sont pas
AnswerServiceTest	8	Sans élément, l'assistant refuse, sans modèle, il annonce l'indisponibilité
LexicalToolSelectorTest	6	Une question sur une donnée en direct choisit la bonne lecture
AnonymousIsNotAuthenticatedTest	5	Aucune réponse n'est servie sans authentification
EvalMetricsTest	4	Les mesures de qualité de la recherche sont justes
SecurityGuardsTest	3	Les garde-fous restent actifs sur les chemins détournés
Total	82	11 classes

ErpResyncFreezeTest mérite un mot, car il vérifie ce que la relance *ne fait pas* : le cas où le progiciel a déjà enregistré le mouvement est le seul dont l'échec se traduirait par du stock sorti deux fois.

Trois suites d'intégration s'y ajoutent, sur une vraie base : l'isolation entre entreprises, l'autorisation des accès, et l'évaluation de la qualité de la recherche, à laquelle la sous-section suivante est consacrée.

8.7.3 Qualité de la recherche documentaire

Les tests précédents disent que l'assistant se comporte comme prévu. Ils ne disent pas s'il retrouve le bon document, et c'est pourtant de là que dépend la valeur de sa réponse. Un assistant qui ne retrouve rien refuse poliment, ce qui est correct sans être utile.

Cette qualité se mesure. Un jeu de trente-neuf questions de référence a été constitué, couvrant

le métier, l’architecture, l’API, la sécurité et l’intégration au progiciel. Pour chacune, la source attendue est écrite à l’avance. Trente et une portent sur la documentation et sont donc notées, les autres relevant de la donnée en direct ou du refus. Le corpus indexé pour la mesure compte **44 documents** découpés en **220 fragments**.

Trois mesures usuelles en recherche d’information sont calculées, sur la seule recherche, avant toute génération.

TAB. 8.7 : Qualité de la recherche sur les 31 questions de référence

Mesure	Ce qu’elle dit	Seuil	Obtenu
Rappel à 5	Le bon document figure parmi les cinq premiers résultats	0,80	0,968
Rang réciproque moyen	À quelle position il apparaît, la première valant 1	0,65	0,890
Gain cumulé à 10	Qualité du classement entier, les premières places pesant plus lourd	—	0,877

Le rappel de 0,968 signifie que sur trente et une questions, trente ramènent la bonne source dans les cinq premiers résultats. Le rang réciproque de 0,890 ajoute que cette source est le plus souvent en tête, ce qui compte parce que le contexte transmis au modèle est plafonné.

Cette évaluation appelle un modèle d’encodage réel. Elle est donc déclenchée à la demande plutôt qu’à chaque *commit*, pour deux raisons : elle consomme le quota du fournisseur, et son résultat n’est pas strictement reproductible. Les seuils, eux, sont écrits dans le test et le font échouer s’ils ne sont pas atteints.

8.8 Conclusion

Ce sprint n’a rien ajouté au trajet d’une livraison. Il a rendu visible ce que les quatre précédents avaient enregistré sans jamais le montrer.

Le responsable dispose d’indicateurs calculés sur des faits, filtrables selon sept critères appliqués avant l’agrégation, comparés à la période précédente, et exportables en PDF. L’administrateur dispose d’une console qui distingue une panne du progiciel d’une panne du système, d’un journal qui dit quelle commande a échoué et pourquoi, et de deux gestes pour repartir sans appeler personne. Le journal d’audit, enfin, est devenu une table unique alimentée par tous les services, ce qui était la condition pour qu’il soit complet.

L’assistant est l’apport le plus visible et le plus facile à mal comprendre. Il ne sait rien de lui-même : il ne répond qu’à partir d’extraits retrouvés ou de lectures faites avec le jeton de son interlocuteur, il cite ce sur quoi il s’appuie, et il refuse quand il n’a pas de matière. Ce refus

n'est pas une limite qu'on aurait subie, c'est ce qui le rend utilisable en exploitation : une réponse plausible mais fausse sur l'état d'une livraison se paie en appel client.

Un point mérite d'être dit franchement. La console de santé montre l'état du système à l'instant où on la regarde, et garde environ une heure d'historique en mémoire. C'est suffisant pour réagir, pas pour enquêter sur un incident de la semaine passée. Une supervision durable, avec conservation longue et alertes, reste à mettre en place le jour où la plateforme servira plusieurs entreprises en production.

Le système est maintenant complet et observable. Le chapitre suivant en mesure la qualité : la couverture des tests, la chaîne d'intégration continue, et le déploiement.

9.1 Introduction

Les cinq chapitres précédents ont montré ce que chaque sprint a livré. Celui-ci regarde l'ensemble sous un autre angle : comment on sait que le produit fonctionne, et comment il arrive sur une machine qui n'est pas celle du développeur.

Les deux questions n'en font qu'une. Une chaîne d'intégration qui ne teste rien laisse passer des régressions. Une suite de tests que personne n'exécute automatiquement finit par mentir. Ce chapitre décrit donc le dispositif complet, des tests unitaires jusqu'aux conteneurs qui tournent sur le serveur.

9.2 La pyramide de tests

Le dépôt compte **783 tests automatisés**. Leur répartition suit la forme attendue : une base large de tests unitaires et une couche d'intégration plus mince.

TAB. 9.1 : Répartition des tests automatisés

Couche	Tests	Ce qu'elle vérifie
Unitaires Java	545	Les règles métier, sans base ni réseau
Intégration Java	60	Le comportement contre une vraie base et un vrai progiciel
React (Vitest)	147	Les composants et la logique de l'interface d'administration
Flutter (Dart)	31	L'application du livreur, dont la file d'attente hors ligne
Total	783	105 fichiers de test

Les deux services les plus fournis sont ceux qui portent le métier : 254 tests pour la gestion des livraisons, 225 pour le connecteur ERP. La passerelle en compte 23, ce qui correspond à son rôle, router les requêtes et appliquer la politique d'autorisation.

Un seul test est désactivé par défaut, l'évaluation de la recherche documentaire décrite au

chapitre 8. Elle appelle un service externe facturé à l'usage et n'a donc pas sa place dans une exécution déclenchée à chaque *commit*. Elle se lance à la demande.

La figure 9.1 montre l'agrégation des rapports produite par la chaîne.

Summary						
617 tests 0 failures 0 errors 100% success rate 4.20s						
Jobs						
Job	Execution time	Failed	Errors	Skipped	Passed	Total
unit-test-backend: [ApiGateway]	44.00ms	0	0	0	23	23
unit-test-backend: [AppBackend]	1.62s	0	0	0	31	31
unit-test-backend: [DeliveryMicroservice]	2.00ms	0	0	0	244	244
unit-test-backend: [DriverService]	156.00ms	0	0	0	19	19
unit-test-backend: [ErpAdapterService]	178.00ms	0	0	0	219	219
unit-test-backend: [AssistantService]	74.00ms	0	0	0	54	54
integration-test-backend-delivery	1.94s	0	0	0	21	21
integration-test-backend-assistant	190.00ms	0	0	1	5	6

FIG. 9.1 : Rapport de tests agrégé par la chaîne d'intégration

9.3 La chaîne d'intégration continue

La chaîne enchaîne cinq étapes, ordonnées par coût croissant : compilation, tests unitaires, analyse de sécurité, tests d'intégration, construction des images. Rien ne démarre si l'étape précédente a échoué.

L'ordre n'est pas décoratif. Une erreur de compilation se voit presque tout de suite, alors que le job du connecteur ERP, qui démarre deux instances du progiciel et attend qu'elles soient réellement utilisables, met trois minutes et dix-neuf secondes. Placer les étapes lentes en dernier évite d'attendre pour apprendre qu'une accolade manquait.

L'étape de construction ne produit jamais de *tag latest*. Chaque image porte l'empreinte du *commit*, le nom de la branche, et le cas échéant le *tag* de version. Un déploiement épingle une empreinte : deux machines annoncées sur la même version exécutent alors le même code. Un *tag* dont le sens change en silence est la façon la plus simple de perdre cette garantie.

La figure 9.2 montre une exécution complète.

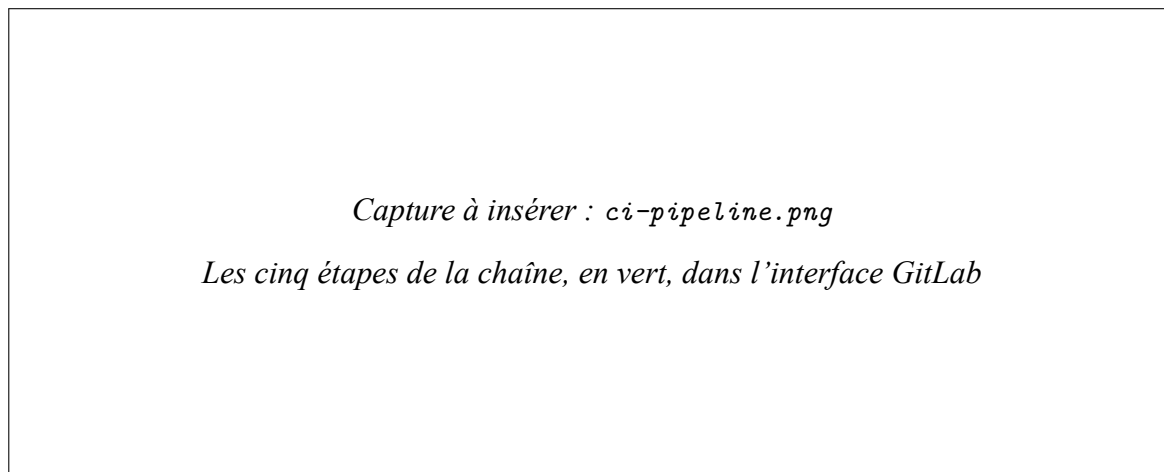


FIG. 9.2 : Les cinq étapes de la chaîne d'intégration

9.4 Le connecteur ERP éprouvé sur deux versions

La suite d'intégration du connecteur s'exécute contre Odoo 16 et Odoo 19 en parallèle, deux instances lancées comme services du job sur un PostgreSQL partagé. Tester les deux bornes de l'intervalle supporté couvre par construction les versions intermédiaires.

Le point délicat n'était pas de lancer le progiciel, mais de savoir quand il est prêt. Son port HTTP répond en quelques secondes, bien avant que les modules de vente et de stock soient installés, et un test lancé à ce moment échoue sur l'authentification plutôt que sur ce qu'il est censé vérifier. L'attente porte donc sur une requête métier réelle, réussie plusieurs fois de suite : un état annoncé comme prêt ne vaut pas un service capable de répondre.

9.5 L'analyse des dépendances

Un analyseur parcourt les dépendances et les fichiers de configuration à chaque exécution, et signale les vulnérabilités de niveau élevé et critique. Le job est déclaré non bloquant.

Ce choix se discute, et c'est volontaire. Une faille héritée d'une dépendance transitive n'est pas toujours corrigible dans la journée. Faire échouer toute la chaîne pour cette raison conduit vite à désactiver le contrôle, et l'information disparaît alors complètement. La rendre visible sans la rendre bloquante la garde sous les yeux de l'équipe.

9.6 Le déploiement

Section à rédiger lundi, après le déploiement sur le serveur.

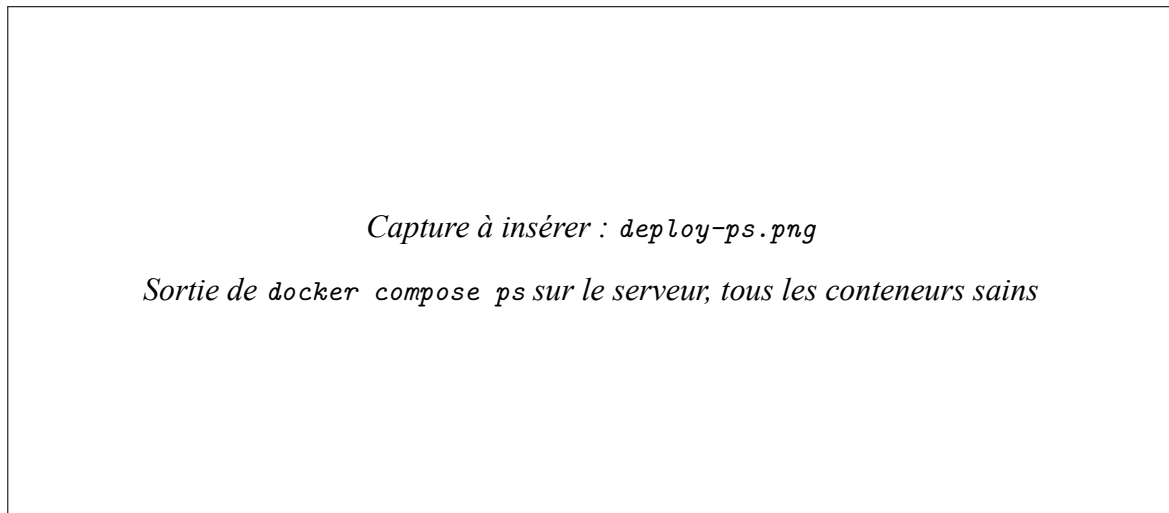


FIG. 9.3 : État des conteneurs sur le serveur de déploiement

9.7 Conclusion

Le dispositif décrit ici tient en une phrase : rien n'arrive sur le serveur qui n'ait traversé les cinq étapes de la chaîne, et l'artefact déployé est celui qu'elle a construit, jamais une recompilation faite sur place.

Reste ce qui n'est pas couvert. Aucun test ne pilote un navigateur, donc les parcours de bout en bout de l'interface restent vérifiés à la main. Le déploiement lui-même n'est pas encore déclenché par la chaîne : les images sont publiées automatiquement, leur mise en service demande une intervention. C'est le premier chantier à ouvrir après la soutenance.

Conclusion générale et perspectives

Le présent projet a permis la conception et le développement complet d'un système intelligent de suivi de livraison, destiné à couvrir le dernier kilomètre, là où le progiciel de gestion perd la trace de la commande. Réalisé au sein de l'entreprise ASM selon une démarche Agile Scrum menée en six sprints, ce travail a abouti à une plateforme qui suit une commande depuis son import dans l'ERP jusqu'à la preuve de livraison qui y retourne.

Sur le plan technique, la solution repose sur une architecture en microservices Spring Boot placés derrière une passerelle unique, un modèle relationnel PostgreSQL dont le schéma est versionné par migrations, RabbitMQ pour les échanges asynchrones et MinIO pour le stockage des preuves. L'identité est déléguée à Keycloak, le suivi en direct est diffusé par WebSocket vers une console d'administration React, et le terrain est équipé d'une application chauffeur Flutter, l'ensemble étant déployé en conteneurs orchestrés par Docker Compose.

L'apport principal réside dans l'intégration agnostique au progiciel de gestion. Quatre ports définissent ce dont le métier a besoin, deux connecteurs les réalisent aujourd'hui pour Odoo et ERPNext, et un mapping guidé appuyé sur un contrat canonique refuse une correspondance incohérente avant qu'elle ne produise des données inexploitable. Un moteur de capacités interroge ce que l'instance sait réellement faire plutôt que son numéro de version, ce qui permet d'absorber les écarts entre éditions sans réécrire le connecteur. La validité de ce découpage a été éprouvée en cours de route : ERPNext n'était pas prévu au moment où les ports ont été définis, et son intégration n'a demandé aucune modification du cœur métier.

Le second apport tient au mode hors ligne de l'application chauffeur. Une écriture faite sans réseau est conservée sur le téléphone, affichée immédiatement, puis rejouée au retour de la couverture avec la même clé et le même horodatage, le serveur reconnaissant une opération déjà traitée. Ce qui échoue malgré tout reste visible dans un centre de synchronisation où le chauffeur décide lui-même de relancer ou d'abandonner.

Par ailleurs, la plateforme intègre les mécanismes attendus d'une exploitation servant plusieurs entreprises : isolation des données par schéma dédié, contrôle d'accès fondé sur dix-huit permissions réparties sur trois rôles et appliqué à l'identique dans chaque service, journal d'audit unifié couvrant les actions sensibles, et assistant interne qui ne répond qu'à partir de sources qu'il cite. La qualité est éprouvée par 783 tests automatisés et une chaîne d'intégration

en cinq étapes, la suite du connecteur ERP s'exécutant en parallèle contre Odoo 16 et Odoo 19.

Ce projet ouvre plusieurs perspectives d'évolution. Aucun test ne pilote aujourd'hui un navigateur, si bien que les parcours complets de la console d'administration restent vérifiés à la main et qu'une régression purement visuelle traverserait la chaîne sans être vue. Une supervision durable, avec conservation longue des métriques et alertes, remplacerait l'heure d'historique que conserve la console de santé. Le destinataire, qui dispose d'un lien de suivi mais que rien n'avertit, gagnerait à être notifié au départ de la tournée. Une réoptimisation en cours de journée, figeant les arrêts déjà effectués pour ne réordonner que ceux qui restent, absorberait un retard ou une commande urgente. Enfin, le connecteur DUX, dont les adaptateurs sont déjà déclarés face aux quatre ports mais restent en attente de la documentation de l'API de l'éditeur, viendra compléter la couverture des progiciels utilisés chez les clients d'ASM.

Ce travail a permis de consolider des compétences en architecture distribuée, en intégration de systèmes hétérogènes et en conception de modes dégradés, dans un contexte où la coupure réseau et l'indisponibilité d'un progiciel tiers relèvent de l'ordinaire plutôt que de l'exception.

Bibliographie

- [SCH 20] Ken SCHWABER et Jeff SUTHERLAND. *The 2020 Scrum Guide*. Creative Commons Attribution Share-Alike 4.0. Nov. 2020. URL : <https://scrumguides.org/scrum-guide.html> (visité le 31/08/2026).
- [SAN 96] Ravi S. SANDHU et al. «Role-Based Access Control Models». In : *IEEE Computer* 29.2 (1996), p. 38-47.
- [COC 05] Alistair COCKBURN. *The Hexagonal (Ports & Adapters) Architecture*. Rapp. tech. HaT Technical Report 2005.02. Humans et Technology, sept. 2005. URL : <https://alistair.cockburn.us/hexagonal-architecture> (visité le 31/08/2026).
- [RIC 18] Chris RICHARDSON. *Microservices Patterns : With Examples in Java*. Shelter Island, NY : Manning Publications, 2018. ISBN : 9781617294549.
- [TOT 14] Paolo TOTI et Daniele VIGO, éd. *Vehicle Routing : Problems, Methods, and Applications*. 2^e éd. MOS-SIAM Series on Optimization 18. Philadelphia, PA : Society for Industrial et Applied Mathematics, 2014.
- [VET 11] Christian VETTER et Dennis LUXEN. «Real-Time Routing with OpenStreetMap Data». In : *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*. Association for Computing Machinery, 2011, p. 513-516. DOI : 10.1145/2093973.2094062.
- [KIS 92] James J. KISTLER et Mahadev SATYANARAYANAN. «Disconnected Operation in the Coda File System». In : *ACM Transactions on Computer Systems* 10.1 (1992), p. 3-25. DOI : 10.1145/146941.146942.

Webographie

- [ASM 26] ASM TUNISIE. *À propos*. 2026. URL : <https://asm-tunisie.com/a-propos/> (visité le 18/08/2026).
- [ONF 26a] ONFLEET. *Onfleet – Last Mile Delivery Software*. 2026. URL : <https://onfleet.com/> (visité le 18/08/2026).
- [ONF 26b] ONFLEET SUPPORT CENTER. *Driver App Settings*. 2026. URL : <https://support.onfleet.com/hc/en-us/articles/10228814951060-Driver-App-Settings> (visité le 18/08/2026).
- [BRI 26a] BRINGG. *Bringg – Delivery and Logistics Orchestration Platform*. 2026. URL : <https://www.bringg.com/> (visité le 18/08/2026).
- [BRI 26b] BRINGG HELP CENTER. *Stay Connected with the Bringg Driver App*. 2026. URL : <https://help.bringg.com/docs/stay-connected-with-the-bringg-driver-app> (visité le 18/08/2026).
- [BRI 26c] BRINGG HELP CENTER. *Set Up Driver Actions in Order Flows*. 2026. URL : <https://help.bringg.com/docs/set-up-driver-actions-in-order-flows> (visité le 18/08/2026).
- [ONF 26c] ONFLEET. *Onfleet Product Update Q1 2026*. 2026. URL : <https://onfleet.com/blog/onfleet-product-update-q1-2026/> (visité le 26/08/2026).
- [BRI 26d] BRINGG. *Predictive Time on Site*. 2026. URL : <https://www.bringg.com/> (visité le 26/08/2026).

الخلاصة : تقدّم هذه المذكرة منصّة لتتبع عمليات التوصيل في الكيلومتر الأخير، قابلة للاستعمال من قبل عدّة مؤسسات ومندمجة مع أنظمة تخطيط مواردها. تغطّي المنصّة استيراد الطلبات من نظام تخطيط الموارد، وتخطيط الجولات، والتنفيذ الميداني عبر تطبيق جوال يعمل دون اتصال بالإنترنت، وإثبات التسليم، وإدارة الإرجاع. تعتمد البنية على الخدمات المصغرة مع عزل البيانات حسب المخطط، ومصادقة مفوّضة عبر OAuth2/OIDC، ومُهايّد لأنظمة تخطيط الموارد تمّ التحقق منه على Odoo وERPNext. تمّت النمذجة باستخدام UML وفق منهجية Scrum.

Résumé : Ce mémoire présente une plateforme de suivi de livraison du dernier kilomètre, exploitable par plusieurs entreprises clientes et intégrée à leurs progiciels de gestion. Elle couvre l'import des commandes depuis l'ERP, la planification des tournées, l'exécution sur le terrain via une application mobile fonctionnant hors connexion, la preuve de livraison et la gestion des retours. L'architecture, orientée microservices, isole les données par schéma, délègue l'authentification en OAuth2/OIDC et intègre un adaptateur ERP agnostique validé sur Odoo et ERPNext. La modélisation a été réalisée en UML et le développement conduit selon la méthode Scrum.

Abstract : This thesis presents a last-mile delivery tracking platform, usable by several client companies and integrated with their enterprise resource planning systems. It covers order import from the ERP, route planning, field execution through an offline-capable mobile application, proof of delivery and returns management. The microservice-oriented architecture isolates data by schema, delegates authentication to OAuth2/OIDC, and integrates an ERP-agnostic adapter validated against both Odoo and ERPNext. Conceptual modelling was carried out using UML and development followed the Scrum agile method.

الكلمات المفاتيح : التوصيل في الكيلومتر الأخير، التتبع الآني، تعدّد المستأجرين، تكامل أنظمة تخطيط الموارد، الخدمات المصغرة، العمل دون اتصال، UML.

Mots clés : livraison du dernier kilomètre, suivi en temps réel, multi-tenance, intégration ERP, Odoo, ERPNext, microservices, mode hors ligne, UML.

Keywords : last-mile delivery, real-time tracking, multi-tenancy, ERP integration, Odoo, ERPNext, microservices, offline mode, UML.